

University of Milano-Bicocca

Department of Informatics, Systems and Communication

Master's Degree Program in Computer Science

ThemisRAG: Knowledge Graph-Grounded Legal Question Answering with LLMs over Statutes and Case Law

Supervisor: Prof. Matteo Luigi Palmonari

Co-supervisor: Dr. Renzo Arturo Alva Principe

Master's degree thesis by:
Nicolò Nicholas Zagami
ID Number 829888

Academic Year 2025-2026

Abstract

Imagine a lawyer who asks an AI assistant what a European regulation really means for a client. The assistant answers at once, confidently and fluently. It may also be wrong: it may cite a provision that doesn't apply, or ignore the judgment in which the Court of Justice settled the question years ago. Large Language Models have taught us to ask instead of search, but in law a convincing answer is not enough. Rules are written in general terms on purpose, and their meaning only becomes clear when the courts interpret them. So a trustworthy legal assistant has to do what a lawyer does: find the rule, then the judgments that interpret it, then the exact passage where the Court explains its reasoning.

This thesis starts from that observation. Retrieval-Augmented Generation grounds a model's answers in external documents, but it usually treats those documents as isolated fragments, chosen because they resemble the question. In law the passage that answers a question often uses entirely different words from the question itself, and the connection between a statute and the judgment that clarifies it is a link, not a similarity. To capture those links, we built a knowledge graph of EU legislation and CJEU case law that reads the structure of each act and judgment directly from the official sources instead of guessing it with a language model. Every act keeps its chapters, articles and recitals; every judgment keeps its reasoning and its operative part, and is connected to the provisions it interprets. We also discussed, from a theoretical perspective, how the graph could represent the evolution of the law over time and its multilingual nature.

Building on this graph, we developed ThemisRAG, a legal question answering system based on a graph-based architecture and a hybrid retrieval pipeline. We chose this combination because it fits the nature of legal texts. Hybrid retrieval combines lexical and semantic search, so it can catch both the precise terminology of the law and the meaning behind a question. The graph lets the system follow the connections between provisions and the judgments that interpret them. Lastly, we compared ThemisRAG with well-known general-purpose graph-based RAG systems, and we analysed the results to understand what worked and what did not; we conclude by looking ahead to possible directions for future work.

Contents

Abstract . . . . . **I**

1 Introduction **1**
1.1 Contributions 2
1.2 Thesis Structure 3

2 Preliminaries **5**
2.1 Text Representation and Information Retrieval 5
2.2 Transformer and Large Language Models 7
2.3 Retrieval-Augmented Generation 9
2.4 Knowledge Graphs 11
2.5 Prompting Techniques 12
2.6 LLM-as-a-Judge and RAGAS 13

3 State of The Art **14**
3.1 Knowledge Graph Construction 14
3.1.1 Construction paradigms 14
3.1.2 Deterministic and rule-based construction 14
3.1.3 LLM-driven construction 15
3.1.4 Evaluating the quality of constructed knowledge graphs 15
3.1.5 Case study: knowledge graph construction in the legal domain 15
3.2 Graph based RAG 17
3.2.1 Microsoft GraphRAG 18
3.2.2 HippoRAG 19
3.2.3 LightRAG 23
3.2.4 PathRAG 26
3.2.5 HyPA-RAG 29

4 ThemisRAG Data Model Construction **33**
4.1 Knowledge Extraction 33

4.1.1 Act . . . 34
4.1.2 Case Law (CJEU) . . . 35
4.1.3 Topics . . . 37
4.2 Data Model . . . 39
4.2.1 Modelling Amendments and Repeals . . . 42
4.2.2 Multilingualism . . . 43

5 ThemisRAG: Proposed Approach . . . 45
5.1 LLM Intent Classification . . . 47
5.2 Definitional Intent Flow . . . 49
5.2.1 Hypothetical Document Embeddings (HyDE) . . . 49
5.2.2 Hybrid Retrieval . . . 50
5.2.3 Reciprocal Rank Fusion (RRF) . . . 51
5.2.4 Cross Encoder . . . 52
5.2.5 Model Choice . . . 52
5.3 Context Wrapping . . . 53
5.3.1 Curation Module . . . 53
5.4 Interpretative Intent Flow . . . 54
5.4.1 Hybrid Retrieval . . . 54
5.4.2 Neighbour Expansion . . . 56
5.4.3 Operative Part Guarantee . . . 58
5.4.4 Citation Bridge . . . 59
5.5 Generator . . . 61

6 Knowledge Graph Construction Evaluation . . . 63
6.1 Dataset . . . 63
6.2 Methodology and Metrics . . . 64
6.3 Results and Analysis . . . 65

7 ThemisRAG Evaluation . . . 68
7.1 Experimental Setting . . . 68
7.1.1 Dataset . . . 68
7.1.2 Models . . . 69
7.1.3 Metrics . . . 70
7.2 Experimental Evaluation . . . 73
7.2.1 Results . . . 74
7.2.2 Qualitative Analysis . . . 78

7.2.3 Error Analysis 92

8 Conclusions 102
8.1 Main Findings 102
8.1.1 Contributions 103
8.2 Limitations 103
8.3 Future Work 103

Bibliography 104

Chapter 1

Introduction

Large Language Models (LLMs) have changed the way people look for information. Instead of searching for documents and reading them, we increasingly just ask a question and expect an answer. The problem is that LLMs can sound convincing while saying things no source supports, and they only know what they saw during training. Retrieval-Augmented Generation (RAG) has become the standard way to deal with both issues: before answering, the system looks up relevant passages in an external collection of documents and asks the model to base its answer on them.

Most RAG systems treat these passages as independent pieces of text and pick the ones that look most similar to the question. This works well when the answer sits in a single passage, but many questions can only be answered by connecting information scattered across different documents or their cross-references, information that may not resemble the question at all. Graph-based RAG systems such as GraphRAG [13], HippoRAG [23], LightRAG [17], and PathRAG [6] tackle this by organizing the documents into a knowledge graph and using its connections during retrieval. They have proven effective on general-purpose text, but they were not designed with any particular domain in mind.

Law is a domain where reliability is absolutely dominant. To answer a legal question it is rarely enough to read the relevant article, because laws need to be interpreted. This is not a flaw: legislators cannot foresee every situation a rule will apply to, so they write rules in general terms on purpose. The task of determining what a rule means in a concrete case is the responsibility of the courts. When a court applies a provision to a specific situation, it settles the questions the provision leaves open, and its judgment becomes a reference for anyone facing a similar question later. In the European Union this role belongs to the Court of Justice of the European Union (CJEU), whose judgments establish how EU law should be read.

A legal assistant should therefore find not only the rule that applies, but also the judgments that interpret it, and the specific passages where that interpretation is given. These

are links between documents, and similarity alone struggles to find them: the passage of a judgment that answers a question often uses quite different words from the question itself. A knowledge graph is a natural way to capture these links, and in the legal domain much of the needed structure is already available.

The goal of this thesis is to build an LLM-based assistant that answers legal questions using both EU legislation and the case law that interprets it, with a knowledge graph connecting the two. This leads to two research questions.

RQ1 Can a knowledge graph of EU legislation and CJEU case law be built deterministically, preserving the structure of legal documents and the links between judgments and the provisions they interpret, despite the structural heterogeneity of the sources?

RQ2 How can a RAG system exploit the structure of a legal knowledge graph, within the acts and between the acts and the case law that interprets them, to answer legal questions, and how well does it do so compared with general-purpose graph-based RAG systems?

- **RQ2.1** Does confining retrieval to the acts a question concerns produce more complete answers than retrieving from a single pool of documents?
- **RQ2.2** Can a small context, selected through the structure of the graph, answer as completely as the larger contexts of the baselines?
- **RQ2.3** Does following the structure of a judgment and its links to the provisions it interprets make answers on case law more precise?

To answer these questions, this thesis presents *ThemisRAG*, a legal question answering system built on a knowledge graph of EU legislation and CJEU case law. ThemisRAG distinguishes questions that only ask what the law says from questions that require understanding how courts have interpreted it, and looks for its sources accordingly.

1.1 Contributions

The main contributions of this thesis are the following. Contribution C1 addresses RQ1, while C2 addresses RQ2.

C1 A structure-preserving knowledge graph of EU legislation and CJEU case law (Chapter 4). We design an extraction pipeline that recovers the hierarchy of an act (chapters, sections, articles, paragraphs, recitals and annexes) directly from the EUR-Lex markup, rather than inferring it from the text with a language model. Judgments

of the Court of Justice are parsed with a per-document, depth-preserving strategy that remains robust to the change of HTML schema between pre- and post-2018 judgments. The strategy keeps the numbering of their paragraphs and their operative part, and links each judgment to the provisions it interprets. Node identifiers are derived from the CELEX number and the structural position alone, so they are stable across language versions. We further propose a theoretical model for amendments and repeals in which each provision is separated from its time-bounded wordings, and the change itself is represented as a node that records which legislative command produced it.

C2 ThemisRAG, an intent-routed hybrid retrieval architecture for legal questions (Chapter 5). ThemisRAG first identifies what a question asks and which acts it concerns, and searches only within them. It then uses the graph to place each retrieved passage in its context: an article in its act and chapter, a paragraph of a judgment in the reasoning of the Court and next to the provisions it interprets. On top of a standard hybrid retrieval pipeline, it adds a small set of mechanisms, each addressing a specific way in which retrieval over legal text fails, such as the gap between the wording of a question and that of a statute, or the conclusion of a judgment sharing no vocabulary with the question. We also report a negative result: a topic layer built with ASKE did not reliably select the paragraphs that answer a question and was removed from the final design.

1.2 Thesis Structure

This thesis is organised as follows.

- **Chapter 1** introduces the problem, the research questions and the contributions of this thesis.
- **Chapter 2** introduces the background concepts: text representation and retrieval, Transformers and LLMs, RAG, knowledge graphs, prompting, and LLM-as-a-judge evaluation.
- **Chapter 3** reviews knowledge graph construction, with a focus on the legal domain, and graph-based RAG systems (GraphRAG, HippoRAG, LightRAG, PathRAG, HyPA-RAG).
- **Chapter 4** describes the extraction pipeline for acts and case law and the resulting data model, including amendments and multilingualism.

- **Chapter 5** presents ThemisRAG, from the classification of the question to the generation of the answer.
- **Chapter 6** evaluates the construction of the knowledge graph and answers RQ1.
- **Chapter 7** evaluates ThemisRAG against LightRAG, PathRAG and HippoRAG and answers RQ2.
- **Chapter 8** revisits RQ1 and RQ2 in light of the results, and discusses the limitations of this work and directions for future work.

Chapter 2

Preliminaries

This chapter introduces the baseline concepts needed to follow the rest of the thesis. It opens with text representation and Information Retrieval, which are central to RAG systems, then moves to the Transformer and LLM architectures. Knowledge Graphs are presented next, as one of the ways RAG can be grounded in a structured knowledge base. The chapter closes with prompting techniques and evaluation methodologies, namely LLM-as-a-judge [46] and RAGAS [14], which have become standard tools in the field.

2.1 Text Representation and Information Retrieval

The numerical representation allows a machine to search or reason over text. This section explains how the representation has evolved and how each representation is applied to retrieve relevant text. It is also important to note that what is relevant in one domain is not necessarily relevant in another. For example, in the legal domain, semantic information is not always the most relevant.

Sparse representations

The earliest approach represents a document as a vector of word counts, one dimension per word in the vocabulary. Term frequency–inverse document frequency (TF-IDF) refines this idea by weighting a word more if it is frequent in a document but rare across the whole collection, since such words are more likely to characterize what the document is about [35].

Picture three short restaurant reviews: d_1 =“the pizza was amazing”, d_2 =“the pasta was amazing”, and d_3 =“the weather today is nice”. If all we did was count word frequency, “the” would look like the most important word in the whole collection, since it shows up in every single review, but it obviously tells us nothing about which review is about pizza and which is about pasta. TF-IDF fixes this by discounting a word for how often it shows up across the *whole* collection, not just inside one document: a word that appears everywhere gets pushed

down to nothing, while a word that only shows up in a couple of documents stands out.

Written as a formula, the weight of a word t in a document d is

$$\text{tfidf}(t, d) \;=\; \text{tf}(t, d) \cdot \log \frac{N}{\text{df}(t)},$$

where $\text{tf}(t, d)$ is simply how many times t shows up in d, N is the number of documents overall, and $\text{df}(t)$ is how many of those documents contain t at least once. Back to the reviews: "the" is in all three (N=3, df=3), so its score is $\log(3/3) = 0$ – it drops out completely, however many times it repeats. "amazing" shows up in two of the three (df=2), so it gets a small positive score. "pizza" shows up in just one (df=1), so it gets the highest score of the three, which is exactly what we would want, since "pizza" is the word that actually tells d_1 apart from the other two.

BM25 relies on the very same trick, rare words count for more, common words count for less, but wires it into a formula built for ranking whole documents against a query, rather than just scoring words in isolation [33]:

$$\text{BM25}(d, q) \;=\; \sum_{t \in q} \text{idf}(t) \cdot \frac{f(t, d)\,(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b\,\dfrac{|d|}{\text{avgdl}}\right)}.$$

The two extra knobs, k_1 and b, stop a word from being worth more just because it happens to repeat many times, and stop long documents from scoring higher purely because they contain more words overall. This kind of matching works well when the vocabulary is shared, but it is exact: a query containing "car" will not match a document that only says "automobile", even though the two words mean the same thing, and the resulting vectors are large and mostly empty, since any single document only ever uses a small part of the vocabulary.

Static embeddings

Word2vec tried to address the problem by learning a dense vector for each word from large amounts of text; such that words used in similar contexts end up with similar vectors [29], that is why "car" and "automobile" result close words even without ever matching exactly. However, these vectors cannot adapt. In fact, once training is complete, the represented word retains the same vector wherever it appears. This is why the word "bank", which can refer to different contexts (for example, a riverbank or a financial institution), leaves the ambiguity unresolved.

Contextual embeddings and dense retrieval

More recent models overcome this limitation by producing different vector representations of the same word depending on the sentence in which it appears. These contextual embeddings are produced by deep neural encoders such as BERT [12], the architecture of which is

introduced later in this chapter. For now, it is sufficient to understand that a piece of text can be encoded as a dense vector that reflects its meaning in context. Once both queries and documents are encoded this way, retrieval becomes a search for nearby vectors rather than for shared words: the query is embedded, and the documents whose embeddings are closest to it are returned, an approach known as dense retrieval, which dual-encoder architectures such as DPR showed can outperform lexical retrieval on open-domain question answering [25]. In practice, lexical and dense retrieval are often combined into a single hybrid system, since the two tend to fail on different kinds of queries.

2.2 Transformer and Large Language Models

From recurrence to attention

Before the Transformer, sequence models processed text one token at a time, carrying a hidden state forward from each word to the next. This made long-range dependencies hard to learn: information from early in a sentence had to survive many update steps before it could influence a word far away, and in practice it often did not, even with gated architectures such as LSTM that were designed specifically to keep signals from decaying over time [19]. The Transformer removes the recurrence altogether [41]. Its core mechanism, self-attention, lets every token look directly at every other token in the sequence and weigh how relevant each one is, so a dependency between two distant words costs no more to learn than one between two adjacent words.

Multi-head attention and positional encoding

A single attention computation captures one notion of relevance between tokens, so the Transformer runs several of them in parallel, each initialized differently and free to focus on a different kind of relationship; their outputs are then combined, which is what multi-head attention refers to [41]. Removing recurrence solves the dependency problem but introduces a new one: without a sequential update, the model has no built-in sense of word order, since attention treats the input as an unordered set. Positional encoding fixes this by injecting information about each token's position directly into its representation before attention is applied, so order is recovered without bringing recurrence back.

Encoder, decoder, and pretrained language models

The original Transformer pairs an encoder, which turns an input sequence into contextual representations, with a decoder, which generates an output sequence from them [41]. Later models kept only one half. Encoder-only models such as BERT are trained to produce

rich contextual representations of existing text, which is what makes them well suited to the embeddings discussed earlier in this chapter [12]. Decoder-only models, by contrast, are trained to predict the next token given everything before it, and scaling this simple objective up to billions of parameters is what produced large language models capable of generating fluent, coherent text and of following instructions with little or no task-specific training [3].

Context, cost, and hallucination

Because every token attends to every other token, the cost of self-attention grows quadratically with the length of the input, which is why every Transformer-based model has a maximum context window beyond which text cannot be processed at all. A large context window does not guarantee that all of it is used well: language models tend to make better use of information placed at the very beginning or end of their input than information buried in the middle, a systematic effect known as the lost-in-the-middle problem [26]. A separate and equally important limitation is that a language model can produce fluent text that is factually wrong or unsupported by any source, a behavior generally referred to as hallucination [22]. Together, the limited and unevenly used context window and the tendency to hallucinate are the two problems that motivate grounding a language model's output in external, retrieved evidence, which is the subject of the rest of this thesis.

2.3 Retrieval-Augmented Generation

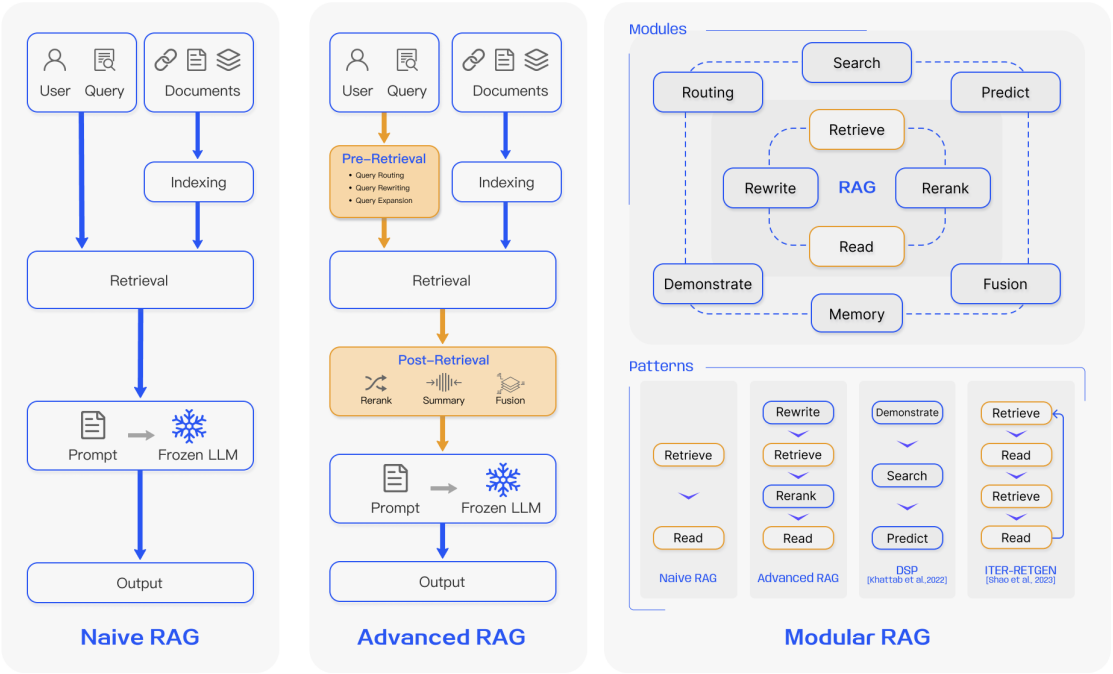


Figure 2.1: Different RAG approaches [16]

Retrieval-Augmented Generation (RAG) has emerged as a strategy for mitigating hallucination in Large Language Models (LLMs) by grounding generation in an external knowledge source, therefore reducing the amount of factually incorrect content in the final response.

The earliest formulation of this paradigm, commonly referred to as *Naive RAG*, follows a three-stage pipeline. The first stage is **indexing**: raw data is collected from heterogeneous formats (PDF, HTML, Word and others), cleaned, and converted to plain text. The resulting text is then segmented into smaller semantically coherent chunks, encoded into dense vector representations through an embedding model, and stored in a vector database. The second stage is **retrieval**: the user query is projected into the same embedding space, and a similarity measure is computed between the query vector and the indexed chunks in order to select the top-k most relevant passages. The third stage is **generation**: the retrieved passages are concatenated with the query into a single prompt, from which the LLM produces a response that is expected to be more pertinent and less prone to hallucination.

Despite its simplicity, this approach exhibits a number of structural limitations:

- **Retrieval quality.** The retrieval stage frequently fails to surface the passages that are actually required to answer the query. Semantic similarity in the embedding space is

only a proxy for relevance, and a high similarity score does not guarantee that a chunk carries the information the query demands. This limitation constitutes the central concern of this thesis.

- **Unsupported generation.** Grounding the model in retrieved evidence does not eliminate hallucination. The model may still produce content that is not entailed by the provided context, either because it falls back on parametric knowledge, because of inherent biases, or because the retrieved passages are themselves noisy or contradictory.

- **Response quality.** When the retrieved context is redundant, excessively long, or only marginally related to the query, the generated answer tends to become verbose and diluted, obscuring the information the user is actually seeking.

To overcome the limitations of the *Naive RAG* paradigm, two successive formulations have been proposed: *Advanced RAG* and *Modular RAG*.

Advanced RAG preserves the sequential structure of the naive pipeline but concentrates on improving retrieval quality through pre-retrieval and post-retrieval strategies:

- **Pre-retrieval.** This stage acts on both the index and the query. Indexing is improved by increasing data granularity, optimizing the index structure, attaching metadata, performing alignment optimization, and adopting mixed retrieval. Query optimization, in turn, aims to reformulate the user's original question into a form better suited to the retrieval task, through techniques such as query rewriting, transformation, and expansion.

- **Post-retrieval.** Once the candidate passages have been retrieved, they must be integrated effectively with the query. Feeding every retrieved document directly into the LLM leads to information overload and dilutes the focus on the relevant details; post-retrieval methods therefore rerank the chunks so that the most pertinent content is relocated to the edges of the prompt, and compress the context by selecting the essential information and discarding what is redundant or marginally related.

Modular RAG takes a further step by abandoning the chain-like structure of the previous two paradigms. Rather than a fixed retrieve–read sequence, the pipeline is decomposed into interchangeable functional modules that can be added, replaced, or rearranged according to the task at hand. Its main features can be summarized as follows:

- **Specialized modules.** Dedicated components extend the basic pipeline, such as a *Search* module for querying heterogeneous sources (search engines, databases, knowledge graphs), a *Routing* module that selects the most suitable pathway for a given

query, and a *Memory* module that exploits the model's own generations to guide retrieval.

- **Independent optimization.** Since each module is self-contained, it can be improved or substituted without redesigning the whole system, making the architecture easier to adapt to domain-specific requirements.
- **Non-sequential control flows.** The process is no longer limited to a single retrieval followed by generation: iterative, recursive, and adaptive strategies allow the system to refine the query over multiple steps and to decide autonomously whether and when further retrieval is needed.
- **Integration with training-based techniques.** The flexible structure facilitates the combination of RAG with fine-tuning of the retriever, of the generator, or of both jointly.

2.4 Knowledge Graphs

Structure and construction

A knowledge graph represents information as a network of entities, shown as nodes, connected by relations, shown as edges, rather than as free-flowing text [21]. Building one from unstructured text starts with recognizing which spans of text refer to entities in the first place, a task known as named entity recognition. Once entities are identified, open information extraction goes a step further and pulls out the relations that hold between them directly from the sentence, without requiring a predefined list of relation types to look for [1]. The result of this process is typically stored as a triple, a simple subject-predicate-object statement such as *Alhandra – born in – Vila Franca de Xira*, which is the basic unit most knowledge graphs are built from. Entities and relations can also be projected into dense vectors, similarly to the text embeddings introduced earlier in this chapter, which makes it possible to measure how similar or how related two entities are even when they are not directly connected by an edge.

Reasoning over the graph

Once a graph exists, several algorithms can exploit its structure rather than the text it was built from. PageRank estimates the importance of a node from how it is connected to the rest of the graph, originally to rank web pages by how many other important pages link to them [31]; the same idea can be seeded from a specific set of nodes instead of the whole graph, so that importance spreads outward from a query rather than being computed

globally, an approach generally referred to as personalized PageRank. Community detection algorithms such as Leiden instead group nodes into densely connected clusters, uncovering a coarser structure than individual entities and relations, one that corresponds more closely to overarching themes than to single facts [38]. Finally, simply following the edges around a node, up to some fixed number of hops, is often enough to gather everything closely related to it without running a full algorithm at all.

2.5 Prompting Techniques

From zero-shot to in-context learning

The simplest way to use a language model is to ask it a question directly and take whatever it answers, which is called zero-shot prompting. Performance usually improves if the prompt includes a handful of worked examples of the task before the actual question, a setting known as few-shot prompting [3]. What makes this effective is in-context learning: the model adapts its behavior to the pattern shown in the prompt without any of its parameters being updated, purely from what it reads in that single forward pass.

Chain-of-thought

Few-shot examples do not have to show only the final answer. If the examples instead walk through the reasoning steps that lead to it, the model tends to produce similar step-by-step reasoning for the new question as well, and this tends to improve accuracy on tasks that require several intermediate steps, such as arithmetic or multi-step logical problems [42]. This technique is known as chain-of-thought prompting, and the same instinct, breaking a problem into intermediate steps rather than jumping straight to an answer, reappears later in this thesis in the context of multi-hop retrieval.

Query rewriting

A prompting technique more specific to retrieval is query rewriting: an LLM is used to reformulate the user's original question into a form better suited to the retriever, before the retrieval step even happens. The original query is not always phrased in a way that matches how the answer is worded in the corpus, and rewriting it first, sometimes several times, can noticeably improve what gets retrieved [28]. This is a pre-retrieval technique in the sense introduced earlier in this chapter, and it recurs throughout the methods discussed in the next chapter.

2.6 LLM-as-a-Judge and RAGAS

Automating evaluation

Judging whether a generated answer is good is not as simple as checking it against a single correct string, since open-ended questions can have many valid phrasings; human judgment scales well to quality but poorly to volume, since every new system or every new version of a system requires a fresh round of annotation. Using a strong language model as the judge instead of a human addresses the volume problem, and it can be done in two ways: pointwise, where the judge assigns a single answer a score on its own, or pairwise, where the judge is given two answers to the same question and asked which one it prefers [46]. Neither is free of bias: judges have been shown to favor longer answers regardless of quality, to be sensitive to which answer is shown first in a pairwise comparison, and to rate an answer more favorably when it resembles what the judge model itself would have generated [46]. These biases matter whenever an LLM judge is used to compare systems, a practice that recurs throughout the next chapter.

RAGAS

RAGAS applies the same LLM-as-a-judge idea specifically to retrieval-augmented generation, and it does so without requiring a human-written reference answer for every question [14]. Instead, it breaks evaluation into a small set of targeted metrics: faithfulness checks whether every claim in the generated answer is actually supported by the retrieved passages, answer relevance checks whether the answer actually addresses the question that was asked, and context relevance checks whether the retrieved passages themselves were worth retrieving in the first place. Separating these three concerns makes it possible to tell whether a weak answer came from poor retrieval, from a poor use of good retrieval, or from both, which a single end-to-end score would not reveal. When a reference answer is available, RAGAS also offers metrics that compare the retrieved context and the answer against it; Section 7.1.3 describes the six metrics used in this thesis, four of which require a reference.

Chapter 3

State of The Art

3.1 Knowledge Graph Construction

A knowledge graph turns a source into nodes and edges that follow a schema. How this is done depends mostly on the source. A structured source, such as a database or a markup standard, can be turned into a graph with fixed rules. Free text, instead, needs some form of language understanding, today usually given by a large language model (LLM). This section reviews both directions, then shows how the quality of a graph is checked, and ends with a case study from the legal domain, where both directions appear side by side.

3.1.1 Construction paradigms

A knowledge graph can be built top-down or bottom-up [37]. In a top-down approach, a schema is fixed first, and data is extracted to fill it. In a bottom-up approach, the data is extracted first, and a schema is derived from it afterward. A construction pipeline usually has three stages: acquiring knowledge from the source, refining it by removing errors and duplicates, and updating it over time as new data arrives [47]. The choice of graph model also matters. A graph can be stored as simple triples, as in RDF, or as a property graph, which lets nodes and edges carry lists and other structured attributes directly; the latter is useful when a node needs to carry more than its relationships [20]. None of this says how knowledge is extracted from the source in the first place. This is the question the rest of this section follows: fixed rules, or a language model.

3.1.2 Deterministic and rule-based construction

When a source is already structured, most of the graph can be built with fixed rules instead of a model. A mapping rule reads a specific field of the source and always produces the same kind of node or edge from it. Declarative mapping languages follow this idea: a user defines once how a tabular, XML, or JSON source maps to a graph, and the same mapping

is then applied to every new record [40]. Large open knowledge graphs, such as DBpedia and Wikidata, are built mostly this way, from structured or semi-structured web sources rather than from free text [20]. The main advantage of this approach is traceability: every node and edge can be traced back to the rule that created it. If an edge is wrong, the fault is in a specific rule, not in an opaque model output, so the source can be checked and the graph corrected without touching the rest of the pipeline.

3.1.3 LLM-driven construction

When the source is free text, its structure is not given and must be inferred. Named entity recognition (NER) and relation extraction (RE) are the classic tools for this task: they find entities in a sentence and the relation between them, and turn each finding into a triple. LLMs have largely replaced these dedicated tools in recent work, since a single model can be prompted, with no or few examples, to read a passage and directly output triples, or even the schema it should follow [2]. Two ways of doing this are common. In a schema-based way, the model is given a fixed set of classes and relations and asked to fill it. In a schema-free way, the model proposes new classes and relations as it reads, and the schema is fixed only afterwards [2]. This also makes it easier to keep a knowledge graph and a language model working together, since a graph built by one model can later ground the answers of another [32]. The main weakness of this approach is reliability: a model can miss a fact, invent one, or extract the same fact differently in two different passages, so its output usually needs some form of check before it can be trusted.

3.1.4 Evaluating the quality of constructed knowledge graphs

Once a graph is built, its quality still needs to be checked, in one of two ways. The first is intrinsic: the graph is checked against a set of fixed dimensions, such as accuracy (do the facts match reality), completeness (is anything missing), consistency (do any facts contradict each other), and timeliness (is the graph up to date) [44]. The second is task-based: the graph is checked by asking it a fixed set of questions and verifying the answers, without judging the graph directly. The two are complementary. The first gives a general picture of the graph. The second shows whether that picture matches what a real user needs.

3.1.5 Case study: knowledge graph construction in the legal domain

The legal domain is a good case study because it puts both directions side by side. Legislation is often published in a controlled, machine-readable format. Court decisions, instead, are written as ordinary text, so their structure must be inferred rather than parsed.

Legislation. [8] build a knowledge graph of the Italian legislation from Akoma Ntoso (AKN), an XML standard for the text of laws. Each AKN tag is mapped to a graph object with a fixed rule: a title tag becomes the title of a law node, an article tag becomes an article node, and a tag marking a change becomes an edge such as AMENDS or ABROGATES. Because the mapping is fixed, errors can be found with simple graph queries; the authors detect and correct thousands of wrong or misclassified edges this way. The graph is not left purely rule-based, though: some properties, such as an article's title or topic, are missing in many nodes because the source tag was left empty. The authors fill these gaps with small, fine-tuned models one to classify the domain of a law, one to generate missing titles and topics while the rest of the graph stays rule-based. [11] follow a similar path with a different formal model: the graph separates an abstract legal text from each of its dated versions, and represents every legal change as an explicit node with a validity interval, again built with rules rather than free generation. Both works also need to represent that laws change over time, without creating a new node for every version: Colombo et al. store each change as a property of the edge that introduces it, while de Martim keeps a separate, dated node for each version. [7] later reuses the Italian legislation graph and adds embeddings to it for retrieval, showing how a rule-built graph can still be extended once it exists.

Case law. [10] build two knowledge graphs from the same set of European Court of Human Rights judgments on violence against women, using two different pipelines, and compare them directly. The first pipeline stays close to the deterministic case: it reads structured metadata already on the court website, such as case number, date, and respondent state, and combines it with a small ontology built by hand. The second pipeline is fully LLM-driven: for each judgment, a retrieval step limits a model to that document's own text, one model proposes an ontology, and a second model generates the graph with few-shot prompts. The LLM-driven pipeline is faster to build and easier to extend to new documents, but its output needs manual checking, since the model sometimes misses or invents a relation. The manual pipeline is slower to build but needs less checking. The authors also compare giving the model the full judgment against giving it only a section chosen by a legal expert: the full text answers more questions but with more noise, while the shorter, chosen section answers fewer questions but more precisely.

Quality. Both papers check their graph in the ways described above. [8] measure accuracy directly, comparing what the graph infers about abrogated laws with what a separate scan of the current legal text finds, and get agreement in 98% of cases. [10] instead validate their graphs with a fixed list of questions the graph should answer, checked by hand against the source text. The first is an intrinsic check, the second a task-based one, and together they cover both ways of asking whether a graph is good enough to use.

Once a graph is built, by any of these means, the next problem is how to query it. This is the subject of the rest of this chapter.

3.2 Graph based RAG

The limitations discussed so far share a common root: retrieval in conventional RAG systems is *flat*. The index is a collection of independent chunks, and similarity matching returns fragmented passages that carry no information about how the concepts they mention relate to one another. This restricts the ability of the system to answer questions requiring either multi-hop reasoning across separate portions of the corpus, or a global understanding of a document that no single passage contains [43].

Graph Retrieval-Augmented Generation (GraphRAG) addresses these limitations by organizing the corpus, during indexing, into a structured representation of the knowledge it contains, typically a graph whose nodes represent concepts and whose edges represent the relations among them. Two families of methods have emerged within this line of work that rely on hierarchical organization: RAPTOR [36], which builds a tree through recursive clustering and multi-level summarization, and Microsoft's GraphRAG [13], which extracts an explicit knowledge graph and applies community detection with LLM-generated summaries. Despite their differences, both pursue the same objective, that of enabling coarse-to-fine retrieval over a corpus that has been given an explicit structure.

The relevant question is whether this additional structuring actually pays off. Recent evidence suggests that it does: on a domain-specific benchmark built from twenty computer science textbooks, most graph-based methods outperform traditional retrieval baselines and, more notably, all of them improve the reasoning capabilities of the underlying language model with respect to the model used alone [43]. Two mechanisms account for this gain. First, structuring the corpus allows the system to retrieve not only the passages that are semantically closest to the question, but also the passages on which the answer depends through multi-hop dependencies, thereby providing evidential support for the reasoning process rather than isolated facts. Second, the summaries produced during construction give access to information that is not explicitly stated anywhere in the original text, which is precisely what thematic and global questions require.

These benefits, however, are not uniform. The same evaluation shows that the improvement depends on the type of question: it is substantial where the model benefits from external evidence, as in true-or-false and open-ended questions, while it can be marginal or even negative where the retrieved content introduces noise that does not precisely match the context of the question. Structuring the corpus, in other words, improves the quality of the evidence available to the model, but it does not remove the need for the retrieval step to be

accurate.

3.2.1 Microsoft GraphRAG

Motivation

The starting observation of [13] is that conventional RAG, which the authors refer to as *vector RAG*, works well only for queries answerable with information localized within a small set of records. It fails on *global* questions directed at an entire corpus, such as "what are the main themes in the dataset?", because answering them is inherently a query-focused summarization task rather than a retrieval task: no small set of chunks suffices, since the answer depends on the corpus as a whole. Prior summarization methods, in turn, do not scale to the volumes of text indexed by RAG systems. GraphRAG is designed to combine the two, scaling with both the generality of the question and the quantity of source text.

How it works

The system uses an LLM to build a graph index in two stages, first deriving an entity knowledge graph from the documents, then pre-generating summaries for all groups of closely related entities:

- **Element extraction.** After chunking, the LLM extracts the important entities of each chunk and the relationships among them, generating a short description for each, and optionally *claims*, that is, verifiable factual statements about the entities. Prompts can be tailored to the domain through few-shot exemplars, which is particularly valuable in specialized fields.
- **Graph construction.** Since an element is typically detected several times across the corpus, the extracted instances are aggregated into nodes and edges, their descriptions are summarized, and the number of duplicates of a relationship becomes the weight of the corresponding edge.
- **Community detection.** The graph is partitioned into communities of strongly connected nodes with the Leiden algorithm, applied recursively so that each level of the resulting hierarchy covers the nodes in a mutually exclusive and collectively exhaustive way, which is what enables divide-and-conquer summarization.
- **Community summarization.** A report-like summary is generated for each community, bottom-up along the hierarchy, with higher-level summaries recursively incorporating the lower-level ones.

At query time the answer is produced by map-reduce over the pre-computed summaries: partial answers are generated from them independently and in parallel, each with a helpfulness score assigned by the model, and the surviving ones are then combined, in decreasing order of score, into the final global answer.

Benefits

The main advantage of the approach lies in the quality of the answers it produces on global questions, which are consistently more comprehensive and more diverse than those obtained through vector RAG. Because the community summaries are pre-computed at indexing time, answering a query is also considerably cheaper than summarizing the source texts directly, and the hierarchical structure allows the level of detail to be traded against cost by choosing the level of the hierarchy from which the summaries are drawn. The summaries are moreover independently useful: they allow a user to scan the themes of a corpus at one level and descend to the linked reports below for further detail, even in the absence of a specific query.

Limitations

The main cost of the approach lies in indexing: building the graph requires invoking the LLM for extraction, description, and summarization at every level of the hierarchy, which makes it among the most resource-intensive of the construction strategies currently in use [43], and the additional descriptions attached to nodes and edges inevitably introduce noise alongside the added content. The advantage, moreover, is concentrated on questions requiring global understanding: vector RAG remains more direct, and the analysis suggests that some of the specific examples and citations that help users reach an informed judgment are lost in the index. Finally, the evaluation covers sensemaking questions over a limited number of corpora, so its generalization to other domains and use cases remains to be established.

3.2.2 HippoRAG

Motivation

HippoRAG [23] starts from a different diagnosis than Microsoft GraphRAG [13]. The obstacle it identifies is not the generality of the question but the isolation of the index: since each passage is encoded independently of the others, conventional RAG cannot be able to work with tasks that require integrating knowledge *across* passage boundaries. Multi-step systems can partially compensate for this by incorporating both retrieval and generation, but they remain restricted to what the authors call *path-following* questions, in which a single chain of entities has to be traversed in a fixed order. They break down on *path-finding* questions,

where the relevant path must be identified among many candidates. The example that motivates the paper, "which Stanford professor works on the neuroscience of Alzheimer's?", is of this kind: neither search entity is sufficient on its own, An iterative process doesn't have a clear way of choosing between the many paths that lead off from it. What is missing is not more retrieval steps but an associative structure that makes the intersection of two large sets directly accessible.

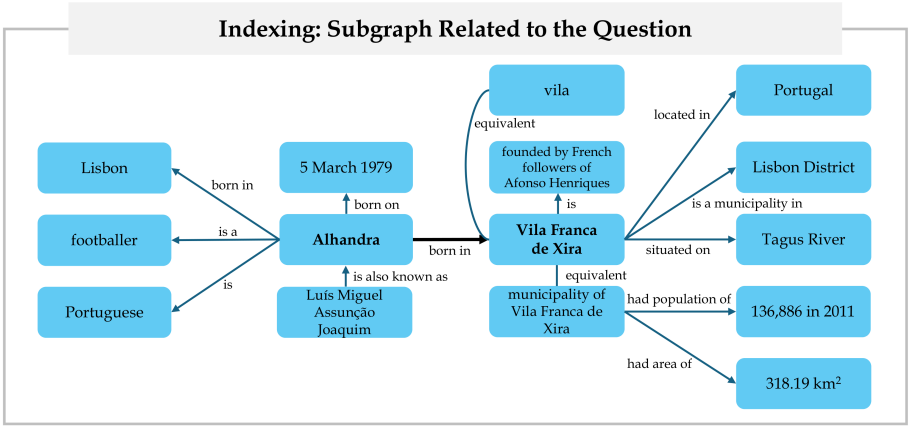
How it works

The design is guided by an analogy with the *hippocampal memory indexing theory*, according to which long-term memory arises from the interaction between the neocortex, which stores the actual representations, and the hippocampus, which holds a sparse index of pointers together with the associations among them. Each component corresponds to one element of this model, and the correspondence is what motivates the choices that distinguish the method:

- **An LLM as artificial neocortex.** Each passage is processed, during offline indexing, by named entity recognition (NER) followed by open information extraction conditioned on the entities just found, which keeps extraction anchored to named entities while still admitting more general concepts.
- **A schemaless knowledge graph as hippocampal index.** Nodes are noun phrases and edges are the extracted relations, with no description or summary attached to either: the graph carries no generated content, only pointers into the corpus, recorded in a node-passage occurrence matrix.
- **Retrieval encoders as parahippocampal regions.** The encoders that link the query to the graph also add synonymy edges between entities whose embeddings are close, connecting near duplicates without an explicit entity resolution step.
- **Personalized PageRank as pattern completion.** At query time the entities extracted from the question seed a random walk, so that probability mass spreads only through their neighbourhood; the resulting node distribution is projected onto the passages through the occurrence matrix, which yields the final ranking.
- **Node specificity as a local importance signal.** Each seed is weighted by the inverse of the number of passages from which its node was extracted, an inverse document frequency term computable from information already available at the node itself.

Figure 3.1 follows one question through both phases. "In which district was Alhandra born?" depends on two passages, one placing the footballer's birth in Vila Franca de Xira

and one making Vila Franca de Xira a municipality of the Lisbon District; since neither contains both facts, no similarity measure can rank the second highly. Extraction turns them into the subgraph of panel (a), where the answer sits two edges away from the only entity the question mentions. Panel (b) shows the search: "Alhandra" receives all the initial mass, and after the walk the node that has gained the most among those absent from the question is "Vila Franca de Xira", the bridge to the answer. No decision was taken about which direction to follow, the relevant passage surfacing because it lies where the diffusion concentrates.



(a) Subgraph extracted from the two supporting passages.

Retrieval: Query NER & Node Retrieval

Question	In which district was Alhandra born?
NER	["Alhandra"]
Node Retrieval	{"Alhandra": "Alhandra"}

Retrieval: PPR

Node Probabilities Changes by PPR

Alhandra	1.000 ⇒ **0.533**	5 March 1979	0.000 ⇒ 0.045
Vila Franca de Xira	0.000 ⇒ **0.054**	Luís Miguel Assunção Joaquim	0.000 ⇒ 0.044
Lisbon	0.000 ⇒ 0.049	Portugal	0.000 ⇒ 0.009
footballer	0.000 ⇒ 0.047	Tagus River	0.000 ⇒ 0.007
Portuguese	0.000 ⇒ 0.046	José Pinto Coelho	0.000 ⇒ 0.004
…			

(b) Node probabilities before and after Personalized PageRank.

Figure 3.1: Indexing and retrieval in HippoRAG for the question "in which district was Alhandra born?". Adapted from [23].

Benefits

A single retrieval step performs multi-hop reasoning, because the graph search reaches passages that are not similar to the query but are connected to it through intermediate entities.

What the design buys is not a faster way of following a chain but the ability to dispense with the chain completely: the walk explores the neighbourhoods of all query entities at once, so the passage lying at their intersection emerges from the probability distribution rather than from a sequence of decisions about which direction to take. This is why the method reaches path-finding questions that iterative retrieval cannot address, and it is consistent with the observation that its advantage lies in retrieving the *complete* set of supporting passages rather than a few more of them for many questions.

Table 3.1 makes the distinction concrete. On a path-following question an iterative method succeeds, since a single place of birth leaves only one path to follow; on a path-finding question it fails in exactly the same way as single-step dense retrieval, returning the same three Stanford neuroscientists who work on other subjects. The problem is not that the iteration is executed poorly, but that no sequence of retrieval steps is guaranteed to find the intersection of two large sets when neither can be enumerated.

	Question	**HippoRAG**	**ColBERTv2**	**IRCoT**
Path-following	In which district was Alhandra born?	Alhandra; Vila de Xira; Portugal	Alhandra; Dimuthu Abayakoon; Ja'ar	Alhandra; Vila de Xira; Póvoa de Santa Iria
Path-finding	Which Stanford professor works on the neuroscience of Alzheimer's?	Thomas Südhof; Karl Deisseroth; Robert Sapolsky	Brian Knutson; Eric Knudsen; Lisa Giocomo	Brian Knutson; Eric Knudsen; Lisa Giocomo

Table 3.1: Top three passages retrieved for the two question types. Adapted from [23].

The same property keeps retrieval cheap in a way that does not scale with the number of hops: since the associations are laid down during indexing, the online phase has only to extract the entities of the question, while an iterative system must read what it retrieves at every round to decide what to retrieve next. The two remain complementary rather than competing, since the graph can also serve as the retriever inside an iterative loop.

The index is additive for the same reason it is cheap online: because nodes and edges carry pointers rather than generated prose, a new document contributes its own entities and connects to what is already indexed through whatever it shares with it, with nothing to recompute and no aggregate to invalidate. This is the sense in which the method behaves as a long-term memory rather than as a periodically rebuilt summary.

Limitations

The design choice that makes single-step multi-hop retrieval possible is also what limits the method, and the reported failures fall into three classes corresponding to the three stages

of the pipeline. The largest, close to half of the analyzed errors, concerns the treatment of the query: since only named entities are extracted from it, everything the question expresses through context is discarded before the search begins, so a query such as "when was one internet browser's version of Windows 8 made accessible?" reaches the graph as the single entity "Windows 8".

The authors describe this as a trade-off between concepts and context, and it cuts in both directions. On a question about the birthplace of the boxer Sergio Villanueva, a dense retriever follows the context to unrelated Spanish navigators while HippoRAG anchors on the entity and retrieves the right passage; on a question about the discovery of atomic number, where the only taggable concept is "protons", the outcome is reversed and HippoRAG drifts towards uranium and nuclear weapons. What the entity-centric index gains on questions built around a named entity, it loses on questions whose concepts are general. The authors mitigate this only partially, by averaging graph scores with those of a dense retriever when the query entities cannot be linked confidently to any node.

The remaining two classes concern extraction, which misses long entities and temporal properties and degrades as passages grow longer, leaving gaps that no later stage can repair; and the search itself, which, receiving the query only through its seeds, cannot distinguish between passages discussing the same entities in different contexts. Beyond these, indexing remains more expensive than in conventional RAG, since every passage requires at least one model call, none of the components is trained for the role it plays, and the scalability of the index is still unproven, the evaluation being confined to corpora of a few thousand passages.

3.2.3 LightRAG

Motivation

LightRAG [17] shares the diagnosis of Microsoft GraphRAG [13], that flat representations produce fragmented answers whenever the answer depends on the interdependencies among several entities, and adopts the same evaluation protocol. Its contribution is therefore positioned against the cost of that approach rather than against its retrieval quality. Two costs are targeted: at query time, answering requires traversing the pre-computed community reports, each of which is a document in its own right; at update time, the arrival of new documents forces the community structure to be dismantled and every report to be regenerated. The objective is to retain graph-structured indexing while keeping retrieval cheap and updates incremental.

How it works

Indexing follows the familiar sequence of chunking and LLM-based extraction, organized in three steps:

- **Extraction.** An LLM identifies entities and the relations among them within each chunk, producing the nodes and edges of the graph.
- **Profiling.** Each node and edge receives a key-value pair, where the key is a word or short phrase suitable for retrieval and the value is a paragraph summarizing the relevant material. Entities use their own name as their only key, while relations may receive several, including keys expressing the global themes of the entities they connect. This asymmetry is what later allows a single index to serve two different retrieval levels.
- **Deduplication.** Identical entities and relations coming from different chunks are merged, which keeps the graph small and the subsequent operations on it inexpensive.

The asymmetry introduced by profiling is visible in Figure 3.2. The entity *Beekeeper* is indexed under its own name and nothing else, while the relation connecting *Honey Bee* to *Industrial agriculture* is indexed under *Agriculture*, *Production* and *Environmental Impact*, none of which is the name of either endpoint. The two sets of keys are therefore addressable by different kinds of query: an entity can only be reached by naming it, whereas a relation can be reached by naming a theme that neither of the entities it connects would carry on its own.

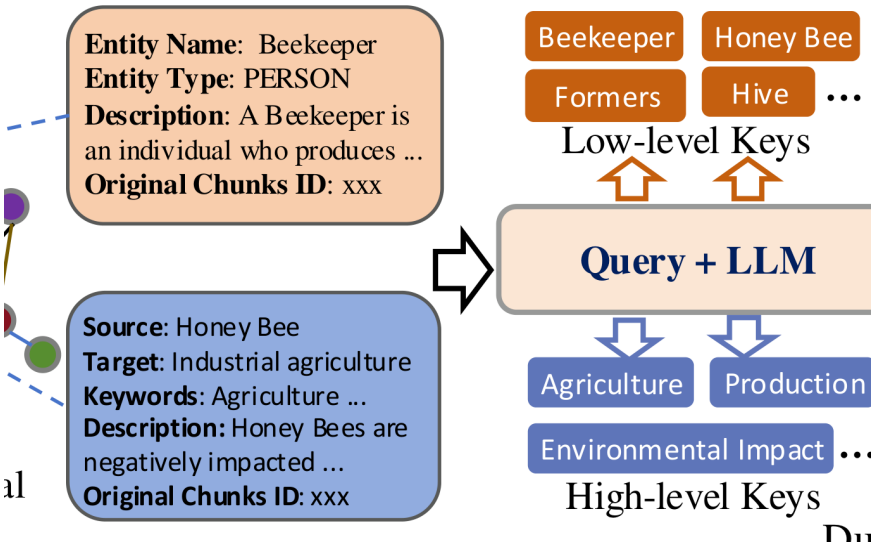


Figure 3.2: Key-value profiles of an entity and of a relation, and the two sets of keys they populate. Adapted from [17].

Retrieval is organized around the observation that queries fall into two regimes, specific ones referring to particular entities and abstract ones referring to themes, and that a single matching strategy serves them badly. The query is therefore decomposed by the LLM into two sets of keywords, used differently:

- **Local keywords** denote concrete entities and are matched against the entity keys, retrieving specific nodes together with their attributes and relations.
- **Global keywords** denote themes and are matched against the relation keys, retrieving the edges whose descriptions aggregate material from several connected entities.

For the query "what metrics are most informative for evaluating movie recommendation systems?", the decomposition yields *accuracy*, *precision*, *recall*, *F1 score* and *user satisfaction* on the local side, and *metrics*, *movie recommendation systems* and *evaluation methods* on the global one. The first set names things the index contains as nodes; the second names what the question is about, which no single node is.

The elements returned by either route are expanded with their one-hop neighbours, and the corresponding values, together with the associated text chunks, form the context passed to the generator. A new document is processed through the same pipeline and merged with the existing graph by union of the node and edge sets, without reconstructing the index.

Benefits

The gain over community-based retrieval is structural rather than incidental. Answering a query costs one model call to produce the keywords plus a vector lookup, whereas traversing community reports requires reading a text of report length for every community considered; on the largest of the four corpora used in the paper this is the difference between fewer than a hundred tokens and the six hundred thousand needed to read the reports of the level of the hierarchy in use. Absorbing a new document, likewise, costs the extraction of its own entities and relations, whereas a community hierarchy has to be recomputed and its reports rewritten, including those of the portions of the corpus that have not changed. Neither cost depends on the size of the collection in the way the corresponding cost does for Microsoft GraphRAG [13].

The ablations offer some support for the two design choices that produce this saving. Removing either retrieval level degrades the answers in the direction one would predict, the low-level route alone being unable to gather enough material for thematic questions and the high-level route alone losing precision on specific ones, which suggests that the two keys attached to the same index are doing distinct work. Removing the original text chunks

from the context, in turn, leaves performance essentially unchanged, which indicates that the values generated during profiling already carry most of what the generator needs.

Limitations

The claims about answer quality rest on weaker ground. They are established by having a language model compare pairs of answers along subjective criteria, on questions that a language model generated from the corpus itself, without any reference answer; the reported margin over Microsoft GraphRAG under this protocol is small, on the order of a few percentage points and reversed on one of the four corpora, and it is not comparable with results obtained on benchmarks that measure retrieval against annotated supporting passages. What the protocol does measure reliably, moreover, is comprehensiveness and diversity of the prose, which are not the same thing as the correctness of the evidence retrieved.

Two structural limitations are independent of how the system is evaluated. The first is that keyword extraction has no fallback. The keywords of the example above are produced from the query alone, with no access to the graph, so nothing guarantees that any of them corresponds to a key that was actually created during indexing; a query whose keywords are badly chosen retrieves the wrong region of the graph, and no later stage is able to detect it, let alone correct it. The second is that the expansion around the retrieved elements stops at one hop, so evidence that lies further away in the graph is unreachable regardless of how relevant it is.

3.2.4 PathRAG

Motivation

PathRAG [6] starts from a similar diagnosis as LightRAG [17]: a flat index misses the connections between entities that a query needs. However, PathRAG argues that current graph-based RAG methods do not actually suffer from too little information. They suffer from too much of it. GraphRAG [13] passes to the LLM everything contained in the communities it selects, and LightRAG passes all the immediate neighbors of the retrieved nodes. Much of this content is not relevant to the specific query, and it becomes noise that dilutes the answer and adds unnecessary token cost. A second issue is presentation: both methods place the retrieved nodes and edges in a flat list, without preserving the order in which they connect to one another, so the LLM has to reconstruct the reasoning path on its own, which hurts the logicality of the answer.

How it works

PathRAG builds its indexing graph in the same way as GraphRAG, with nodes as entities and edges as their relationships, each carrying a short text description. Given a query, the system proceeds in three stages:

- **Node retrieval.** An LLM extracts keywords from the query. Each keyword and each node identifier is encoded into an embedding, and the nodes whose identifiers are closest to the keywords are retrieved, up to a fixed number.
- **Path retrieval.** For every pair of retrieved nodes, the system searches for the paths that connect them and keeps only the most reliable ones. Reliability is computed with a flow-based pruning algorithm: a unit of resource starts at the first node and spreads to its neighbours, shrinking at each step by a decay factor and by the number of neighbours it has to split across. A path is scored by the resource that reaches its nodes on average, and paths whose resource falls below a threshold are dropped before the search goes any further. Because the resource decays with distance, nodes close to the starting point score higher than distant ones, so the algorithm naturally favors short, strong connections over long, weak ones.
- **Answer generation.** Each retained path is converted into text by concatenating the description of its nodes and edges in order. Because LLMs tend to pay more attention to the beginning and the end of a prompt than to its middle, the paths are placed in ascending order of reliability, with the single most reliable path positioned right before the query is answered.

Figure 3.3 shows the process for the query "What organic methods have been effective in reducing pest populations?". The keywords "organic methods" and "pest populations" are extracted and matched to nodes such as *ladybugs* and *aphids* in the indexing graph. The path retrieval stage then searches the neighbourhood of these nodes and finds several candidate paths, for instance *ladybugs* to *aphids* directly, and the longer path through *spider* and *parasitic wasp*. Each path receives a flow value computed from the propagated resource, and only the highest-scoring paths are kept and turned into text for the final prompt.

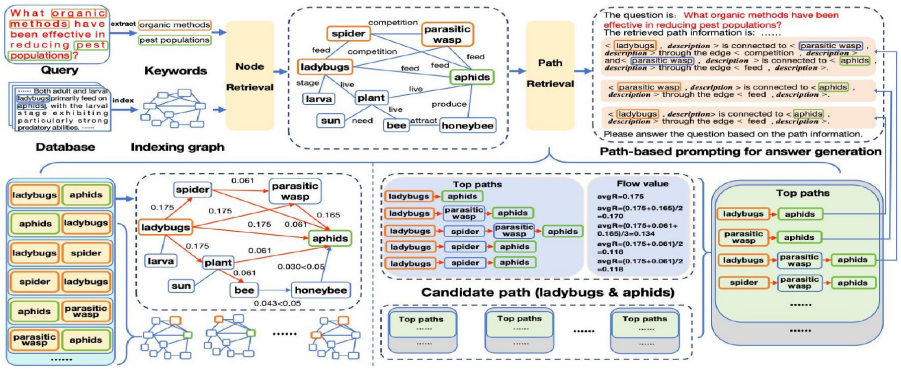


Figure 3.3: Node retrieval, path retrieval, and path-based prompting in PathRAG for the query about organic pest control. Adapted from [6].

Benefits

Because only the most reliable paths are kept, PathRAG avoids the redundancy that affects GraphRAG and LightRAG, without needing to inspect every node in a community or every neighbour of a retrieved node. Figure 3.4 illustrates the difference: GraphRAG keeps everything inside the selected communities, LightRAG keeps every neighbour of the query-related nodes, while PathRAG keeps only the paths that connect them. In experiments across six datasets, PathRAG reaches higher win rates than GraphRAG, LightRAG, NaiveRAG, HyDE, G-retriever, and HippoRAG, on comprehensiveness, diversity, logicality, relevance, and coherence. The gain in coherence and logicality matches the design choice of keeping the path structure in the prompt: a version that flattens the paths into an unordered list of nodes and edges performs worse than the path-based version. The pruning strategy is also light to compute, since the number of nodes it needs to explore is bounded by the decay factor and the threshold, and stays well below the size of the full indexing graph. In practice, this keeps the token cost of PathRAG lower than LightRAG while still improving answer quality.

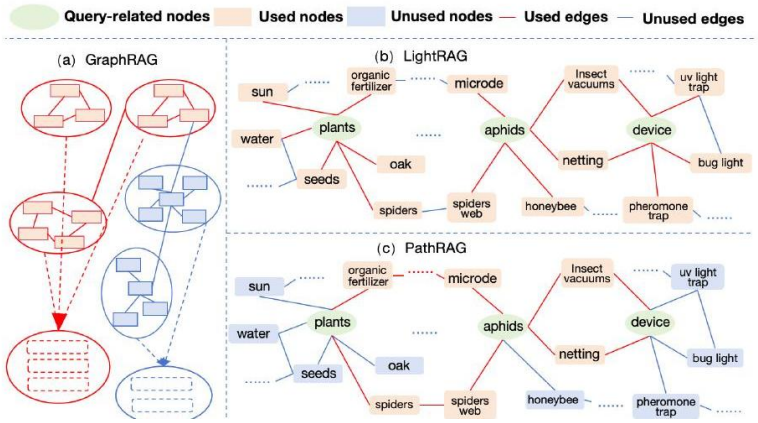


Figure 3.4: What each method keeps from the indexing graph. GraphRAG keeps whole communities, LightRAG keeps one-hop neighbourhoods, PathRAG keeps only the key paths. Adapted from [6].

Limitations

Node retrieval in PathRAG depends on the same keyword-to-embedding matching used in LightRAG, so it inherits the same weakness: if the keywords extracted from the query do not align well with any node identifier in the graph, the wrong region of the graph is searched, and no later stage can correct it. Answer quality is measured mostly through pairwise comparisons made by an LLM, on questions that were themselves generated by an LLM from the corpus, which is a less strict test than checking answers against annotated supporting passages. The two hyperparameters that control the pruning, the decay rate and the threshold, need to be tuned for the domain at hand, and they directly affect both which paths survive and how much the retrieval costs.

3.2.5 HyPA-RAG

Motivation

HyPA-RAG [24] targets a different setting from the graph-based methods discussed so far. Instead of a general-purpose corpus, it is built for AI legal and policy documents, using New York City's Local Law 144 as the test case. In this domain, mistakes are costly, and standard RAG systems tend to use the same retrieval settings for every query, regardless of how simple or complex it is. This wastes tokens on simple questions and under-serves complex ones that need several passages to be answered correctly. Legal text also mixes precise, repeated terminology with concepts that only become clear once several sections are read together, so dense retrieval alone can miss both exact wording and the links between clauses.

How it works

HyPA-RAG combines three ingredients: adaptive parameter selection, hybrid retrieval, and an optional knowledge graph. The pipeline has three stages:

- **Pre-retrieval.** A query complexity classifier, a fine-tuned DistilBERT model, assigns each query to a complexity class based on how many separate passages it is likely to need. This class selects the retrieval hyperparameters to use: the number of query rewrites, the number of chunks to retrieve, and, when the knowledge graph is active, the number of keywords to extract and how many hops of the graph to search.
- **Retrieval.** A dense retriever and a sparse retriever, BM25, each return an initial set of chunks, which are merged with reciprocal rank fusion and optionally reordered by a reranker. In parallel, if the knowledge graph branch is active, keywords from the query are used to pull relevant triplets from a knowledge graph built beforehand by extracting subject, predicate, and object triplets from the corpus with an LLM.
- **Generation.** The retrieved text chunks and, when available, the knowledge graph triplets are combined with the original query and passed to the LLM to produce the final answer.

The system has two configurations: *Parameter-Adaptive RAG*, which uses only the adaptive hybrid retrieval, and *Hybrid Parameter-Adaptive RAG*, which adds the knowledge graph branch. Figure 3.5 shows the full pipeline: a user query first passes through the complexity classifier, which sets the parameters for the rest of the system, then the hybrid retriever and the knowledge graph retriever each contribute their own results, text chunks on one side and knowledge graph triplets on the other, and both are placed in the context window of the LLM together with the original query to generate the answer.

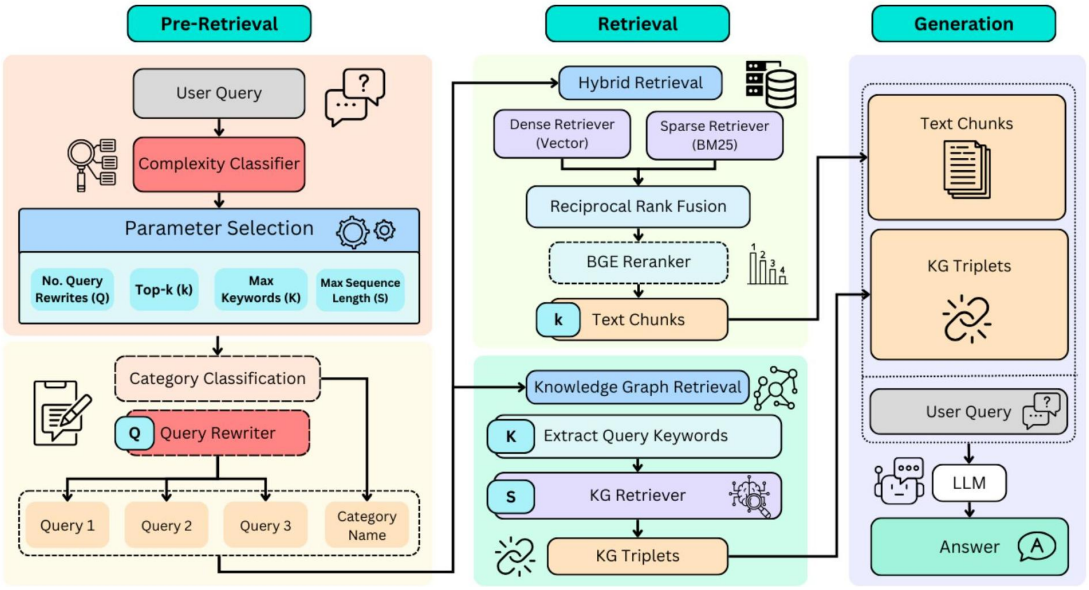


Figure 3.5: The HyPA-RAG pipeline, from query complexity classification to hybrid and knowledge graph retrieval and final generation. Adapted from [24].

Benefits

Adaptive parameter selection consistently beats fixed top-k retrieval. In the reported experiments, the Parameter-Adaptive configuration reaches a faithfulness score of 0.90, against 0.85 for the best fixed-k baseline, and its correctness score is also higher. Because the number of chunks and query rewrites is chosen per query, simple questions are answered with fewer tokens while complex ones receive a wider context, which keeps the average cost down without sacrificing quality on harder queries. Adding a reranker and a query rewriter on top of the adaptive parameters gives a further improvement in faithfulness and correctness, showing that the components combine well. The evaluation of chunking strategies also finds that a domain-specific splitting rule, based on the section markers used in the legal text, retrieves more relevant context than generic sentence-level or semantic chunking.

Limitations

Adding the knowledge graph does not always help: in the reported ablations, the full HyPA-RAG configuration keeps the same correctness score as the version without the graph, but its absolute correctness score is lower, which suggests the extra triplets can add complexity without adding value. Correctness is judged by a single LLM-based evaluator, whose scores are checked against only one reviewer's human judgment, so the reliability of this evaluation is not established at scale. The complexity classifier is trained on synthetic queries generated

by an LLM over the same legal corpus, which may pass on that LLM's own biases and limit how well the classifier generalizes to real, unseen queries. Finally, the system is evaluated on a single domain and a single law, so it remains to be seen whether the same benefits carry over to other legal or policy corpora.

Chapter 4

ThemisRAG Data Model Construction

4.1 Knowledge Extraction

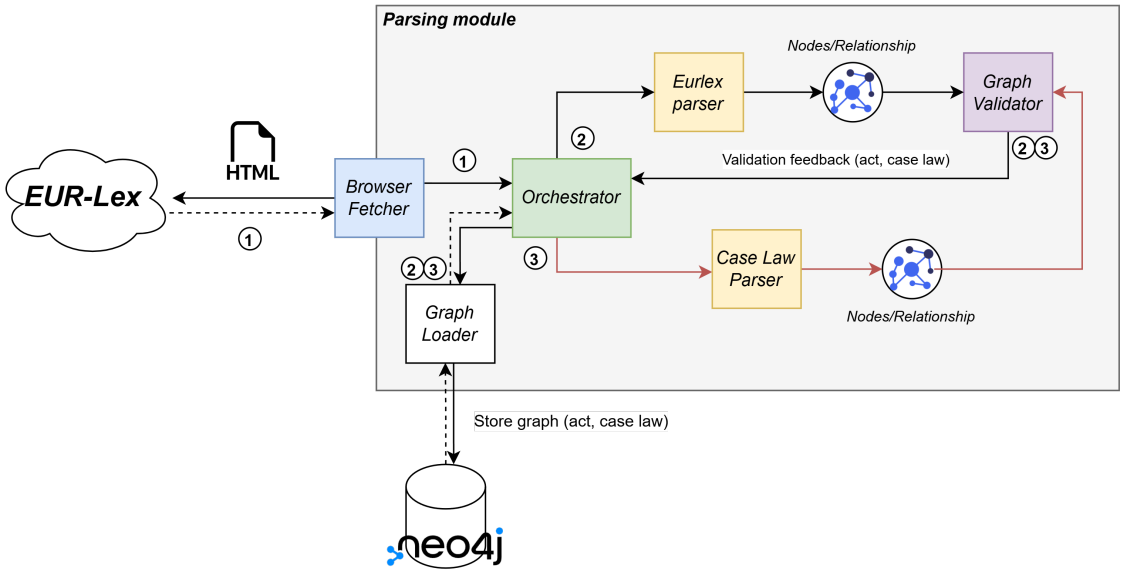


Figure 4.1: Knowledge extraction pipeline

EUR-Lex [1] was chosen as the source of legislation and case law for the graph. Since EUR-Lex publishes its documents as well-structured HTML, whose markup encodes the hierarchy of the text explicitly, the structure of the graph is recovered from the document rather than inferred from its text.

[1]EUR-Lex is the European Union's official web portal, offering free and comprehensive access to EU legislation and legal documents

Case law parsing forms a separate stage of the pipeline, though one coupled to the acts: the references are taken from the document information tab that EUR-Lex publishes alongside each act, which lists the judgments interpreting it and the provision each one concerns, restricted to concluded judgments; Case law parsing is therefore performed only once the act graph has been validated and stored. The validation of both is discussed in Chapter 6.

4.1.1 Act

Every act in EUR-Lex is identified by its CELEX[2] key, This identifier is the entry point of the whole pipeline.

The system can parse several acts in one run, given a list of CELEX numbers: the browser fetcher retrieves the documents and passes them to the orchestrator, which hands each one to the EUR-Lex parser to extract nodes and relationships, validate them and build the graph. The retrieved files are stored in a local cache, which also serves as the reference for graph validation.

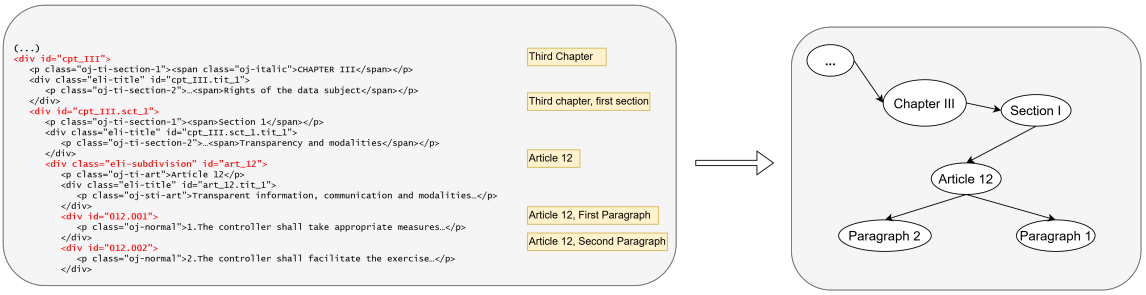


Figure 4.2: Document structure during parsing phase

As mentioned previously, the documents will be parsed according to their hierarchical structure. In the HTML, several divisions have been numbered with unique IDs, representing the elements of an act: chapters, sections, articles, paragraphs, recitals and annexes. This structure may or may not contain some sections, but the basic structure remains the same. Lastly, entities and relationships will be shipped to the Neo4j database in a single transaction to guarantee data integrity. After that, embeddings will be calculated for the most important entities. To guarantee faster queries, indexes of the most frequently used nodes are also created.

[2]CELEX number is a unique identifier given to a document (act, judgment, etc.) and is universal across EUR-Lex document languages.

4.1.2 Case Law (CJEU)

The case law document does not have the same structure as general acts. This is why the parsing system extracts the most generic parts of the document, such as the title, topics, sections and paragraphs (within each section), while maintaining the depth of the document in order to represent it in a graph.

This choice is also informed by corpus-linguistic studies of the structure of CJEU judgments. Trklja [39] shows that the sectional structure of CJEU judgments is not stable over time. Through a diachronic analysis of the EUCLCORP corpus (9,434 judgments between 1953 and 2016), they demonstrate that while sections such as *Grounds* and *Decision on Costs* occur with the highest consistency throughout the whole period, others (*Summary*, *Subject of the Case*, *Parties*, *Operative Part*) have been progressively dropped since the late 1990s and early 2000s. Even where headings are present, they are not applied systematically; for example, subsections within *Grounds* such as *Arguments of the Parties* appear only in some post-2000 judgments. Given this variability, hard-coding a fixed heading-to-node mapping would result in a brittle parser across decades of case law. Therefore, a generic, depth-preserving extraction strategy is adopted that can adapt to the presence, absence or renaming of individual sections.

Dealing with legacy and modern schema. At first glance, parsing case law seems straightforward. EUR-Lex judgements display headings and paragraphs via explicit CSS classes (e.g. `title-grseq-2`), which appear to directly encode the position of each section within the document hierarchy. However, a closer look at judgments from different years reveals that this is not the case: the same logical section can be tagged with different classes depending on the period. More importantly, the same class name does not correspond to a fixed hierarchical level across documents.

Two broad naming conventions coexist in the corpus. Judgments up to roughly 2018 use unprefixed classes such as `title-grseq-2` and `title-grseq-3` for their heading levels (*legacy schema*); more recent judgments prefix the same class family with `coj-` and additionally introduce `coj-sum-title-1` for what used to be the outermost level (*modern schema*).

Document Section	**C-645/19 (2021)**	**C-131/12 (2014)**
Legal context	`coj-sum-title-1`	`title-grseq-2`
European Union law	`coj-title-grseq-2`	`title-grseq-3`

Table 4.1: Mapping of the same logical sections to different HTML class names across judgments

Table 4.1 illustrates the shift for the section *Legal context*: in C-645/19 (2021) it is tagged `coj-sum-title-1` and its child *European Union law* is `coj-title-grseq-2`; in C-131/12 (2014) the very same pair of sections is tagged `title-grseq-2` and `title-grseq-3` respectively.

A per-document, depth-preserving strategy. This approach makes no assumptions about the meaning of a class in the abstract. For each document, the parser only collects the heading classes that occur in it and orders them according to the intrinsic rank encoded in the class name itself (the numeric suffix, with `sum-title-1` always ranked above `title-grseq-n`), and then renumbers them consecutively starting from zero. Therefore, the resulting depth is local to the document being parsed. The class that appears first in a given judgement becomes depth 0; its immediate children become depth 1, and so on regardless of what that same class was called or meant in a judgement published a decade earlier. This is how the parser addresses the structural heterogeneity of the sources that RQ1 refers to: no assumption about a class is carried from one judgment to the next.

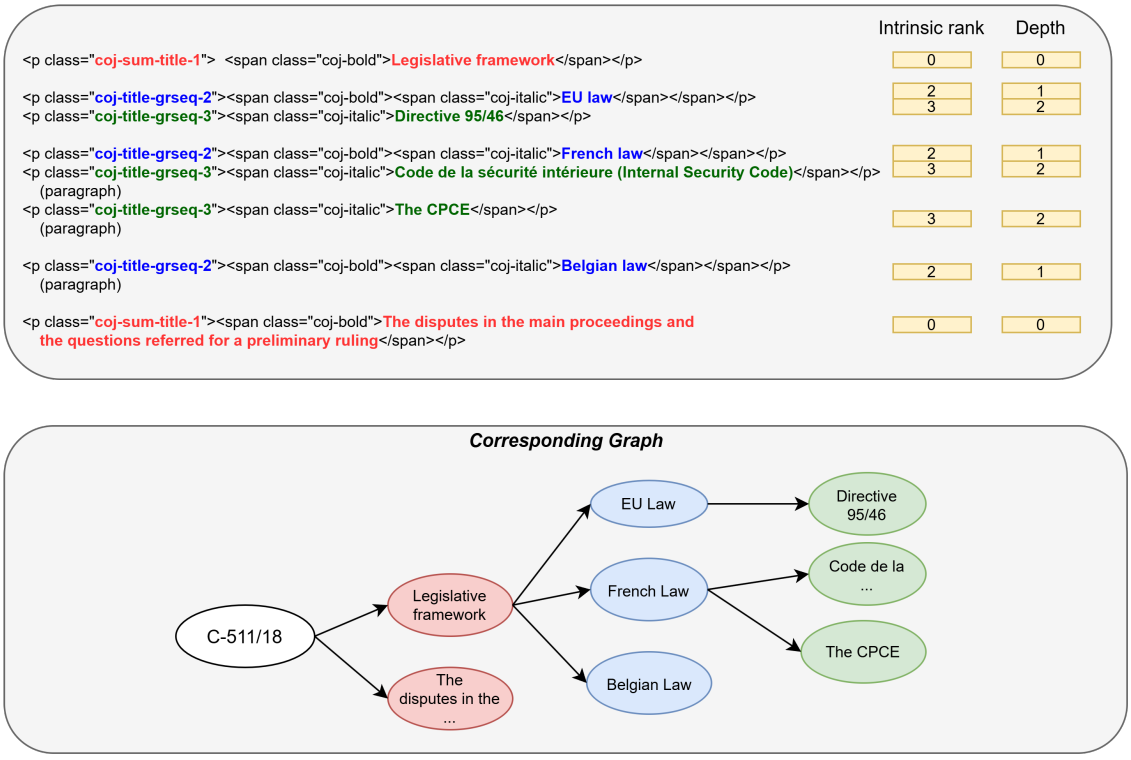


Figure 4.3: A modern schema document example representing intrinsic rank, depth and final graph.

4.1.3 Topics

Topics nodes were created to improve retrieval. The first idea we had was to assign topics to nodes to filter the graph and include only relevant parts. We opted for a zero-shot method called ASKE (Automated System for Knowledge Extraction) [5], which is an automated process for knowledge extraction that starts with a list of concept labels and expands them against the corpus. The initial list was generated by us and comprised concepts from various categories; the other part was taken from paragraph chunks. This was done to enable terminology enrichment to work not just on a small set of categories, but also to identify cross-category terms that could be useful for filtering.

Our seed contained 97 labels, deliberately phrased so that each one names a subject as a specific act frames it, for example *data minimisation principle* or *right of access by the data subject*, rather than a bare term such as *data* or *access*. Acts share a vocabulary while regulating different things, a generically phrased label attracts text from all of them at once. Anchoring each label to the act that frames it was the countermeasure.

Before the cycle runs, each paragraph is split into sentences and lemmatised [3], giving *chunks* small enough to fit the fixed window of the embedding model. The first sentence of a paragraph is discarded, because in EUR-Lex markup it usually carries only the paragraph number. Chunks and concept labels are then projected into the same space by a sentence transformer (`all-MiniLM-L6-v2`).

Each generation of the cycle applies four phases.

Chunk classification. The cosine similarity between every chunk and every active concept is computed, and a chunk is assigned to each concept it exceeds the threshold α with. The assignment is zero-shot: a chunk may land under several concepts, or under none.

Deactivation. A concept that classified no chunk is marked inactive and takes no further part in the cycle. This keeps a seed label foreign to the corpus from being enriched into noise.

Terminology enrichment. For each surviving concept, candidate terms are extracted from the chunks it classified using TF-IDF over unigrams and bigrams. Each candidate is given its WordNet [4] definition, that definition is embedded, and the term is scored by how close its definition sits to the concept. The score carries a discriminative penalty, $\sigma(w, c_i) - 0.5 \max_{j \neq i} \sigma(w, c_j)$, which demotes terms sitting equally close to every concept. The top γ terms clearing β join the concept's vocabulary.

[3] a process that transforms inflected words (such as conjugated verbs or plural nouns) into their basic dictionary form, called a lemma

[4] WordNet is a large lexical database of English. Nouns, verbs, adjectives and adverbs are grouped into sets of cognitive synonyms (synsets), each expressing a distinct concept

Concept derivation. The terms inside a concept are clustered by affinity [5] propagation. Each cluster that separates cleanly becomes a new concept, labelled by the term at its centroid. This is the step that lets the layer grow beyond its seed.

We ran the cycle for 15 generations with $\alpha = 0.4$, $\beta = 0.4$ and $\gamma = 7$. The chunk-level assignments are then aggregated back up to the paragraph that produced them: a paragraph keeps its three highest-scoring topics, scored by the best of its chunks, and each becomes a `Topic` node linked to that paragraph. Table 4.2 follows one seed through a single generation to make the mechanism concrete.

Phase	**Result for the seed "data minimisation principle"**
Classification	Article 5(1)(c) GDPR, "personal data shall be adequate, relevant and limited to what is necessary in relation to the purpose for which they are processed", clears α and is assigned. So do chunks of Article 25 (data protection by design and by default) and Recital 39.
Deactivation	The concept classified chunks, so it stays active. A seed with no assignment at all would be dropped here.
Enrichment	TF-IDF over the classified chunks proposes *purpose*, *necessity*, *adequate*, *personal data*, *processing*. *Processing* sits close to almost every concept in the layer and is demoted by the discriminative penalty; *necessity* and *purpose limitation* survive and join the vocabulary.
Derivation	Clustering the enriched vocabulary separates the terms about *how much* data is collected from those about *how long* it is kept, and the second cluster spawns a new concept labelled from its centroid, close to *storage limitation*.

Table 4.2: One ASKE generation applied to a single seed concept. The four phases assign chunks, discard sterile concepts, extend the concept's vocabulary, and split it when its terms fall into distinct clusters.

This layer was part of the model from the outset, but was later dismantled. Filtering the corpus by topic did not reliably select paragraphs that answered the question, and the benefit of searching the corpus indirectly was too small to justify the layer.

[5] affinity propagation is a clustering algorithm based on the concept of "message passing" between data points

4.2 Data Model

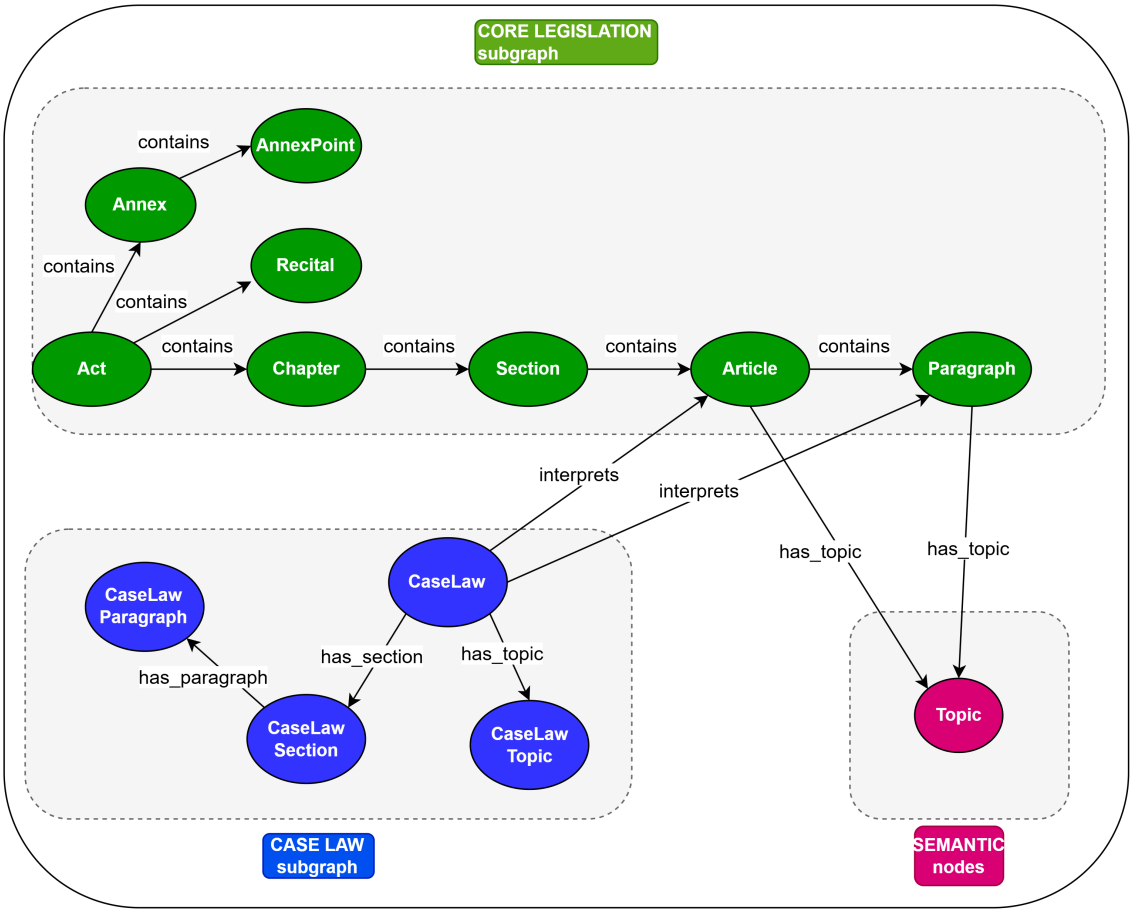


Figure 4.4: ThemisRAG data model

The data model features three subgraphs. The first one represents the core legislation and the structure of the Act, as parsed by our system in a hierarchical manner. Each node has a unique ID, which is defined by the referring Act and the part it represents in the document. The third, the semantic layer of `Topic` nodes, belongs to the topic-based design of Section 4.1.3 and is no longer used by retrieval.

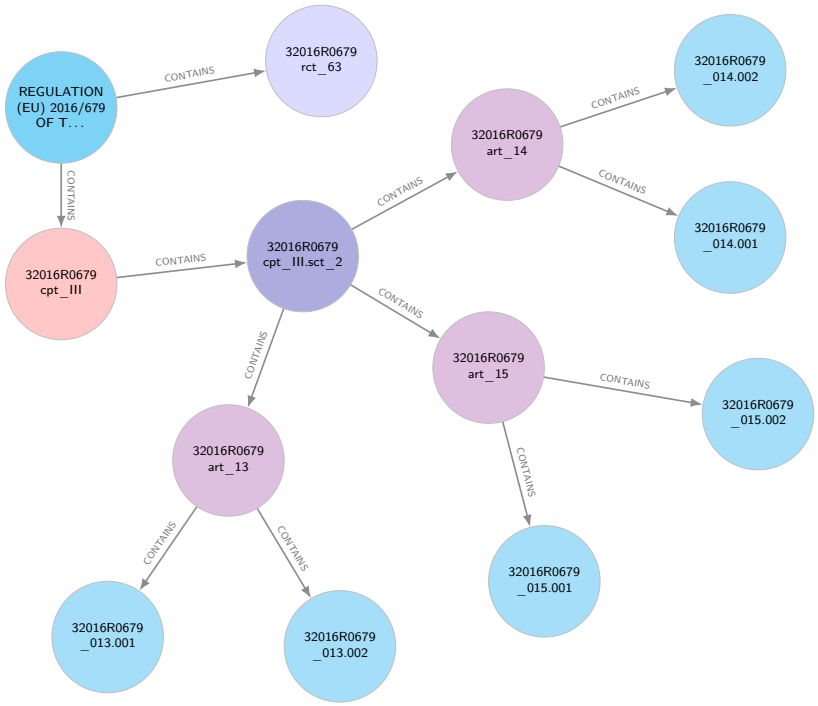


Figure 4.5: This is an example of a core legislation subgraph representing the tree structure of a piece of a document, from act to paragraph.

Table 4.3: Identifier schemes for the legislation nodes. Every identifier combines the CELEX number of the act with the element's own identifier, so that numbering that repeats across acts does not collide.

Node	**Scheme**	**Example**	**Denotation**
`Act`	`<celex>`	`32016R0679`	GDPR
`Chapter`	`<celex>cpt_<roman>`	`32016R0679cpt_III`	Chapter III, *Rights of the data subject*
`Section`	`<chapter>.sct_<n>`	`32016R0679cpt_III.sct_2`	Chapter III, Section 2, *Information and access to personal data*
`Article`	`<celex>art_<n>`	`32016R0679art_15`	Article 15, *Right of access by the data subject*
`Paragraph`	`<celex>_<art:3>.<par:3>`	`32016R0679_015.001`	Article 15, paragraph 1
`Recital`	`<celex>rct_<n>`	`32016R0679rct_63`	Recital 63
`Annex`	`<celex>anx_<roman>`	`32024R1689anx_III`	AI Act, Annex III
`AnnexPoint`	`<annex>.p_<pos:3>`	`32024R1689anx_III.p_007`	AI Act, Annex III, seventh point[†]

[†] The annex point identifier is *positional*: the published markup exposes no element per point, so `p_007` denotes the seventh textual unit of the annex, not necessarily the point numbered 7. The citation in the form a lawyer writes is kept separately, in the `point_label` property.

Moreover, every textual node (articles, paragraphs, recitals, annex points and judgment paragraphs) carries a *textEmbedding* property that allows the system to perform a dense search with the following configuration:

- Embedding model is `text-embedding-3-small` from OpenAI
- Embedding dimension equal to 1536
- similarity type cosine
- Quantization enabled (default)

When it comes to the case law subgraph, we reuse the same concept to keep the structure as generic as possible, matching just the sections that a general document is usually composed of, such as sections, paragraphs of text, titles and topics. The *CaseLaw* node represents the parent node to which every subnode refers. The core legislation references, *CaseLaw Section* and *Paragraph*, are the structure already seen in Figure 4.3 and widely discussed. *CaseLaw Topics*, on the other hand, are the list of topics displayed at the top of each case law document. This could be useful if we were to design a powerful topic-based filter.

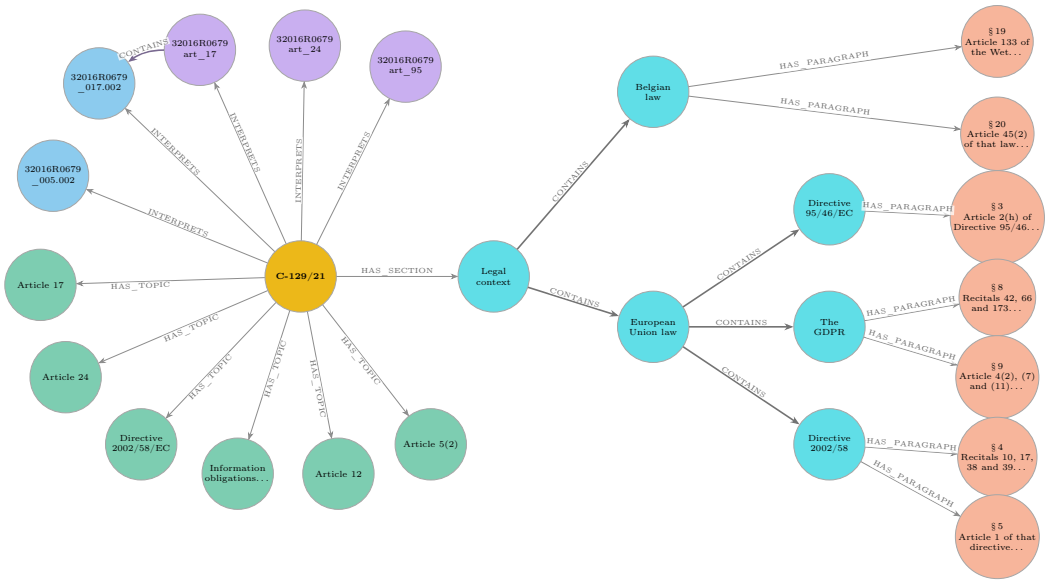


Figure 4.6: The subgraph of judgment C-129/21. The judgment is anchored to the legislation at two different granularities: to whole articles (`art_17`, `art_24`, `art_95`) and to individual paragraphs (`005.002`, `017.002`); note that this is just a piece of the entire tree. On green nodes there are *CaseLaw Topics*, on the right is the entire structure of the case law, featuring *CaseLaw Section* in blue and *CaseLaw Paragraph* in orange.

The next two subsections discuss two extensions that were only modelled in theory, as there was neither the opportunity nor the resources to study them separately.

4.2.1 Modelling Amendments and Repeals

An amendment does not rewrite a text. It closes the validity of one wording and opens another. A graph that overwrites a `text` property therefore loses the event that matters. The standard way to model this, and the one adopted by SAT-Graph [11], is to separate the *provision* from its *wordings*, and to turn the change itself into a node.

A real case makes this concrete. Regulation (EU) 2026/1744, in Article 1, point (2)(a), provides that in Regulation (EU) 2024/1689 "Article 2 [...] paragraph 2 is replaced by the following". The new wording applies from 27 July 2026. In our schema the affected provision is a single node, the `Paragraph` identified by `32024R1689_002.002`.

Figure 4.7 shows the result. The provision node keeps its identifier and its place in the `CONTAINS` hierarchy, but no longer carries text. Two version nodes hang below it, each holding one wording, its embedding, and a validity interval. An `Amendment` node records the event and connects three ends: the version it terminates, the version it introduces, and the provision that commanded the change, here Article 1 of the amending regulation. That third edge is what makes provenance answerable. Without it the graph records *that* the paragraph changed, not *which* legislative command changed it.

Repeal is the same mechanism with one end missing: the `Amendment` terminates a version and introduces none, which is what distinguishes a repealed provision from one merely superseded. Populating these nodes is largely a parsing problem rather than an inference problem, because EUR-Lex consolidated texts already mark each passage with the instrument that produced its current wording.

The cost of the model is that text moves off the nodes that currently hold it, which touches the loader, the vector indexes and retrieval. A lighter variant keeps the current wording on the provision node and stores only superseded wordings separately. It is additive and cheap, but it gives up version-level queries over the upper hierarchy, so questions such as "what did Chapter II say on 1 January 2025" remain out of reach.

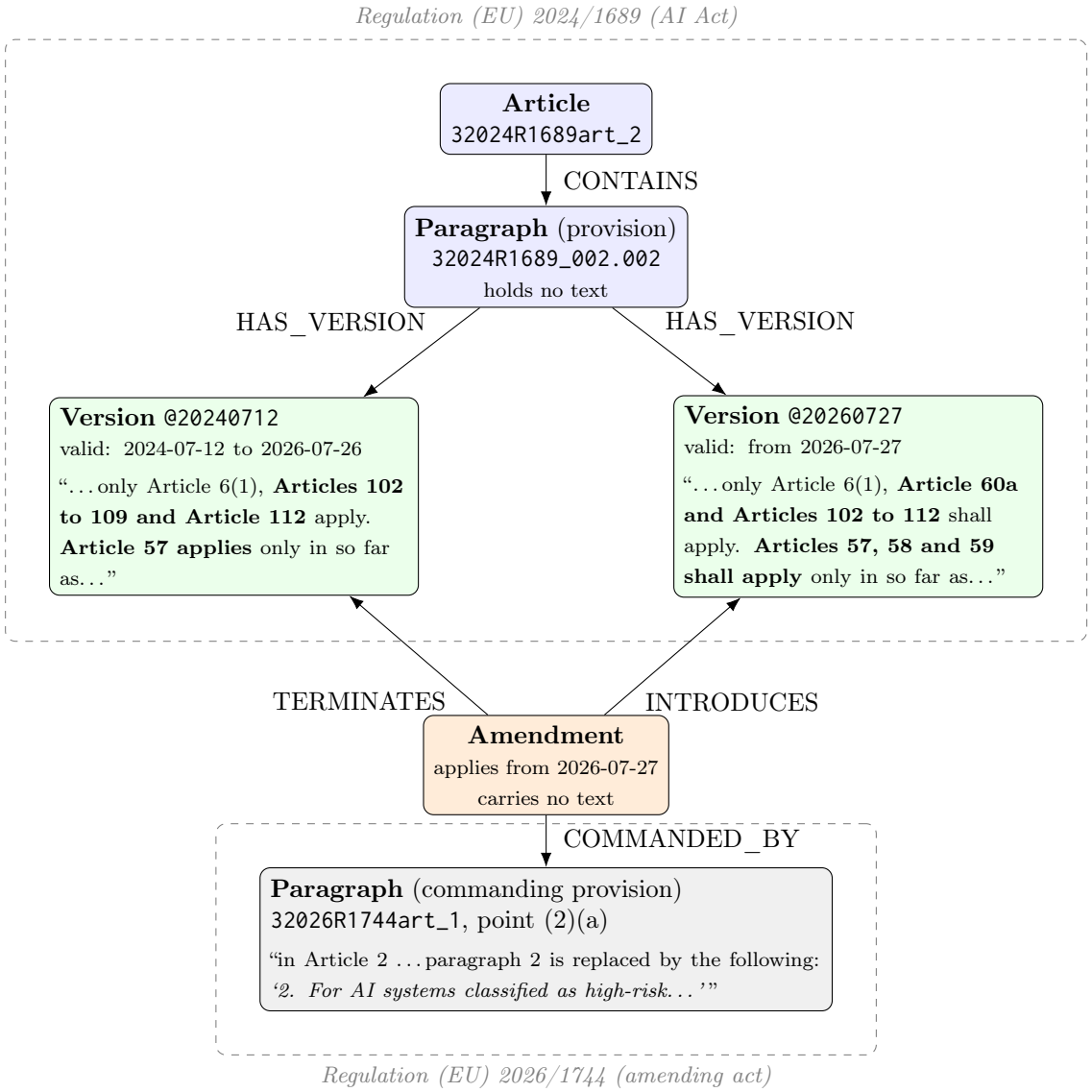


Figure 4.7: Amendment of Article 2(2) of the AI Act by Regulation (EU) 2026/1744. The provision keeps its identifier and holds no text; each wording becomes a version node with its own validity interval, and both remain in the graph. The `Amendment` node carries no text and bridges two documents, linking the terminated version, the introduced version, and the provision that commanded the change.

4.2.2 Multilingualism

EU legislation is enacted in twenty-four languages, all equally authentic. No version is a translation of a privileged original. This guarantee has a structural counterpart, which we verified directly. The GDPR in English, Italian and German gives three documents that differ in wording but expose the same structural anchors: articles `art_1` to `art_99` and recitals

`rct_1` to `rct_173` in all three. Only the language segment of the request URL changes.

Our identifiers are therefore already language-independent, though not by design. An identifier such as `32016R0679art_12` is built from the CELEX number and the structural position alone, so it names the same provision in every language version. Parsing a further language would require no change to the parser. It would attach new text and new embeddings to nodes that already exist.

The model still has no notion of language. Each textual node carries a single `text` property with no language beside it, the vector indexes cover one monolingual embedding space, and retrieval has no preferred or fallback language. Supporting a second language would require a language property on textual nodes, per-language vector indexes, and a cross-lingual embedding model.

We did not attempt this. Multilingual retrieval was never an object of this work, and the constraint was one of resources rather than of architecture. It is worth noting that the two extensions discussed in this section are the same extension seen twice. Both ask for a layer between a provision and its concrete realisation: one indexed by time, the other by language. SAT-Graph provides exactly this, separating a temporal version of a provision from the wording that expresses it in a given language, so a design that adopts one is already most of the way to the other.

Chapter 5

ThemisRAG: Proposed Approach

This chapter presents ThemisRAG, the retrieval architecture built on the knowledge graph described in Chapter 4. Every question goes through the same pipeline (Figure 5.1). It is first classified, to decide which acts it concerns and whether it asks what the law says or how the courts have interpreted it (Section 5.1). It then follows one of two flows: the definitional flow retrieves from the acts alone (Section 5.2), while the interpretative flow also retrieves the case law that interprets them (Section 5.4). Finally, the generator writes the answer from the retrieved passages (Section 5.5).

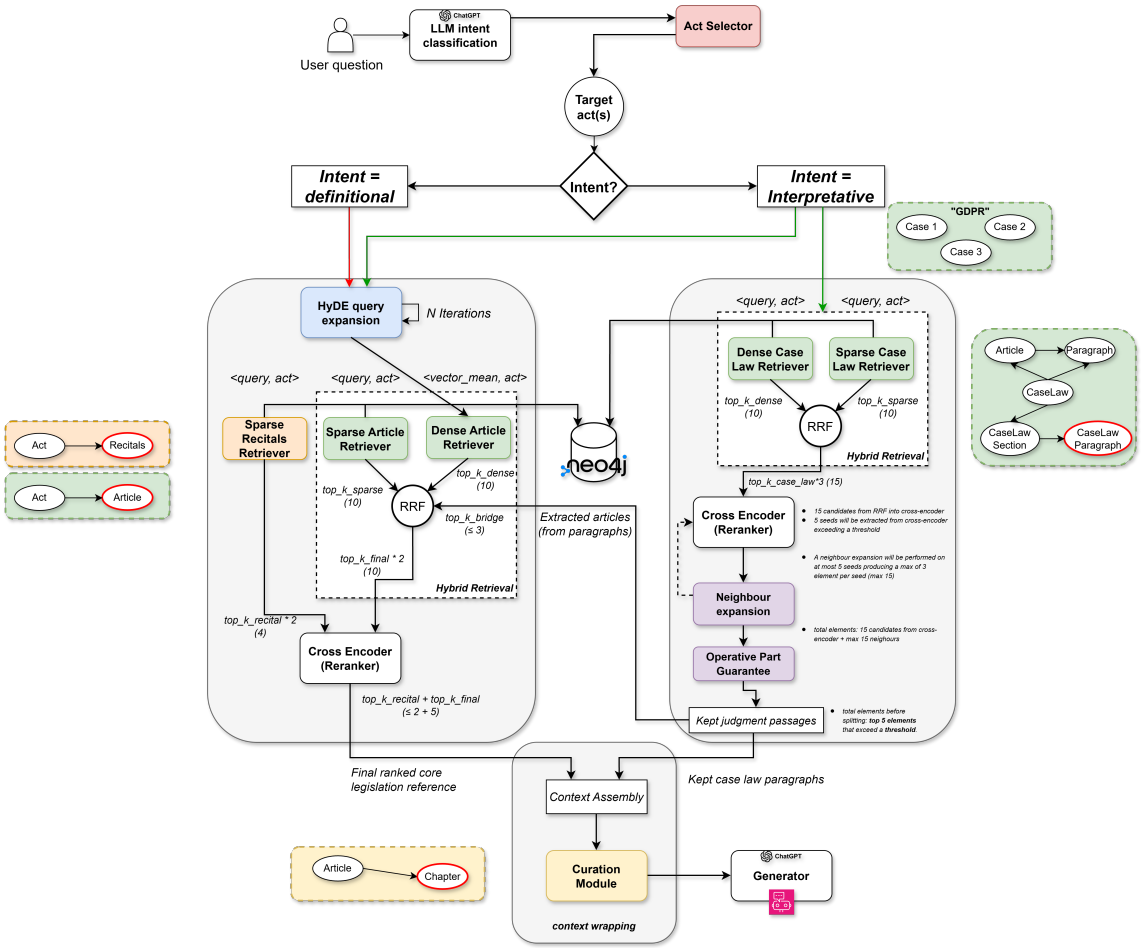


Figure 5.1: Hybrid RAG architecture

Every step of the pipeline is configurable. Table 5.1 lists the parameters of the retrieval steps, which the following sections discuss one by one.

Parameter	Default	Range	Role
Query expansion			
`use_hyde`	on	bool	Expand the query into hypothetical legal passages before dense search
`hyde_iterations`	3	1–5	Hypothetical passages sampled and averaged into one query vector
`use_query_decomposition`	off	bool	Split a compound question and retrieve per sub-question
Retrieval breadth			
`top_k_dense`	10	1–30	Passages requested from each vector index
`top_k_sparse`	10	1–30	Passages requested from each BM25 index
Fusion and reranking			
`rrf_k`	60	1–120	Reciprocal Rank Fusion constant
`use_reranker`	on	bool	Cross-encoder second stage; when off, fusion order is preserved
`top_k_final`	5	1–15	Articles admitted to the context, as guaranteed slots
Recitals			
`use_recitals`	on	bool	Retrieve recitals alongside articles
`top_k_recitals`	2	0–10	Recitals admitted; the candidate pool is twice this
`recital_score_threshold`	0.3	0–1	Minimum cross-encoder score for a recital
Case law (interpretative flow)			
`use_case_law`	on	bool	Search judgment paragraphs on INTERPRETIVE queries
`top_k_case_law`	5	0–10	Judgment passages admitted; the candidate pool is three times this
`case_law_score_threshold`	0.3	0–1	Minimum cross-encoder score for a judgment passage
`case_law_neighbours`	2	0–5	Paragraphs read after each hit, and one before; 0 disables expansion
`guarantee_operative`	on	bool	Grant the operative part an additional slot when none was selected
`top_k_bridge`	3	0–10	Cited provisions fused into the article ranking; 0 disables the bridge

Table 5.1: Configurable parameters of the hybrid retrieval method. Each is declared with its type, default and admissible range, so it can be adjusted from the interface or swept by the evaluation harness without a code change.

5.1 LLM Intent Classification

The graph offers two kinds of sources to answer a question: the legislation and the case law that interprets it. The first step of the pipeline decides which of the two the question needs. It assigns the question to one of two categories: *definitional*, for questions that an act can answer on its own, and *interpretative*, for questions that also require the judgments construing the act. In the same step it selects the acts the question concerns, so that retrieval is confined to them.

A single LLM call returns the intent and a relevance score for each act. An act is kept if its score exceeds a threshold, and only the kept acts are searched in the following steps. The classification prompt uses few-shot prompting: it includes example questions with their intent and the acts they concern.

Question

```
"Who is the 'controller' under the GDPR?"
```

Answer

```
RawClassification (LLM output)
{
  "intent": "DEFINITIONAL",
  "act_relevances": [
    {"celex": "32016R0679", "relevance": 0.97},   // GDPR
    {"celex": "32023R2854", "relevance": 0.12},   // Data Act
    {"celex": "32022R0868", "relevance": 0.10},   // Data Governance Act
    {"celex": "32024R1689", "relevance": 0.05}    // AI Act
  ],
}

QueryClassification (after thresholding)
{"intent": "DEFINITIONAL", "acts": ["32016R0679"]}
```

Figure 5.2: A definitional question that concerns only the GDPR.

Question

```
"Is the administrator of a social network fan page a joint controller together with the platform
operator?"
```

Answer

```
RawClassification (LLM output)
{
  "intent": "INTERPRETIVE",
  "act_relevances": [
    {"celex": "32016R0679", "relevance": 0.93},   // GDPR
    {"celex": "32023R2854", "relevance": 0.14},   // Data Act
    {"celex": "32024R1689", "relevance": 0.08},   // AI Act
    {"celex": "32022R0868", "relevance": 0.06}    // Data Governance Act
  ],
}
QueryClassification (after thresholding)
{"intent": "INTERPRETIVE", "acts": ["32016R0679"]}

Target case law:
MATCH (cl:CaseLaw)-[:INTERPRETS]->(target)
MATCH (act:Act)-[:CONTAINS*]->(target)
WHERE act.id IN $acts
```

Figure 5.3: An interpretative question that concerns only the GDPR.

For an interpretative question, the selected acts also delimit the case law. The Cypher query in the last rows of Figure 5.3 retrieves every judgment linked by `INTERPRETS` to a provision contained in one of the selected acts: this set is the candidate pool of the interpretative flow (Section 5.4).

5.2 Definitional Intent Flow

The definitional flow retrieves from the selected acts alone, and its retrieval unit is the whole article. It combines dense and sparse search. The survey in [18] reports that sparse retrievers often outperform dense ones on legal text, where exact word correspondence matters; combining the two joins the precision of the former with the contextual depth of the latter, as in [24].

5.2.1 Hypothetical Document Embeddings (HyDE)

A source of degradation in dense retrieval is the stylistic asymmetry between the two sides of the retrieval problem. Not only are a user question and a legal provision different in content, they also differ in register, length and syntactic form: the former is short and colloquial, often framed as a factual scenario, while the latter is prescriptive, formulaic and lexically dense. Since a bi-encoder embeds both into the same space, this mismatch can place genuinely relevant provisions at a considerable distance from the query vector, regardless of their relevance.

Hypothetical Document Embeddings (HyDE) [15] mitigates this asymmetry by shifting the comparison from question–document to document–document. Rather than embedding the query directly, an LLM that follows instructions is prompted to generate a hypothetical passage that answers it: a text that, while not necessarily factually correct, reproduces the register and terminology of the target corpus. Retrieval is then carried out using the embedding of this synthetic passage so that both sides of the similarity computation now share the same stylistic distribution.

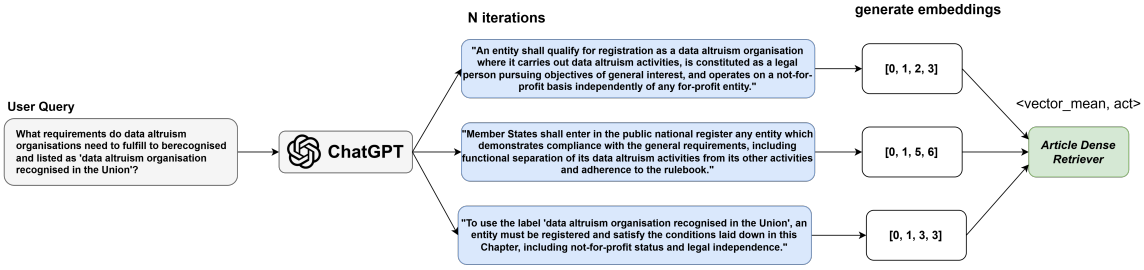


Figure 5.4: Here is an example of HyDE using a dataset question.

To reduce the influence of any individual generation, several hypothetical passages are sampled at a temperature of 0.7. This setting is non-deterministic by design: sampling diversity is what makes averaging informative, since idiosyncratic or hallucinated details vary across samples, while the shared domain-specific signal does not. Each passage is encoded independently and the resulting vectors together with the embedding of the original query, which anchors the representation to the actual question are averaged to create a single dense vector. A nearest-neighbour search is then performed against this averaged representation.

This single-pass procedure requires no additional supervision, making it attractive in the legal domain where annotated query–document pairs are costly to obtain. The cost is one generation step per query, adding latency ahead of retrieval.

5.2.2 Hybrid Retrieval

The hybrid retrieval branch operates on two complementary signals over the same retrieval unit, the whole article. The dense signal is served by a native Neo4j vector index over the Article nodes, queried through their `textEmbedding` property; the index is created at graph-build time and the retriever only binds to it via *LangChain's Neo4jVector* wrapper. The sparse signal is provided by *LangChain's BM25Retriever*[1], whose corpus is not a persistent index but is materialised at query time from the graph itself, through a parametrised Cypher query that walks `(:Act)-[:CONTAINS*]->(:Article)` and returns each article's text together with its act, title and chapter. The resulting BM25 index is cached per combination of target acts, so the construction cost is paid once per act set rather than once per query.

Each branch returns a ranked list of at most ten passages (*top_k_dense* and *top_k_sparse*, both configurable). The cap is exact on the sparse side, which is already restricted to the target acts by construction, but only an upper bound on the dense side: the vector index spans all acts, so out-of-scope hits are discarded after the search by matching the CELEX prefix of their identifier.

Recitals follow a deliberately asymmetric path: they are retrieved by BM25 alone, over a separate cached index built the same way, and never contribute a dense list. A recital is a short, self-contained paragraph of stated legislative intent, well served by lexical matching, and the hypothetical passages used to expand the dense query are written in the register of an article rather than of a recital.

[1] BM25 (Best Matching 25) is a ranking algorithm used by search engines to score how relevant a specific document is to a given search query

5.2.3 Reciprocal Rank Fusion (RRF)

Retrieval produces two ranked lists, one sparse and one dense, alongside a third one discussed in Section 5.4. They cannot be merged by adding their scores: cosine similarity is bounded and, over a corpus of four regulations sharing a single stylistic register, heavily compressed, whereas BM25 is unbounded above and its magnitude depends on query length and corpus statistics. A score of 0.82 in one list and 14.3 in the other carry no common meaning.

Reciprocal Rank Fusion [9] sidesteps the problem by discarding magnitude and keeping the only quantity both systems express in the same unit: the position a document occupies in its own list. Given a document d and a set of ranked lists $\mathcal{L}$, its fused score is

$$\mathrm{RRF}(d) \;=\; \sum_{L \in \mathcal{L}} \frac{1}{k + \mathrm{rank}_L(d)} \tag{5.2.1}$$

where $\mathrm{rank}_L(d)$ is the one-based position of d in L and the sum runs only over the lists in which d appears. Table 5.2 applies the formula, with $k = 60$, to the top of the two lists returned for the query *"when is a data protection impact assessment mandatory?"*.

		Rank			
Article	Title	Dense	Sparse	RRF(d)	Fused
Art. 35	Data protection impact assessment	1	1	$\frac{1}{61} + \frac{1}{61} = 0.0328$	1
Art. 36	Prior consultation	2	3	$\frac{1}{62} + \frac{1}{63} = 0.0320$	2
Art. 9	Special categories of data	—	2	$\frac{1}{62} = 0.0161$	3
Art. 24	Responsibility of the controller	3	—	$\frac{1}{63} = 0.0159$	4

Table 5.2: Fusion of the dense and sparse lists for a single query. A dash marks a document absent from that list, contributing nothing to the sum.

The table contains the whole design. Article 35, returned first by both retrievers, wins on agreement. Article 36 is the informative case: it is first in neither list, yet it outranks Article 9, which *was* second in the sparse list, because two mediocre placements sum to more than one good placement in isolation. That inversion is the point of the formula. It follows from the size of k: within a list, first and tenth place differ by barely 13% ($\frac{1}{61}$ against $\frac{1}{70}$), so position carries little weight, whereas appearing in a second list roughly doubles the score.

Being a plain sum, the fusion is also agnostic to how many lists it receives: the branch of 5.4 joins as one additional ranked list rather than as an alternative retrieval path, and when it is empty the fused ranking is identical to the one produced by the sparse and dense branches alone. The resulting list, truncated to a shortlist, is what the reranking stage receives.

5.2.4 Cross Encoder

Fusion yields a single ranked list, but its ordering rests on positional evidence alone: RRF records that two retrievers agreed on a document, not how well that document answers the question. The list is therefore truncated to a shortlist and rescored by a cross-encoder [30]. Unlike the bi-encoder behind the dense branch, which embeds query and document independently and compares the two vectors only at the end, a cross-encoder processes the pair (q, d) as a single input, so a candidate is read in light of the question instead of being compressed beforehand into a query-agnostic vector. Nothing can be precomputed, and scoring n candidates costs n forward passes: hence the stage runs on a shortlist of a few dozen documents rather than on the corpus.

Reranking restores, on a single scale, the magnitude that RRF discards. Since every score now comes from one model, absolute thresholds become meaningful: the top `top_k_final` articles are kept as guaranteed slots, while recitals, annex points and judgment paragraphs are admitted only above a per-branch minimum, so weak matches are dropped instead of padding the prompt. Each candidate is scored on its full text.

5.2.5 Model Choice

The reranker is loaded through the `CrossEncoder` interface of `sentence-transformers`, and the checkpoint is selected by the `RERANK_MODEL` environment variable so that alternatives can be compared without touching the code. The deployed default is `zeroentropy/zerank-2`, a cross-encoder of roughly four billion parameters built on Qwen3-4B and released under Apache 2.0.

The choice is driven by the domain. Legal relevance is seldom lexical: the provision answering *"who must carry out an impact assessment"* is the one that assigns the duty, not the one that mentions assessments most often, and separating the two requires reading the article as an argument rather than as a bag of terms. This is where general-purpose rerankers degrade most, and where zerank-2 reports its widest margin. On the legal split of its evaluation suite it scores 0.664 NDCG@10, against 0.557 for Gemini 2.5 Flash and 0.526 for Cohere Rerank-3.5, a relative improvement of roughly 20% and 26% respectively; the same margin does not appear on finance, where all three models are within two points of one another.[2]

Two configuration choices make a model of this size practical on a single 16 GB GPU. Weights are loaded with `dtype="auto"`, which respects the checkpoint's native `bfloat16` precision instead of promoting it to `float32`: the latter roughly doubles the memory footprint and, on this hardware, spills to host memory. The model is pinned to CUDA and instantiated

[2]Figures published by the model authors [45]; the legal domain is the widest of the seven splits reported.

once at class level, so its weights are loaded a single time and shared across every query and every branch.

5.3 Context Wrapping

Reranking leaves a set of passages, not a prompt. Before synthesis, each passage is decorated with a bracketed source header naming the instrument and the locator a lawyer would actually cite: `[Data Governance Act, Chapter II — Scope, Article 3]` for a provision, `[C-645/19, The first question, para. 71]` for a judgment paragraph. The header is not cosmetic. The synthesis prompt forbids recalling an article number from memory and requires every citation to be copied verbatim from a header present in the context, so the decoration is what makes an answer traceable to a specific provision and what the attribution stage later matches against.

The decorated passages are then concatenated, in branch order, into a single context block. Articles occupy guaranteed slots and come first; recitals follow, admitted only above their own threshold. The passages contributed by the interpretative branch, and the reason they are placed last, are discussed in Section 5.4. The block is finally interpolated into the answer-synthesis template together with the question and an optional guidance string, produced by the module described below.

5.3.1 Curation Module

The curation module is an optional stage that sits between retrieval and synthesis. It removes such passages using a cheap LLM call before the context is assembled.

It filters, but never rewrites. The passages are presented to the model, numbered and headed. The model then returns a JSON object containing indices of the passages to keep and the subset that governs[3] the question, as opposed to passages that are retained as cross-reference or exception support. Selecting by index rather than generated text is a deliberate constraint: statutory wording and article numbers reach synthesis exactly as they left the graph. This means that the stage cannot introduce a paraphrase that a later citation would be built on.

[3] *Governing* is used in its legal sense: the provision whose subject matter *is* what the question asks about, and which therefore supplies the applicable rule. It is narrower than *relevant*, since a cross-reference, an exception or a clarifying recital may be necessary to answer correctly without governing the question. A judgment is never the legal basis: it construes a provision rather than enacting one.

Esempio

```
INPUT  (passage texts elided; only the source headers are shown)

  [0] [GDPR, Chapter IV - Controller and processor, Article 35 -
       Data protection impact assessment]  1. Where a type of processing ...

  [1] [GDPR, Chapter IV - Controller and processor, Article 24 -
       Responsibility of the controller]   1. Taking into account the ...
  [2] [GDPR, Chapter IV - Controller and processor, Article 36 -
       Prior consultation]                 1. The controller shall consult ...
  [3] [GDPR, Recital 91]                    (91) This should in particular ...

  [4] [GDPR, Chapter II - Principles, Article 9 -
       Special categories of personal data] 1. Processing of personal data ...

OUTPUT

  {"keep": [0, 3, 4], "governing": [0]}
```

Figure 5.5: Curation of the shortlist for *"when is a data protection impact assessment mandatory?"*. Article 35 states the condition under which the assessment becomes compulsory and is marked governing; Recital 91 elaborates its large-scale criterion and Article 9 is cross-referenced by Article 35(3)(b), so both are kept as support. Article 24, a general statement of the controller's responsibility, and Article 36, which governs what follows an assessment rather than when one is due, share the vocabulary of the question without governing it and are dropped.

The governing subset is used twice. The corresponding passages are moved to the front of the context and their headers are written into a guidance string instructing the answer stage to build on them and treat the remainder as support. The stage is fail-open, meaning that an empty selection or unparseable output causes the full retrieved set to be passed through unchanged. This is because dropping the one provision that answers the question would be unrecoverable downstream, whereas an extra passage would merely be noise.

5.4 Interpretative Intent Flow

Interpretative flows involve a new section of the entire graph system: case law. After the act classification, the pipeline will extract the relevant case law to provide the user with cases that might offer an interpretation of the requested core legislation.

5.4.1 Hybrid Retrieval

The role of hybrid retrieval and Reciprocal Rank Fusion (RRF) has been widely discussed in previous flows in Section 5.2. What is important here is understanding which nodes are involved in the retrievers in order to predict what will happen next.

Sparse retrieval uses a BM25 index built at query time, starting from the list of target acts and extracting case law with an `INTERPRETS` relationship to a core legislative part (article or paragraph). The system then traverses the depth of the case, extracting information from sections and paragraphs.

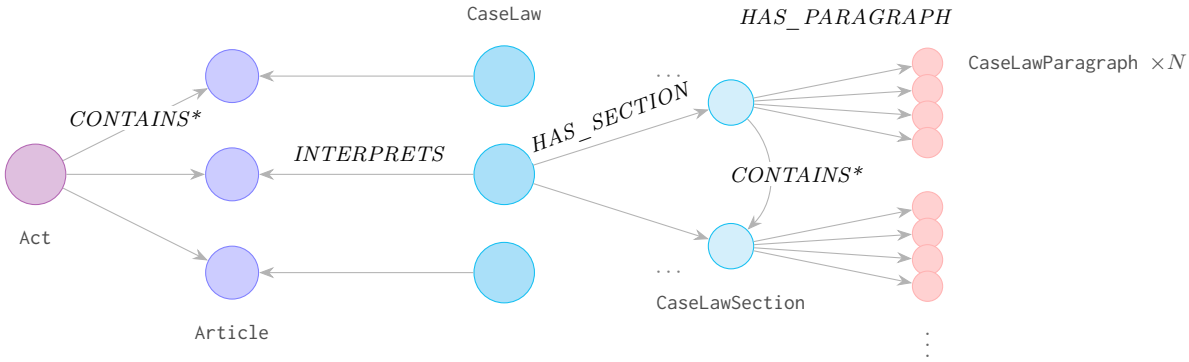


Figure 5.6: The set of candidates for the interpretative branch. From the classified act, the query reaches the provisions it contains. It then interprets them in the opposite direction to every judgment construing one of them. It also descends each judgment's section hierarchy to the paragraph level, which is the most reliable retrievable unit.

All selected paragraphs will be compiled into a document containing metadata for use in subsequent pipeline steps. This document will then be scored using BM25 based on its page content, and the results will be fed into Reciprocal Rank Fusion. The following is an example of the document structure:

Esempio

```
Document(
    page_content="It follows that the requirement, under Article 46(1) of the GDPR, "
                 "that individuals whose personal data are transferred to a third "
                 "country pursuant to standard data protection clauses must be "
                 "afforded a level of protection essentially equivalent to that "
                 "guaranteed within the European Union by that regulation, read in "
                 "the light of the Charter, cannot be circumvented.",
    metadata={
        "id": "62018CJ0311_par_105",
        "celex": "62018CJ0311",
        "case_number": "C-311/18",
        "number": 105,
        "is_operative": False,
        "section_heading": "Consideration of the questions referred",
        "type": "case_law",
    },
)
```

Figure 5.7: A judgment paragraph as passed to the retrievers: its text and the metadata used by the following steps.

The cross-encoder will now reorder the results and provide the final outcome.

5.4.2 Neighbour Expansion

The unit in which the matching did by retrieval is most precise is rarely the unit that carries complete meaning. A short passage is easy to rank correctly but difficult to understand while isolated, and a long passage is the opposite. Current RAG systems address this through a family of strategies, sentence-window retrieval and related auto-merging retrieval popularized through retrieval frameworks under the name *small-to-big retrieval* [27]. The idea of retrieving on a small unit and widening it is established in the information retrieval literature, where passage retrieval was introduced precisely for long documents whose relevant content is localized [34, 4].

This branch adopts the idea and adapts it to the way a judgment is written. Retrieval matches where the court *discusses* a question at length using the same vocabulary as the question. However, it misses where the court *answers* it in a single sentence that does not name anything because everything it refers to was named in the paragraphs immediately before.

Therefore, the expansion is driven by the document's rhetorical structure. Once the first reranking pass is complete, the passages with the highest scores that meet the threshold become *seeds*, at most `top_k_case_law` of them, and the paragraphs around each seed are collected.

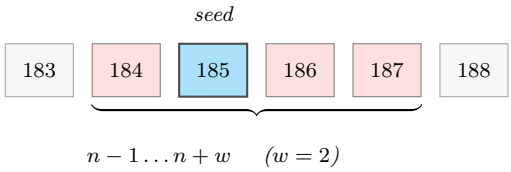


Figure 5.8: Neighbour expansion around a seed. The seed's own neighbours are generated arithmetically from its identifier: one paragraph before and w after, the asymmetry following the order in which a judgment argues.

Judgment paragraph identifiers are sequential by construction, `<celex>_par_<n>`, so given a seed numbered n and a window w the neighbours are obtained arithmetically as $n-1, n+1, n+2, \ldots, n+w$. The window is asymmetric rather than centred on the hit, because a judgment builds towards its conclusion: what precedes a passage is largely what has already been matched, while what follows it is where the point is resolved.

Unlike in sentence-window retrieval, neighbours are treated as candidates in their own right. They are scored using the same cross-encoder, merged into the same ranking, and are subject to the same threshold and cap. Consequently, a neighbour is only admitted if it outscores whatever it replaces, including its own seed. Consequently, the number of passages that reach the context remains unchanged, but their identity changes. An expansion that finds nothing better leaves the selection exactly as it was.

The expansion is *performed once* and is *not recursive*: a neighbour admitted to the ranking does not itself become a seed. A second round would expand the candidate set exponentially and, more importantly, would diverge with each iteration, moving further and further away from the point at which retrieval actually engaged with the question. A single round with $w = 2$ suffices, since a conclusion follows its reasoning within one or two paragraphs.

Example. Consider the query *"is the Privacy Shield a valid basis for transfers to the United States?"* against C-311/18. Reranking scores paragraph 185, which discusses the limitations United States law places on the rights of data subjects, at 0.71; it clears the threshold and becomes a seed. With $w = 2$ the branch generates `62018CJ0311_par_184`, `. . . _par_186` and `. . . _par_187`, resolves them in the paragraph set, and scores them.

Paragraph 186 is the one that matters: it states, in substance, that the Decision is therefore incompatible with Article 45(1) of the GDPR read in the light of the Charter. No retriever could have found it, since it contains neither *Privacy Shield*, nor *transfer*, nor *United States*, all of which were named in the paragraphs above it. Scored against the query, however, it reaches 0.88, higher than the seed that led to it, and enters the context ahead of it.

5.4.3 Operative Part Guarantee

A CJEU judgment concludes with the operative part, which consists of numbered statements in which the Court provides its response to the referred questions. Everything before this is reasoning; the operative part is the holding, which is the binding element to which other courts and institutions are bound. When it comes to answering questions about a case, it is therefore the most valuable part of the text.

It is also the passage retrieval is least likely to find. It is drafted as a formal recitation of the instrument under review rather than as a discussion of the problem, so it shares almost no vocabulary with a question expressed in ordinary legal terms. The operative part of C-311/18 states that *Commission Implementing Decision (EU) 2016/1250 is invalid*: a passage that answers the question of the validity of the Privacy Shield precisely, despite containing no mention of *Privacy Shield*, *transfer* or *adequacy*. Neither the sparse nor the dense retriever ranks it, and it never enters the candidate pool at all.

The branch therefore fetches it instead of searching for it. Its structural role is recorded on the node itself, in the boolean `is_operative` property read from the published judgment, so the passage can be addressed directly. After the threshold and the cap have been applied, if no operative passage is among the selected ones, the operative paragraphs of the highest-ranked judgment are read from the scoped set already held in memory, scored by the cross-encoder, and the best of them is granted a slot of its own, beyond `top_k_case_law`. The judgment is chosen by the rank of its passages, not by frequency: it is the one that produced the top-scoring passage of the whole branch.

Three conditions bound the stage, and together they make it safe. It runs only when no operative passage was already selected, so it never duplicates. It grants an additional slot rather than competing for an existing one, so it never displaces a retrieved passage. And the passage it recovers must clear the same threshold as any other, so a judgment whose holding is irrelevant to the question contributes nothing. The stage can add one passage or none, never more, and it is governed by the `guarantee_operative` parameter.

The same structural property that makes the operative part worth recovering also excludes it from the expansion of Section 5.4.2. It is a list of holdings rather than a reading order, so the paragraphs around it carry no argumentative continuity; the exclusion is automatic, operative identifiers being formed with `_op_` rather than `_par_`.

Example. On the query used above, none of the passages surviving selection is operative, and the highest-ranked among them belongs to C-311/18. Its operative paragraphs are read directly and scored; the fourth reaches 0.91, the highest score of the entire branch, and is added to the context. Table 5.3 summarises the outcome of both recovery mechanisms.

Passage	Origin	Score	Outcome
para. 180	retrieved	0.74	kept
para. 185	retrieved	0.71	kept (seed)
para. 186	neighbour of 185	0.88	kept
para. 168	retrieved	0.29	dropped, below threshold
op. 4	operative guarantee	0.91	kept, additional slot

Table 5.3: Case law selection for *"is the Privacy Shield a valid basis for transfers to the United States?"*. The two highest-scoring passages in the final context were both absent from the candidate pool: one was reached from a neighbour, the other fetched because it is the operative part. Neither could have been found by lexical or semantic search.

5.4.4 Citation Bridge

The branch now holds the judgment passages that answer the question, but a judgment is not a rule. The Court states how a provision must be read; the provision is what binds. An answer quoting case law alone therefore leaves the reader without the rule it rests on, so the selected passages have to be paired with the provisions they turn on.

The graph cannot say which those are. Its only link between a judgment and an act, `INTERPRETS`, holds for the judgment as a whole: it records that C-645/19 concerns Article 58, not that paragraph 75 does.

C-645/19 is linked to thirteen provisions, including Article 57 and the whole range from Article 60 to Article 68, which together form the cooperation and consistency machinery between supervisory authorities. All thirteen correctly describe the subject matter of the judgment, but only some of them answer any particular question put to it, and the relation does not tell which. Being linked to a judgment is therefore no guarantee that a provision belongs in the answer.

The details are taken from the judgment's own text instead, because a court shows the difference that the relationship cannot. When it limits the scope of a question, it uses a range. When it identifies the rule it is applying, it uses a single provision, which is linked to a specific paragraph. The two forms have different weightings, and both appear in the same sentence of C-645/19. The Court starts its answer by saying what the field is:

> *"Article 55(1), Articles 56 to 58 and Articles 60 to 66 of Regulation 2016/679, read together with Articles 7, 8 and 47 of the Charter, must be interpreted as meaning that..."*

and contributes nothing to the bridge. Nine provisions are named here, but as a scope rather than as a holding, and admitting them would inject nine articles sharing a single

score, reproducing in another form the coarse signal the mechanism exists to replace. Later in the same paragraph the Court identifies the rules it actually turns on:

> *"... under the national legislation adopted in order to transpose* ***Article 58(5) of that regulation*** *... even though it is not the 'lead supervisory authority', within the meaning of* ***Article 56(1) of that regulation****..."*

and these two are kept. Both are pinpoint citations, both are attached to a proposition the Court is asserting rather than to a delimitation of the field, and both name the act, which is what distinguishes them from the references to the Charter in the first passage. The bridge is thus a syntactic discrimination motivated by drafting practice, not a judgement of relevance; that judgement is made later, by the reranker.

Each provision recovered this way inherits the reranking score of the passage that cites it, taking the highest where several passages cite the same one, so that a provision named by the strongest passage enters the ranking above one named by a marginal passage. Ranking by relevance of the citing passage, rather than by how often an article is cited, is what keeps the mechanism from drifting back towards the frequency signal it replaces. At most `top_k_bridge` provisions are kept.

What enters the fusion. The provisions form a ranked list and join the fusion of the act branch alongside the dense and sparse lists. They receive no privilege there: they are candidates, and must survive truncation of the pool, cross-encoder reranking, and competition for the guaranteed article slots like any other. The bridge buys a provision a place in the candidate pool; it does not buy it a place in the answer. On the query above it contributed Articles 56 and 58, both of which the reranker then placed first and second among the articles, ahead of provisions retrieved conventionally.

What enters the context. The judgment passages do not pass through the fusion at all. They have already been scored, thresholded and capped within their own branch, so they are appended directly to the context in slots of their own, decorated with the citation header that identifies each as a ruling about a provision rather than as the provision itself. Articles and judgments therefore never compete: the case law branch adds to the context without displacing anything from it, and adds to the article ranking without commanding it. When the branch retrieves nothing, or resolves no citation, the fused ranking is bit-for-bit the one the definitional flow produces, so no fallback path exists.

5.5 Generator

The final stage receives a prompt assembled from three parts: the curated context, the user's question, and the guidance string naming the governing provisions where the curation module identified any. Nothing else is supplied. The model is `gpt-4o-mini`, invoked at temperature 0, since the task is not to compose but to report what the retrieved provisions state, and reproducibility matters more than variety in a system whose answers are meant to be verifiable.

The prompt constrains generation rather than inviting it. Every claim must rest on the retrieved content alone, and every legal reference must be copied verbatim from the bracketed source header of the passage supporting it, never recalled from memory. This is what the citation headers built in Section 5.3 exist for: a reference that does not appear in the context cannot be produced, so the commonest failure mode of a legal assistant, a plausible article number attached to a correct statement of law, is made structurally difficult rather than merely discouraged. Where the context does not support a complete answer, the model is instructed to say so instead of closing the gap from its own knowledge.

The answer is a single piece of connected legal prose. The prompt forbids section headers, bullet points and numbered lists, and calibrates length and citation density against reference answers included in the template, because the register being aimed at is that of a legal commentary rather than of a summary: every sentence is expected to carry a substantive rule, cited to the provision it comes from. The constraint matters most in the interpretative flow, where the two kinds of retrieved material have to remain distinguishable within continuous prose. The provision supplies the legal basis and the judgment passage supplies its construction, and the citation headers keep the roles apart in the text itself: a statement of law is cited to `[GDPR, Chapter VI . . . , Article 58]`, the reading the Court gave it to `[C-645/19, The first question, para. 75]`.

One optional stage surrounds the generator: an attribution stage can split the finished answer into sentences and tag each with the markers of the sources supporting it, without rewriting the text; and is used when traceability is being examined (it obviously doubles the number of model calls per query).

Question. *Can a supervisory authority of a Member State bring legal proceedings before its national courts for cross-border data processing infringements when it is not the lead supervisory authority under the GDPR?*

Answer. Yes, according to Article 55(1) of the GDPR, each supervisory authority is competent for the performance of the tasks assigned to it within its own Member State. The Court has held in C-645/19 that a supervisory authority which, under national legislation transposing Article 58(5) of the GDPR, has the power to bring any alleged infringement of that regulation to the attention of a court may exercise that power in relation to cross-border data processing, even if it is not the lead supervisory authority, provided that it acts in one of the situations where the GDPR confers competence on it and respects the cooperation and consistency procedures laid down in Articles 56 to 66. This means that the supervisory authority can initiate or engage in legal proceedings against infringements of the GDPR related to cross-border data processing, as long as the conditions specified in the regulation are met.

Retrieved context (12 passages).
Provisions: GDPR Articles 56, 58, 65, 60, 55
Recitals: GDPR Recitals 127, 131
Case law: C-645/19, paragraphs 75, 42, 43, 105 and operative ruling 1

Figure 5.9: A complete answer produced by the interpretative flow, with the context it was generated from. Articles 56 and 58 were contributed by the citation bridge; paragraph 43 was recovered by neighbour expansion; the operative ruling was retrieved directly. The answer draws on five of the twelve passages and names no provision absent from the context.

Chapter 6

Knowledge Graph Construction Evaluation

The graph described in Chapter 4 is constructed based on rules and draws from diverse, non-uniform sources. The question we posed in RQ1 is whether this works, that is, whether the structure of the documents and the links between rulings and provisions remain intact after construction. We will discuss this in this chapter. We performed the test on court records, which exhibit the greatest variability in form and constitute the majority of the graph. Judgments and their associated links undergo the same checks, and we discuss our findings in this regard at the end of Section 6.3.

6.1 Dataset

The evaluation set is a random sample of EU acts, rather than a selected one. CELEX identifiers were drawn at random from the 2010–2024 regulations and directives, with a fixed seed to enable reproduction of the sample, and every identifier served by EUR-Lex as XHTML was retained. This is important because the parser was developed using a small number of acts at the beginning of our experimentation, after which the parsing phase took a temporary back seat. Selecting further acts manually would have meant selecting acts that resemble those four, and the outcome would reveal more about the selection process than the parser itself.

The sample contains 101 acts: 96 regulations and 5 directives, published between 2010 and 2024. They differ in shape as much as in size. Nineteen are divided into chapters and 82 are not; seven group their articles into sections below the chapter; 66 have annexes and 35 do not. The smallest has 2 articles and 5 nodes, the largest 113 articles and 3021 nodes.

One property of the sample should be stated plainly. Random sampling of the CELEX space returns mostly small amending and implementing regulations, so the median act has

only three articles, and 24 of the 101 have twenty or more. The large acts carry most of the content; the small ones test whether the builder handles trivial shapes without breaking.

No manual annotation was needed. The reference is the source document itself: every piece of published text must appear in the graph, exactly once.

6.2 Methodology and Metrics

An iterative process. The process of creating the graph did not adhere to a single design or implementation. Each new act revealed a structure that the parser had not encountered before, causing it to break in different ways. The work therefore proceeded in cycles: the builder was run over a wider set of acts; what was lost was examined; the assumption behind the loss was identified and fixed; regression tests were run and the act dataset was incrementally grown to always check a larger pool of acts, which is a fundamental step in building bulletproof systems, especially for pipelines that must ingest hundreds of files.

What makes this possible is that the system never writes directly to the database. The builder emits into a recorder and produces a *plan*, the full set of nodes and edges the write would create. The plan is checked, and only a plan that passes is replayed into Neo4j. Evaluation is that same process stopped one step earlier: an act is taken to the point where it would be written, checked, and the result recorded instead. Nothing in this chapter was written to the graph. When an issue occurs, it is also crucial to conduct a thorough investigation to identify the root cause. This involves logging blocked cases in a dedicated, persistent system so that they can be retried once the implementation has been fixed. In order to build a more robust system, we should also address edge cases manually and gather a diverse set of problem instances in order to improve the parser implementation.

What is measured. Parsing has no accuracy score, because there is no set of correct answers to compare against. What can be measured is whether anything was lost, whether the result is well-formed, and whether it is reproducible.

- **Conservation.** Every piece of text in the source HTML must appear in the graph exactly once. Losing a fragment and storing it twice are both failures. This is the main measure.
- **Structure.** No edge may point at a node that was never created, no identifier may be written twice with different values, the containment edges must form a tree, and every edge must be a connection the schema allows.
- **Determinism.** The builder is run twice over the same input and the two plans are compared. A build that is not reproducible cannot be measured.

- **Yield.** An act must produce articles and paragraphs. This check was added during the work, for a reason explained in 6.3.

How the numbers moved. Four rounds of the cycle, each triggered by a different structure, and each verified by re-running the whole sample:

Round	What changed	Acts fully conserved
Start	–	12 / 32
Articles outside chapters	parser	18 / 32
Annex numbering	parser	21 / 32
Annex tables	parser	77 / 101
Annex labels	the check	101 / 101

Table 6.1: Progress across the four rounds. The sample was widened from 32 to 101 acts between the third and fourth round.

No round caused a regression. At every step the acts that already passed still passed, the full test suite stayed green, and the annexes of the AI Act, whose labels are used elsewhere in the system, came out identical to the previous run. That last check was the strictest one: the AI Act produced the same 221 annex points, with the same labels and the same text, after every change.

6.3 Results and Analysis

All 101 acts in the sample are now built with no text lost, no structural violation, and an identical result on a second run. Together they produce 1524 articles, 5813 paragraphs, 2722 recitals, 8693 annex points and 19216 nodes. For the acts, this answers the first part of RQ1 positively: the graph is built deterministically and the structure of every act is preserved.

The interesting part is not the final number but what each round found. Each round was triggered by a shape of act the parser had not anticipated, so the rounds are also a measure of the structural heterogeneity of the sources that RQ1 refers to, and of the fact that it could be absorbed without a regression.

The builder assumed every act has chapters. The first and largest problem. The parser looked for articles only inside chapter divisions, so an act that lists its articles directly produced no articles at all. Twelve of the first 32 acts have this shape, including the Cybersecurity Act and MiCA, and every one of them was built with recitals only.

The failure was perfectly predictable: an act with no chapter markup in the source produced no articles in the graph, in all cases and only in those cases. The fix was to read

the articles that sit outside any chapter and attach them to the act, in the same position as recitals. Retrieval needed no change, because the query that loads articles already walks a variable-length path.

Annexes are not always numbered in Roman. The parser accepted annex divisions labelled with Roman numerals and nothing else. Acts that number their annexes with digits lost the whole annex, silently. The source inventory, which counts what the act contains, had always used the wider pattern, so the two halves of the system disagreed about what an annex was.

An annex table is not one point. The parser treated each annex table as a single point: a marker cell and a text cell. That works for the common case, where each numbered point sits in its own small table. It fails whenever an annex *is* a table, such as a list of currencies, a control list of goods or a schedule of reporting fields, because only the first row survived.

This was the widest problem in the sample. Of 51 failing acts, 46 contained at least one table outside the marker-and-text shape, and none of the acts that passed contained one. The fix was to read tables row by row, keeping the two-cell case exactly as before and storing any other row as an unlabelled point.

One problem was in the measurement, not the parser. The last round is different in kind and should be read as such. After the table fix, 24 acts still reported lost text, and the missing pieces were entries such as "Cauliflowers" or "Order No". Those entries were in the graph: in a two-cell row the first cell is stored as the point's label and the second as its text, and the conservation check only counted text. It was reporting as lost something the builder had kept.

Counting labels as well brought all 101 acts to full conservation. This improved the measurement, not the parser, and the distinction is worth keeping: the acts were not built any better than before.

A gap the checks did not cover. One further finding came from an accident. During the work the EUR-Lex download started returning the same landing page for every act instead of the requested document. The parser then extracted nothing, and the checks reported success: with no source text and no extracted text, "nothing was lost" is true but meaningless.

Conservation alone therefore cannot tell a perfect build from an empty one. A minimum-yield check was added on top of it, requiring an act to produce at least one article and one

paragraph. It is a reminder that a check comparing two sides of a process says nothing when both sides are empty.

What is still open. Annex points are now 8693, against 566 before the table fix, and most of the new ones are rows of data tables with no label to cite them by. They are stored as annex points like any other, and the test that separates a citable point from a data row is whether its label is set. The annex branch of retrieval is disabled, so none of this reaches an answer today; if it is enabled, that filter is where to start.

For judgments and their links, the evidence for RQ1 is weaker than for the acts. The gate guarantees that the judgments in the corpus were stored without loss, but they have not been evaluated on a random sample, so this chapter does not show that the parser generalises to the whole of CJEU case law. The check on `INTERPRETS` establishes that every edge points to an existing provision, not that EUR-Lex declares the right one: the graph preserves the links as EUR-Lex publishes them, and no independent source of truth for their correctness was available to this work. Both evaluations are left to future work (Chapter 8).

Chapter 7

ThemisRAG Evaluation

7.1 Experimental Setting

7.1.1 Dataset

This work features two datasets. The first comprises 53 questions from three acts: the General Data Protection Regulation (GDPR), the Data Governance Act and the Data Act. This dataset comprises pairs of questions and answers, which were written in collaboration with lawyer colleagues from a Belgian university in order to obtain a more precise ground truth. Each question may or may not have a reference to an article, paragraph, chapter, etc., which allows us to work with questions that cite a specific acronym but not the article to which it refers. This was useful for improving our retrieval process and enabling us to recognise patterns even when they are not explicit.

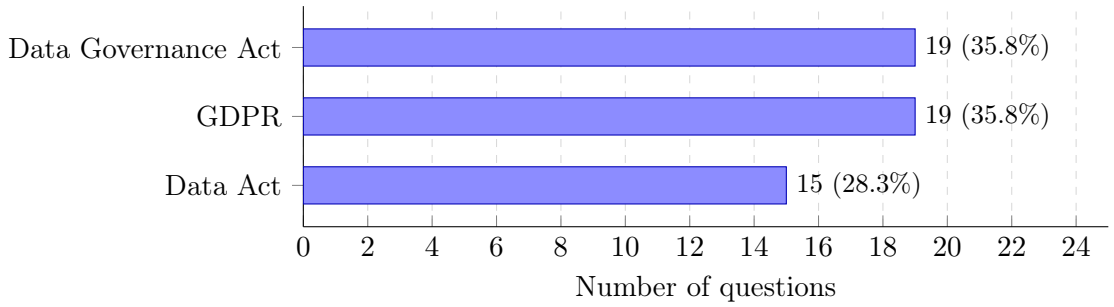


Figure 7.1: Distribution of the 53 questions of the legislation golden dataset across the three acts: Data Governance Act (Regulation (EU) 2022/868), GDPR (Regulation (EU) 2016/679) and Data Act (Regulation (EU) 2023/2854). One Data Act question, on the unfair terms of Article 13, has no ThemisRAG output and is excluded from the evaluation, which uses 52.

However, given that the initial golden dataset comprised a limited number of questions, thereby excluding the part of the graph pertaining to case law, a decision was made to

construct a new one. This new dataset was derived from three cases associated with the General Data Protection Regulation (GDPR). This particular choice was necessitated by the fact that the GDPR is the sole legislation that authorises the scraping of concluded cases. The construction of this model was achieved through the use of generative models, which sought to emulate the structural characteristics of the questions and answers as they were articulated in the initial dataset.

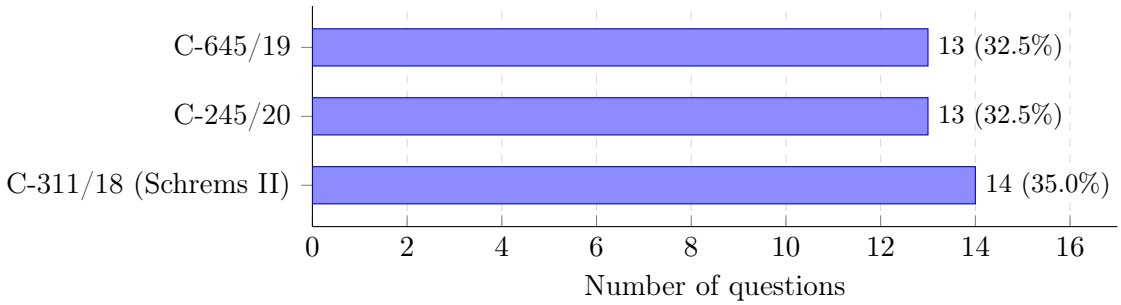


Figure 7.2: The 40 case-law questions across the three judgments of the Court of Justice of the European Union. Two questions, one on C-311/18 and one on C-245/20, have no ThemisRAG output and are excluded from the evaluation, which uses 38.

7.1.2 Models

ThemisRAG relies on one language model for every step that requires one, an embedding model for dense retrieval, and a cross-encoder for reranking (Table 7.1). The language model is `gpt-4o-mini`, called at temperature 0 everywhere except in query expansion. Embeddings are computed with `text-embedding-3-small`, both for the nodes of the graph and for the queries, and the reranker is `zeroentropy/zerank-2` (Section 5.2.4).

Table 7.1: Models used by ThemisRAG and by the evaluation.

Role	Model	Setting
Intent classification	`gpt-4o-mini`	temperature 0
Query expansion (HyDE)	`gpt-4o-mini`	temperature 0.7, three samples
Dense retrieval	`text-embedding-3-small`	1536 dimensions, cosine similarity
Reranking	`zeroentropy/zerank-2`	cross-encoder, about 4B parameters, `bfloat16`
Curation	`gpt-4o-mini`	temperature 0
Answer generation	`gpt-4o-mini`	temperature 0
Judge (RAGAS)	`gpt-4o-mini`	all six metrics
Judge embeddings	`text-embedding-3-small`	answer relevancy only

The baselines are run with their own retrieval components but with the same generation model, `gpt-4o-mini`, so that the differences between systems come from what they retrieve

and how they present it, not from the model that writes the answer.

7.1.3 Metrics

The metrics are divided into competency levels such as retrieval, grounding and correctness. We made this choice on purpose to better understand how well each part of our architecture is performing and where we need to improve or prioritise work.

Notation. For a single test item, q denotes the question, a the answer produced by the system under evaluation, $\mathcal{C} = (c_1, \ldots, c_K)$ the ranked list of the K context passages the system retrieved and passed to its generator, and r the reference answer written by a domain expert. Several metrics build on two operations carried out by the judge:

- *claim decomposition*, written claims(t), rewrites a text t into a set of short, self-contained factual statements;
- *entailment verification*, written $\mathrm{NLI}(s, t) \in \{0, 1\}$, asks the judge whether the statement s can be inferred from the text t, in the sense of natural language inference.

The six metrics observe three distinct stages of the pipeline, summarised in Table 7.2. Keeping the stages apart is essential for interpreting the results: a failure in retrieval and a failure in generation can produce the same wrong answer, but they are visible in different metrics.

Table 7.2: Overview of the evaluation metrics: what each one compares and whether it requires the expert reference.

Stage	Metric	Compares	Uses r
Retrieval	Context precision	$\mathcal{C}$ against r	yes
	Context recall	r against $\mathcal{C}$	yes
Generation	Faithfulness	a against $\mathcal{C}$	no
	Answer relevancy	a against q	no
Correctness	Factual precision	a against r	yes
	Factual recall	r against a	yes

Context precision. For each retrieved passage c_k the judge returns a binary verdict v_k, equal to 1 if c_k was useful to arrive at the reference answer r for the question q. The verdicts are combined as the average precision of the ranked list:

$$\text{Context precision} \;=\; \frac{\sum_{k=1}^{K} \text{precision at } k \cdot v_k}{\sum_{k=1}^{K} v_k}, \qquad \text{precision at } k \;=\; \frac{1}{k}\sum_{j=1}^{k} v_j, \tag{7.1.1}$$

with a context precision of 0 when no passage is judged useful.

What it expresses. Whether the useful passages are ranked above the useless ones, that is, how clean the top of the retrieved list is. The metric is rank-sensitive: an irrelevant passage placed *before* a relevant one lowers the score. Conversely, irrelevant passages ranked *after* the last relevant one do not affect it, so context precision does not measure the total amount of noise handed to the generator, only the order in which it appears.

Context recall. The reference r is split into statements $s_1, \dots, s_m$, and the judge decides for each one whether it can be attributed to the retrieved context:

$$\text{Context recall} \;=\; \frac{1}{m} \sum_{i=1}^{m} \mathrm{NLI}\big(s_i, \mathcal{C}\big). \tag{7.1.2}$$

What it expresses. The share of the expert answer whose content is present in what the system retrieved. It is an upper bound on what a perfectly faithful generator could recover: information missing from $\mathcal{C}$ cannot appear in a grounded answer. When the reference contains statements the indexed corpus does not support, for instance doctrinal commentary absent from the legal text, context recall is capped below 1 for every system alike.

Faithfulness. The answer is decomposed into statements $S_a = \text{claims}(a)$, and each is verified against the concatenation of the retrieved passages:

$$\text{Faithfulness} \;=\; \frac{1}{|S_a|} \sum_{s \in S_a} \mathrm{NLI}\big(s, \mathcal{C}\big). \tag{7.1.3}$$

The score is undefined when the answer contains no factual statement.

What it expresses. The share of the answer that is grounded in the retrieved context, the complement of the hallucination rate with respect to $\mathcal{C}$. It checks the generator's adherence to its input and makes no reference to r: it therefore says nothing about whether the context, and hence the answer, is correct. An answer that faithfully restates an irrelevant or wrong passage obtains a perfect score.

Answer relevancy. The judge generates $n = 3$ questions $\hat{q}_1, \dots, \hat{q}_n$ for which a would be an appropriate answer, and flags each generation as non-committal when the answer is evasive. With $e(\cdot)$ the embedding function and $\cos(\cdot, \cdot)$ the cosine similarity,

$$\text{Answer relevancy} \;=\; \big(1 - \nu\big) \cdot \frac{1}{n} \sum_{i=1}^{n} \cos\big(e(q), e(\hat{q}_i)\big), \tag{7.1.4}$$

where $\nu = 1$ if all n generations are flagged non-committal, and $\nu = 0$ otherwise.

What it expresses. How directly the answer addresses the question that was actually asked. Incomplete answers, digressions and redundant content widen the questions that can be reverse-engineered from a, move them away from q in the embedding space, and lower the score; evasive answers score zero. Like faithfulness, it is independent of r and does not assess correctness. In the legal domain it tends to penalise answers that qualify their scope or cite several provisions, because such answers support broader reconstructed questions.

Correctness metrics. The two factual-correctness metrics are the only ones that compare the answer directly with the expert reference, and are therefore the most direct measure of answer quality. Both the answer and the reference are decomposed into claims, $A = \text{claims}(a)$ and $R = \text{claims}(r)$, and each claim is verified against the opposite text. This yields

$$\begin{aligned} \text{true positives} &= \sum_{s \in A} \text{NLI}(s, r), \\ \text{false positives} &= |A| - \text{true positives}, \\ \text{false negatives} &= \sum_{s \in R} \big(1 - \text{NLI}(s, a)\big), \end{aligned} \tag{7.1.5}$$

from which the two metrics are obtained. Both scores are rounded to two decimals.

Factual precision.

$$\text{Factual precision} \;=\; \frac{\text{true positives}}{\text{true positives} + \text{false positives}} \;=\; \frac{1}{|A|} \sum_{s \in A} \text{NLI}(s, r). \tag{7.1.6}$$

What it expresses. The share of what the system asserts that is endorsed by the expert. The metric is conservative: a claim that is correct but absent from the reference counts as a false positive, so an answer that adds accurate information beyond the reference is penalised.

Factual recall.

$$\text{Factual recall} \;=\; \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}. \tag{7.1.7}$$

What it expresses. How much of the expert answer the system recovers, that is, the completeness of the answer. Note that the true positives count supported claims of the *answer*, whereas the false negatives count unsupported claims of the *reference*. The score therefore coincides with the fraction of reference claims covered only when claims correspond one-to-one, and otherwise approximates it; an answer that restates a covered point in several claims raises the true positives and is rewarded accordingly.

Why precision and recall are reported separately. RAGAS can combine the two into an F_β score; we deliberately do not. The reference answers are encyclopaedic, and some of their content cannot be supported by the indexed corpus. A single F score would merge two signals of very different nature: reference claims no system can retrieve, which lower recall, and additional grounded claims in the answer, which lower precision. Keeping the two apart lets each failure be attributed to its cause.

Reading the metrics jointly. No single metric measures the quality of a legal answer, and several combinations of scores are diagnostic in their own right:

- *high faithfulness, low factual precision*: the answer is faithful to a context that does not answer the question, a retrieval failure that the generation metrics alone would not reveal;
- *high context recall, low factual recall*: the relevant information was retrieved but not used, typically because it was diluted in a large amount of irrelevant context;
- *high context precision, lower context recall*: retrieval is selective and well ranked but misses part of the evidence, a trade-off whose effect on the final answer is read in the correctness metrics.

7.2 Experimental Evaluation

ThemisRAG is compared with three graph-based RAG systems that are established reference points in the literature: LightRAG [17], PathRAG [6] and HippoRAG [23]. Each baseline is run from its original implementation, wrapped in an evaluation harness that submits the questions, collects answer and retrieved context, and scores them with the metrics of Section 7.1.3, using the same judge for all systems.

The section proceeds from the surface to the mechanisms. Section 7.2.1 reports the aggregate comparison and where the advantage of ThemisRAG holds; the following sections examine individual questions to explain these results, and close with the cases in which the architecture fails, including those where a baseline does better. Throughout, the results are read against the three sub-questions of RQ2: whether confining retrieval to the acts a question concerns makes answers more complete (RQ2.1), whether a small context can be as complete as the larger contexts of the baselines (RQ2.2), and whether the structure of a judgment and its links to the provisions it interprets make answers on case law more precise (RQ2.3).

7.2.1 Results

Table 7.3 reports the mean of each metric on the legislation and case-law datasets. Systems are compared question by question: p-values are two-sided Wilcoxon signed-rank tests on the paired differences between ThemisRAG and each baseline, and differences smaller than 0.01 are treated as ties, both in the tests and in the per-question counts.

Table 7.3: Mean scores per system. Best value per column and dataset in bold.

		Retrieval		Generation		Correctness	
Dataset	System	Context precision	Context recall	Faithfulness	Answer relevancy	Factual precision	Factual recall
Legislation (n=52)	ThemisRAG	**0.984**	0.797	0.943	0.754	**0.478**	**0.657**
	LightRAG	0.851	**0.921**	0.914	0.787	0.452	0.449
	PathRAG	0.902	0.896	**0.980**	**0.792**	0.424	0.444
	HippoRAG	0.981	0.864	0.953	0.749	0.416	0.392
Case law (n=38)	ThemisRAG	**0.980**	0.911	0.954	**0.895**	**0.538**	**0.641**
	LightRAG	0.825	0.961	**0.985**	0.835	0.444	0.610
	PathRAG	0.860	0.945	0.942	0.719	0.349	0.527
	HippoRAG	0.950	**0.978**	0.940	0.876	0.493	0.571

Retrieval and context. On legislation ThemisRAG answers from a smaller context than the baselines: it passes on average 3.5 passages to the generator, against 5.0 for HippoRAG, 5.8 for PathRAG and 13.7 for LightRAG. On case law the difference remains only against LightRAG: ThemisRAG passes 5.3 passages, in line with HippoRAG (5.0) and PathRAG (5.9), against 13.2 for LightRAG (Figure 7.3, left).

ThemisRAG has the lowest context recall on both evaluated datasets, 0.797 on legislation and 0.911 on case law. The gap, however, is concentrated on a few questions: on most of them every system retrieves all that the reference needs, and the difference is significant only against LightRAG on legislation (0.921, $p = 0.008$) and against HippoRAG on case law (0.978, $p = 0.016$).

The right panel shows that this lower context recall does not translate into a less complete answer: the baselines retrieve a larger share of what the reference needs, yet it is ThemisRAG that reaches the highest factual recall on both datasets. Against LightRAG and PathRAG, what ThemisRAG leaves out is mostly noise. Its context precision is 0.984 on legislation and 0.980 on case law, against 0.851 and 0.825 for LightRAG and 0.902 and 0.860 for PathRAG, and the difference is significant against both on both datasets ($p < 0.001$ against LightRAG, $p = 0.002$ and $p = 0.005$ against PathRAG). The advantage is consistent rather than driven by a few questions: on legislation ThemisRAG's context is more precise than LightRAG's on 36 of the 52 questions and less precise on 3, and more precise than PathRAG's on 20 and less precise on 2; on case law the counts are 26 against 0 and 10 against 2. HippoRAG, on the other hand, retrieves as cleanly as ThemisRAG (0.981 and 0.950, no significant difference) and from a context of similar size. This answers RQ2.2: a small context, selected through

the structure of the graph, does not cost completeness.

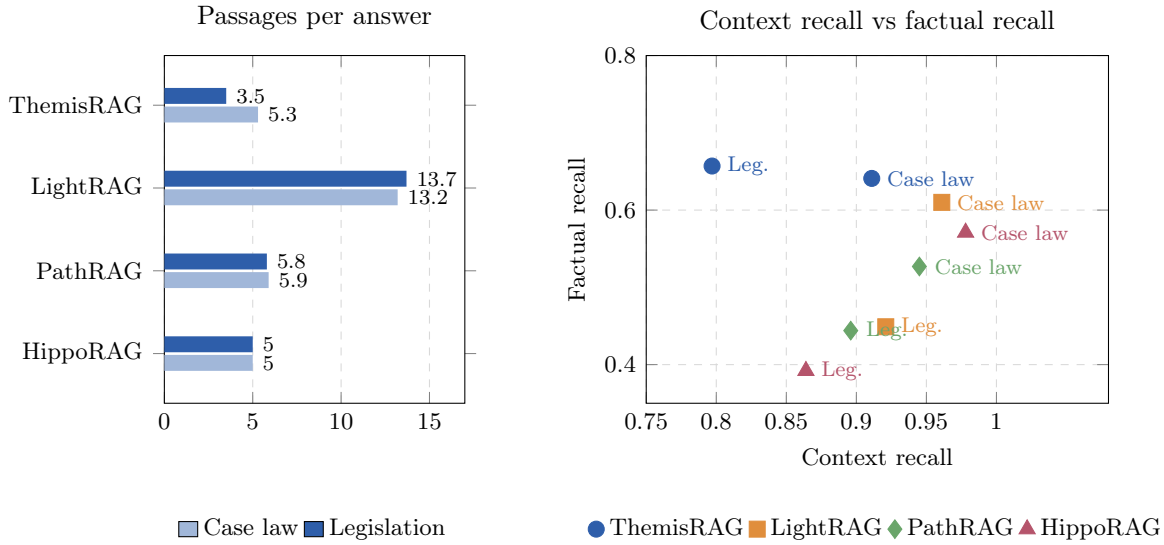


Figure 7.3: Left: average number of text passages passed to the generator; for PathRAG, graph entities and relations are not counted. Right: context recall against factual recall, one point per system and dataset.

Part of this precision is due to the curation module (Section 5.3.1), which keeps the governing provisions and the passages that support them and drops the rest. Curation, however, can only select among the passages it receives: it cannot guarantee a correct answer if the governing provision was never retrieved. What makes the small context sufficient is therefore decided earlier, when the question is routed to the act it concerns.

ThemisRAG classifies each question before retrieving anything (Section 5.1): the classifier selects the target acts, and both retrievers search only within them. The baselines index the acts as a single pool of chunks or entities, and since the acts share most of their vocabulary (*data holder*, *data intermediation*, *processing*), a question about one act retrieves text from the others. Figure 7.4 measures the effect. Only 3.6% of the legislative provisions ThemisRAG retrieves belong to an act other than the one the question concerns, against 26.8% for HippoRAG, 46.1% for LightRAG and 50.7% for PathRAG. A provision from another act appears in the context of 3 of the 52 questions for ThemisRAG, against 28 for HippoRAG, 47 for PathRAG and 51 for LightRAG.

In ThemisRAG the only source of cross-act retrieval is a classification error, and such errors are costly when they occur: the three misrouted questions have a factual recall of 0.15, 0.23 and 0.45, against 0.680 for the other 49.

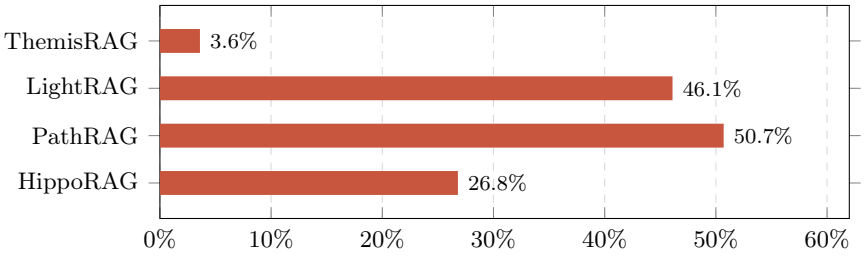


Figure 7.4: Share of retrieved legislative sources that belong to an act other than the one the question concerns. For PathRAG the act of each chunk is inferred from its text (see footnote).

Generation. Faithfulness is high for all four systems, between 0.91 and 0.99 on both datasets: all of them answer from their context rather than from the memory of the generator model. The only significant difference is on legislation, where PathRAG is more faithful than ThemisRAG (0.980 against 0.943, $p = 0.009$), scoring higher on 19 of the 52 questions and lower on 6. Answer relevancy shows no significant difference on legislation. On case law ThemisRAG is the most relevant (0.895), significantly more than LightRAG (0.835, $p < 0.001$) and PathRAG (0.719, $p < 0.001$), and on par with HippoRAG (0.876, $p = 0.065$).

Correctness. Figure 7.5 breaks the correctness metrics down question by question. On legislation the advantage is in factual recall, and it is consistent: ThemisRAG recovers more of the expert answer than LightRAG, PathRAG and HippoRAG on 41, 37 and 43 of the 52 questions respectively ($p < 0.001$ in all three cases). On factual precision the picture is more balanced. ThemisRAG asserts fewer claims unsupported by the reference on 29 to 31 of the 52 questions, a majority but a narrow one, and the difference is not significant against any baseline. On legislation, ThemisRAG's answers are therefore more complete than those of the baselines, which, together with the routing results above, answers RQ2.1 positively; the paragraph on individual acts below qualifies this answer.

On case law the relation is reversed. ThemisRAG still has the highest factual recall (0.641), but the margin is smaller and significant only against PathRAG (0.527, $p = 0.014$); against LightRAG (0.610) and HippoRAG (0.571) it is higher on 21 and 19 of the 38 questions, with no significant difference. The advantage moves to factual precision: ThemisRAG asserts fewer unsupported claims than LightRAG on 25 of the 38 questions ($p = 0.015$) and than PathRAG on 28 ($p < 0.001$). On judgments ThemisRAG does not recover much more of the expert answer than the baselines, but asserts less that the reference does not support.

HippoRAG is the exception on case law. ThemisRAG has a higher mean on factual precision (0.538 against 0.493) and factual recall (0.641 against 0.571), but the per-question comparison is close to even (19 against 17 and 19 against 15), and neither difference is

significant. On judgments the two systems perform comparably. The answer to RQ2.3 is therefore positive against LightRAG and PathRAG, and open against HippoRAG.

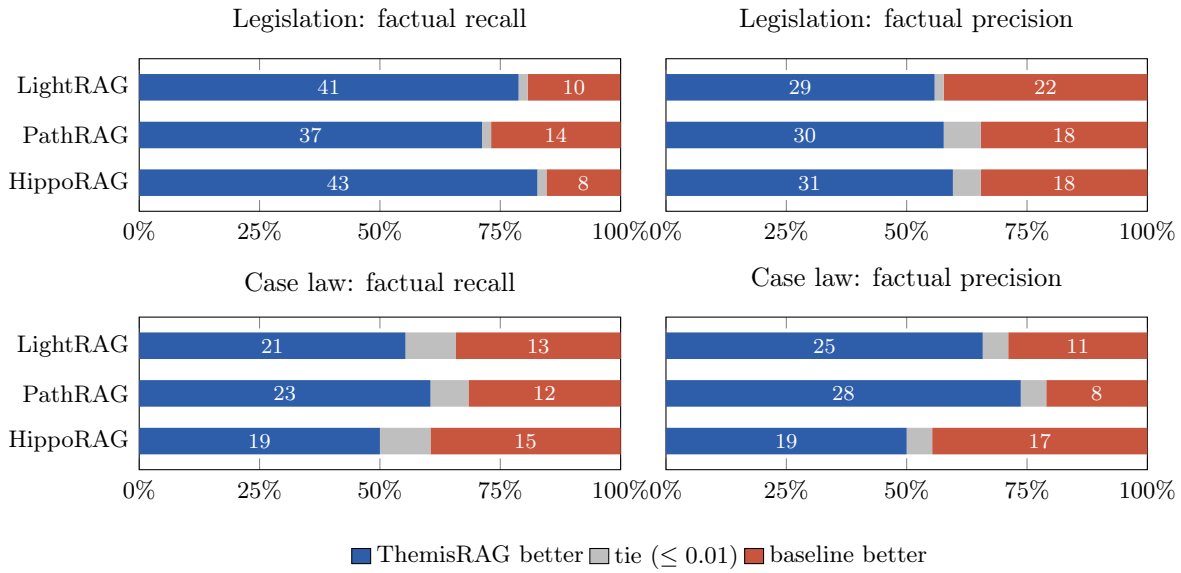


Figure 7.5: Per-question comparison of ThemisRAG against each baseline. Bars give the share of questions; labels give the number of questions.

Per act. The legislation advantage is not uniform across acts (Figure 7.6). On the Data Act and the Data Governance Act it is near-total: ThemisRAG is better than each baseline on 12 to 13 of 14 and 16 to 18 of 19 questions respectively. On the GDPR it largely disappears: its factual recall falls to 0.479, below PathRAG (0.574), and its context recall to 0.52, against 0.99 and 0.93 on the other two acts. The answer to RQ2.1 therefore holds on two of the three acts; Section 7.2.3 examines why it does not on the GDPR.

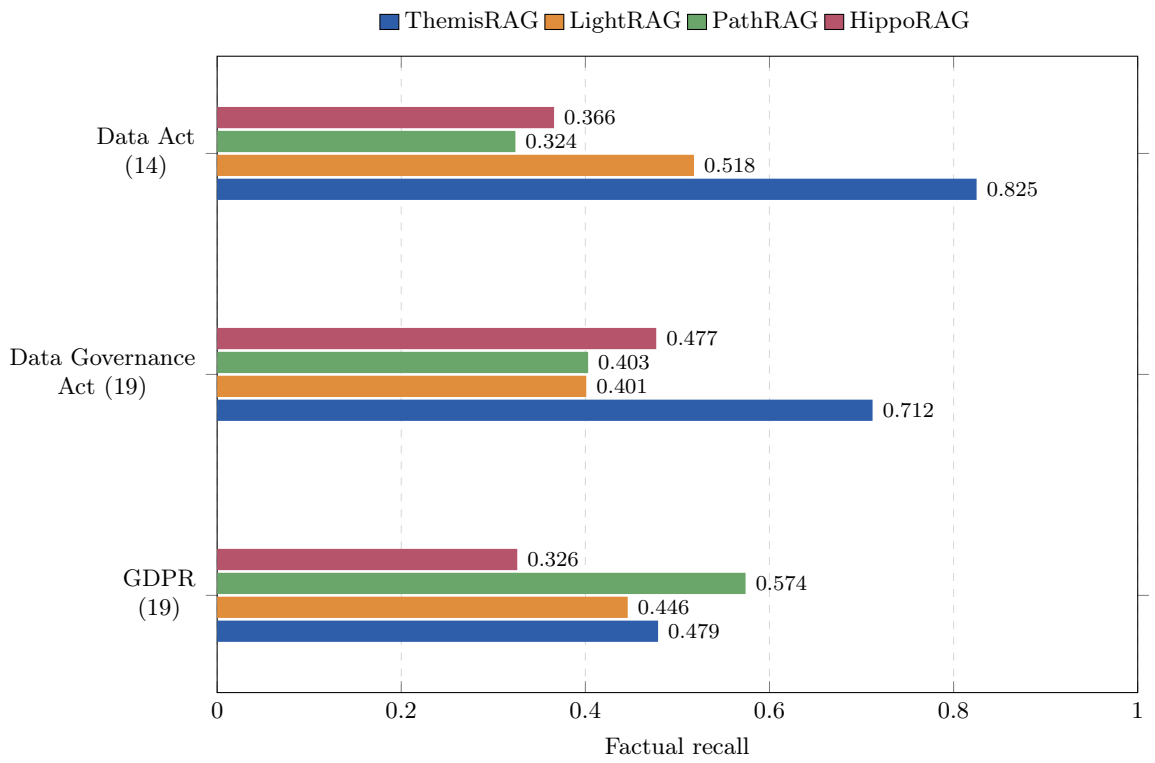


Figure 7.6: Mean factual recall per act on the legislation dataset (number of questions in brackets).

7.2.2 Qualitative Analysis

The results point to three strengths of ThemisRAG:

- it identifies with high reliability the act or acts needed to answer a question (49 of the 52 legislation questions are routed correctly) and searches within them, so text from other regulations rarely reaches its context;
- it answers from a few passages that are almost all relevant, where the baselines pass many and partly off-topic ones;
- on case law, it asserts less that the reference does not support.

The four examples below show where these strengths come from, two on legislation and two on case law: Examples 1 and 2 illustrate RQ2.1 and RQ2.2, Examples 3 and 4 RQ2.3. Each follows one question from the passages ThemisRAG retrieves to the answer it gives, and places those passages in the graph: the act, chapter or section of the judgment they belong to, and the links between them.

Each example is presented in the same way. A first table sets the reference answer beside the answer of ThemisRAG. The reference is divided into steps, each with its own colour and number, and the same colour marks the passage of an answer that states that step; an underline in the same colour marks a step stated only in part. A second table shows the answers of the three baselines, highlighted in the same way, and the scores of every system on the question. A checklist then summarises which answer states which step. The highlighting is our reading of the answers, not the RAGAS judgement, and on a single question the two can disagree; the aggregate metrics of Section 7.2.1 average this out. Excerpts are quoted verbatim, with Markdown formatting removed and omissions marked [. . .]. In the figures, filled nodes are passages ThemisRAG retrieved, dashed nodes are passages the reference draws on that ThemisRAG did not retrieve, and light nodes are the structure that places them; solid edges are graph edges walked at query time, dashed edges are links derived from the text of a passage or from paragraph numbering.

Example 1: a precise context against a noisy one. The question in Table 7.4 asks how the Data Act's obligation to inform the user "in a clear and comprehensible manner" can be fulfilled. ThemisRAG retrieves two passages (Figure 7.7): Article 3, which lays down the obligation in its paragraph 2, and Recital 24, which explains how to meet it. Article 3(2) governs the question and Recital 24 supports it, and here the support carries the answer: the article says what the seller must do, the recital how it can be done. The recital hangs off the act rather than off a chapter and is retrieved by its own branch, so the provision and its explanation reach the generator together, and the answer states four of the five steps of the reference.

Table 7.4: Example 1, Data Act: the reference answer and the answer of ThemisRAG.

Question. How can the obligation for a communication 'in a clear and comprehensible manner' be fulfilled?

Reference	**ThemisRAG**
"As per Recital 24, the information obligation could be fulfilled by maintaining a stable uniform react locator (URL) on the web[1] [*sic*], which can be distributed as a web link or QR code, pointing to the relevant information[2], which could be provided by the seller, renter or lessor, which may be the manufacturer, to the user before concluding the contract for the purchase, rent or lease of a connected product[3]. It is, in any case, necessary that the user is able to store the information in a way that is accessible for future reference[4] and that allows the unchanged reproduction of the information stored[5]."	"[...] Before concluding a contract for the purchase, rent, or lease of a connected product, the seller, rentor, or lessor (which may include the manufacturer) is required to provide specific information to the user in a clear and comprehensible manner[3]. This includes: [...] (Article 3(2)). [...] This could be facilitated through stable URLs[1] or QR codes that point to the relevant information[2], ensuring that users can store this information for future reference[4] (Recital 24)."

The baselines mostly achieved no realiable result, in fact, based on (Table 7.5). LightRAG and PathRAG have contexts with several chunks of text from different acts than the Data Act (LightRAG has seven of the fifteen chunks from different acts). HippoRAG declined to answer and had the same issue. ThemisRAG partially covered the expert answer (0.50) and received fewer supported claims (0.19) because it mentioned extra information that wasn't in the reference.

Table 7.5: Example 1: the baseline answers and the scores of every system on this question. No baseline answer states a step of the reference.

System	Answer (excerpt)	Expert answer covered	Claims supported
ThemisRAG	see Table 7.4	**0.50**	0.19
LightRAG	"The obligation to communicate 'in a clear and comprehensible manner' can be fulfilled by adhering to several best practices [...] Using Plain Language: Information should be presented using simple, everyday language [...] Visual Aids: Utilizing diagrams, charts, or images can complement the text [...]" Sources: 15 chunks, 8 Data Act, 5 AI Act, 1 GDPR, 1 Data Governance Act	0.00	0.00
HippoRAG	"The passages do not contain information regarding the obligation for communication 'in a clear and comprehensible manner.' As such, I cannot provide an answer [...]" Sources: 5 passages, 1 Data Act, 3 AI Act, 1 GDPR	0.11	**0.50**
PathRAG	"[...] Simplicity and Clarity: Communication should use simple language and avoid jargon [...] Accessible Formats: Providing information in various accessible formats ensures inclusivity. This includes large print, audio formats, Braille, and language translations as necessary [...]"	0.00	0.00

Expert answer covered: factual recall. Claims supported: factual precision.

Table 7.6: Example 1: the steps of the reference and the answers that state them (✓ stated, (✓) stated in part, – missing). For ThemisRAG, in brackets, the passage the step is drawn from.

Step of the reference	ThemisRAG	LightRAG	PathRAG	HippoRAG
1 A stable URL on the web	✓ (Rec. 24)	–	–	–
2 Distributed as a web link or QR code	✓ (Rec. 24)	–	–	–
3 Provided by the seller, renter or lessor before the contract	✓ (Art. 3(2))	–	–	–
4 The user can store the information for future reference	✓ (Rec. 24)	–	–	–
5 The stored information can be reproduced unchanged	–	–	–	–

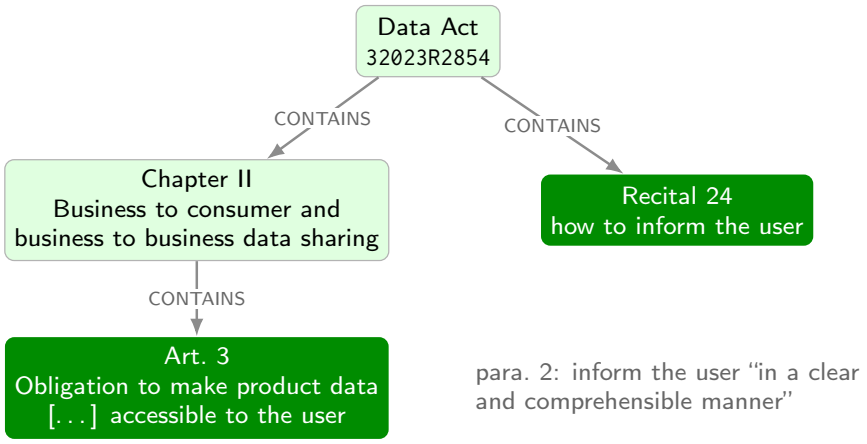


Figure 7.7: Example 1. The two passages ThemisRAG retrieves: the article that lays down the obligation and the recital that explains how to fulfil it. Recitals hang off the act, not off a chapter, and are retrieved by a branch of their own.

Example 2: a chapter as a chain of provisions. The reference in Table 7.7 is built in two steps: Article 14 of the Data Act obliges a data holder to make data available when the requesting body demonstrates an exceptional need, and Article 15 defines when such a need exists. All five passages ThemisRAG retrieves come from Chapter V of the Data Act, which is the chapter governs this obligation: Articles 14, 15, 17, 18 and 20, five of its nine articles (Figure 7.8) the chapter contains. The context is therefore a coherent unit of the act, and its articles refer to each other: Article 14 applies "where [the requesting body] demonstrates an exceptional need, as set out in Article 15", and Articles 17 and 20 refer back to Article 15 as well. These cross-references are in the text of the articles, not in the graph, but because the chapter is in context, the generator can follow them consistently according to the act. The answer follows the two steps of the reference, from the obligation of Article 14 to the cases of exceptional need of Article 15, and states all four steps of the reference.

Table 7.7: Example 2, Data Act: the reference answer and the answer of ThemisRAG.

Question. In which cases is a data holder obliged to make data available to a public sector body, the Commission, the European Central Bank or a Union body?

Reference	**ThemisRAG**
"As per Article 14, such an obligation exists when a public sector body, the Commission, the European Central Bank or a Union body demonstrate an exceptional need for which they need to make use of that data[1]. As per Article 15, such an exceptional need shall be: (a) limited in time and scope[2]; (b) considered to exist in two cases: (i) response to a public emergency, whereby that data could not have been obtained otherwise in a timely manner[3]; or (ii) fulfilment of tasks in the public interest[4] (e.g. production of official statistics or the mitigation of or recovery from a public emergency)."	"[...] A data holder that is a legal person (excluding public sector bodies) must make data available upon a duly reasoned request from a public sector body, the Commission, the European Central Bank, or a Union body, provided that the requesting entity demonstrates an exceptional need as specified in Article 15[1]. [...] Public Emergency: If the data requested is necessary to respond to a public emergency and the requesting entity is unable to obtain such data by alternative means in a timely and effective manner[3] (Article 15(1)(a)). [...] A public sector body, the Commission, the European Central Bank, or a Union body is acting based on Union or national law and has identified specific data that is necessary to fulfill a specific public interest task, such as producing official statistics[4] or responding to a public emergency (Article 15(1)(b)(i)). [...] Limitations: The exceptional need is limited in time and scope[2], and the obligation does not apply to microenterprises and small enterprises (Article 15(2))."

The baselines answer the two cases of exceptional need well (Table 7.8), but not the obligation they qualify. LightRAG and HippoRAG present the cases as a list of situations in which data must be shared, without the requirement that the requesting body demonstrate the need, and neither say that the need is limited in time and scope. Five of the twelve chunks LightRAG retrieves come from the Data Governance Act or the GDPR, and HippoRAG cites the recitals of the act as "paragraphs". PathRAG covers both cases but frames them as general duties of data holders, and RAGAS finds none of its claims supported by the reference (factual precision 0.00). ThemisRAG achieved a perfect coverage of sources to build a reliable answer (1.00), however The precision of ThemisRAG (0.50) is lowered by details of the same articles that the reference does not mention.

Table 7.8: Example 2: the baseline answers, highlighted with the colours of Table 7.7, and the scores of every system on this question.

System	Answer (excerpt)	Expert answer covered	Claims supported
ThemisRAG	see Table 7.7	**1.00**	**0.50**
LightRAG	"A data holder is obliged to make data available to a public sector body, the Commission, the European Central Bank, or a Union body under the following circumstances, which are defined as exceptional needs(1): Public Emergency Response: When there is an urgent requirement to respond to a public emergency, and the requested data is necessary3. [...] Other Exceptional Needs: [...] The specific data requested is vital for fulfilling a statutory task carried out in the public interest4, which has been explicitly provided for by law [...]" Sources: 12 chunks, 7 Data Act, 3 Data Governance Act, 2 GDPR	0.60	0.25
HippoRAG	"A data holder is obliged to make data available to a public sector body, the Commission, the European Central Bank, or a Union body in the following cases(1): Exceptional Need Related to Public Emergency: Under paragraph (64), in situations of public emergencies (e.g., public health emergencies, natural disasters), data holders must provide data upon request3. [...] According to paragraph (65), [...] The requesting body must show that the data is necessary for a specific public interest task explicitly provided for by law4 [...]"	0.40	0.17
PathRAG	"Data holders, defined as individuals or entities managing data, have specific obligations [...] Public Emergency Situations: Data holders must provide data when requested by public sector bodies, the Commission, the European Central Bank, or Union bodies to address public emergencies. The urgency of the situation necessitates that the data sought cannot be obtained through alternative means in a timely manner3 [...] Public Interest Tasks: If a public sector body, the Commission, the European Central Bank, or a Union body requires specific data to fulfill a mandated public interest task and cannot obtain it by other means, the data holder is obliged to comply4. [...] Exhaustion of Other Means: Before a data holder is obligated to comply with such requests, the requesting body must demonstrate that all other avenues for obtaining the required data have been exhausted(1). [...]"	0.75	0.00

Expert answer covered: factual recall. Claims supported: factual precision.

Table 7.9: Example 2: the steps of the reference and the answers that state them (✓ stated, (✓) stated in part, – missing). For ThemisRAG, in brackets, the article the step is drawn from.

Step of the reference	ThemisRAG	LightRAG	PathRAG	HippoRAG
1 The obligation arises when the requesting body demonstrates an exceptional need	✓ (Art. 14)	(✓)	(✓)	(✓)
2 The need is limited in time and scope	✓ (Art. 15)	–	–	–
3 First case: a public emergency, data not obtainable otherwise in time	✓ (Art. 15)	✓	✓	✓
4 Second case: tasks in the public interest	✓ (Art. 15)	✓	✓	✓

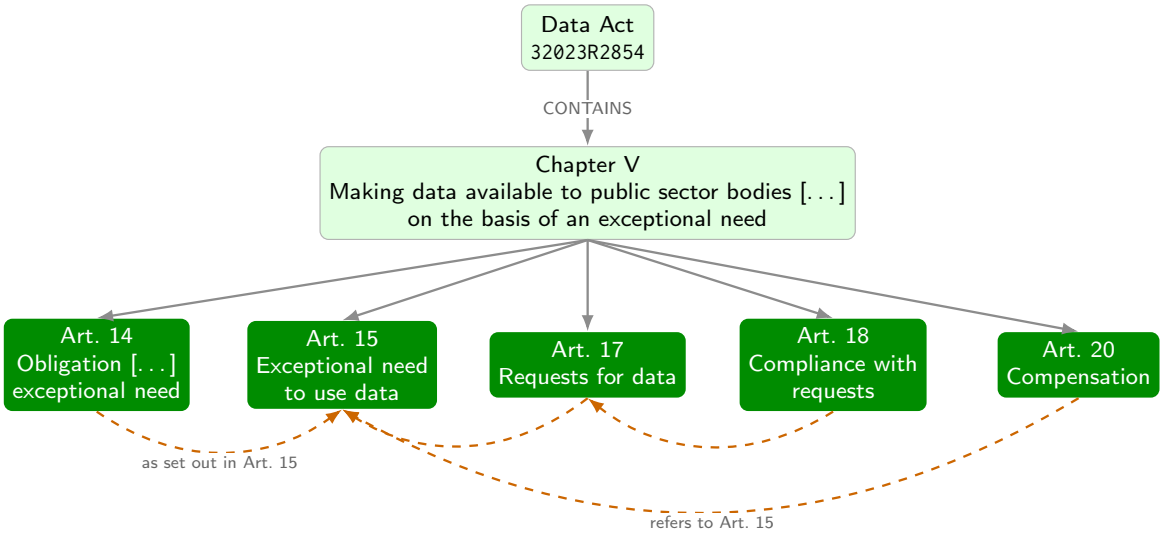


Figure 7.8: Example 2. The five articles ThemisRAG retrieves all belong to Chapter V of the Data Act. The dashed edges represent the cross-references in the text of the articles.

Example 3: from the judgment to the article it applies. The next example concerns C-645/19 (*Facebook Ireland and Others*), a case on whether the Belgian supervisory authority, which is not the lead authority under the GDPR one-stop-shop mechanism,[1] could bring legal proceedings against Facebook for cross-border processing before a national court.

The reference in Table 7.10 is drawn from paragraphs 58 and 60 to 62 of C-645/19 and restates the procedure of Article 56(2) to (5) of the GDPR. ThemisRAG retrieves paragraphs 58 and 60, both in the section of the judgment that answers the first question referred

[1]Under the one-stop-shop mechanism, a company that processes personal data in several Member States answers to a single *lead supervisory authority*, that of the Member State of its main establishment (Article 56(1) GDPR). The other supervisory authorities concerned cooperate with it under the procedure of Article 60 rather than act on their own.

(Figure 7.9). Paragraph 58 cites Article 56(2), and the citation bridge brings Article 56 into the context; Article 60, which governs the cooperation procedure, is retrieved by the act branch. The answer follows Article 56 paragraph by paragraph, from competence (56(2)) to informing the lead authority (56(3)) and to the two outcomes (56(4) and 56(5)), and states all five steps of the reference.

Table 7.10: Example 3, C-645/19: the reference answer and the answer of ThemisRAG.

Question. How must a supervisory authority that is not the lead authority proceed when it intends to handle a complaint about cross-border processing that concerns only its own Member State?

Reference	**ThemisRAG**
"Under Article 56(2) GDPR, a supervisory authority which is not the lead authority is competent to handle such a complaint if its subject matter relates only to an establishment in its Member State or substantially affects data subjects only in that Member State[1]. It must, however, inform the lead supervisory authority without delay[2] under Article 56(3), and the lead authority decides within three weeks whether it will handle the case[3]. If it does, the cooperation procedure of Article 60 applies, and the authority that informed it may submit a draft decision of which the lead authority must take the utmost account[4] (Article 56(4)). If it does not, the authority that informed it handles the case in accordance with Articles 61 and 62[5] (Article 56(5))."	"[...] According to Article 56(2), a supervisory authority that is not the lead supervisory authority is competent to handle a complaint if: The subject matter of the complaint relates only to an establishment in its own Member State. The processing substantially affects data subjects only in that Member State[1]. [...] The supervisory authority that intends to handle the complaint must inform the lead supervisory authority without delay about the matter[2], as stipulated in Article 56(3). [...] Upon being informed, the lead supervisory authority has a period of three weeks to decide whether it will take over the handling of the case or allow the notifying supervisory authority to proceed[3]. [...] If the lead supervisory authority decides to handle the case, the procedures set out in Article 60 regarding cooperation between supervisory authorities will apply[4]. This includes the exchange of relevant information and the possibility for the notifying authority to submit a draft decision for consideration by the lead authority[4]. [...] If the lead supervisory authority decides not to handle the case, the notifying supervisory authority will then proceed to handle the complaint according to Articles 61 and 62 of the GDPR[5]."

Here the baselines come close in substance, and the difference lies in precision (Table 7.11). LightRAG states four of the five steps and the remaining one in part, nearly as much as ThemisRAG, but adds a step the reference does not contain, the duty to comply with local law, and its factual precision is 0.67. HippoRAG stops at the lead authority's decision, without the deadline, the cooperation procedure or the fallback of Articles 61 and 62, and attributes the duty to inform to Recital 127 rather than to Article 56(3). PathRAG

follows the same procedure without its legal bases: its two outcomes refer to "the outlined procedures" and "its established procedures" instead of Article 60 and Articles 61 and 62. ThemisRAG is the only system whose every claim is supported by the reference: its factual precision is 1.00.

Table 7.11: Example 3: the baseline answers, highlighted with the colours of Table 7.10, and the scores of every system on this question.

System	Answer (excerpt)	Expert answer covered	Claims supported
ThemisRAG	see Table 7.10	**0.75**	**1.00**
LightRAG	"[...] The supervisory authority not acting as the lead authority is competent to handle complaints lodged concerning cross-border processing if the subject matter relates solely to an establishment within its Member State[1] [...] the supervisory authority is required to inform the lead supervisory authority without delay[2]. [...] the lead supervisory authority must decide within three weeks whether it will handle the case[3]. If the lead authority chooses to handle the case, the local authority can submit a draft for its decision(4). [...] the local supervisory authority that received the complaint will then proceed to handle it independently, following Articles 61 and 62 of the regulation[5] [...] Ensuring Compliance with Local Laws: The local supervisory authority must adhere to the laws [...] applicable within its jurisdiction [...]"	0.60	0.67
HippoRAG	"[...] the supervisory authority is competent to handle the complaint if the subject matter relates only to an establishment in its Member State[1] [...] The supervisory authority must inform the lead supervisory authority without delay about the matter[2], as stated in Recital 127. [...] the lead supervisory authority will decide whether it will handle the case(3) [...]"	0.25	0.50
PathRAG	"[...] The non-lead supervisory authority must first ascertain that the subject matter of the complaint relates exclusively to an establishment located within its territorial jurisdiction[1] [...] the supervisory authority is required to inform the lead supervisory authority regarding the case[2]. [...] the lead supervisory authority has a defined period (typically three weeks) to decide whether it will take over the case[3]. [...] it will then proceed with the outlined procedures according to GDPR provisions(4). [...] the non-lead supervisory authority will proceed with its investigation and handle the complaint per its established procedures(5). [...]"	0.57	0.50

Expert answer covered: factual recall. Claims supported: factual precision.

Table 7.12: Example 3: the steps of the reference and the answers that state them (✓ stated, (✓) stated in part, – missing). For ThemisRAG, in brackets, the paragraph of Article 56 the step is drawn from; Articles 61 and 62, which step 5 names, were not retrieved.

Step of the reference	ThemisRAG	LightRAG	PathRAG	HippoRAG
1 Competent if the complaint concerns only its own Member State	✓ (Art. 56(2))	✓	✓	✓
2 Inform the lead authority without delay	✓ (Art. 56(3))	✓	✓	✓
3 The lead authority decides within three weeks	✓ (Art. 56(3))	✓	✓	(✓)
4 If it handles the case: Article 60 and a draft decision	✓ (Art. 56(4))	(✓)	(✓)	–
5 If not: the informing authority handles it under Articles 61 and 62	✓ (Art. 56(5))	✓	(✓)	–

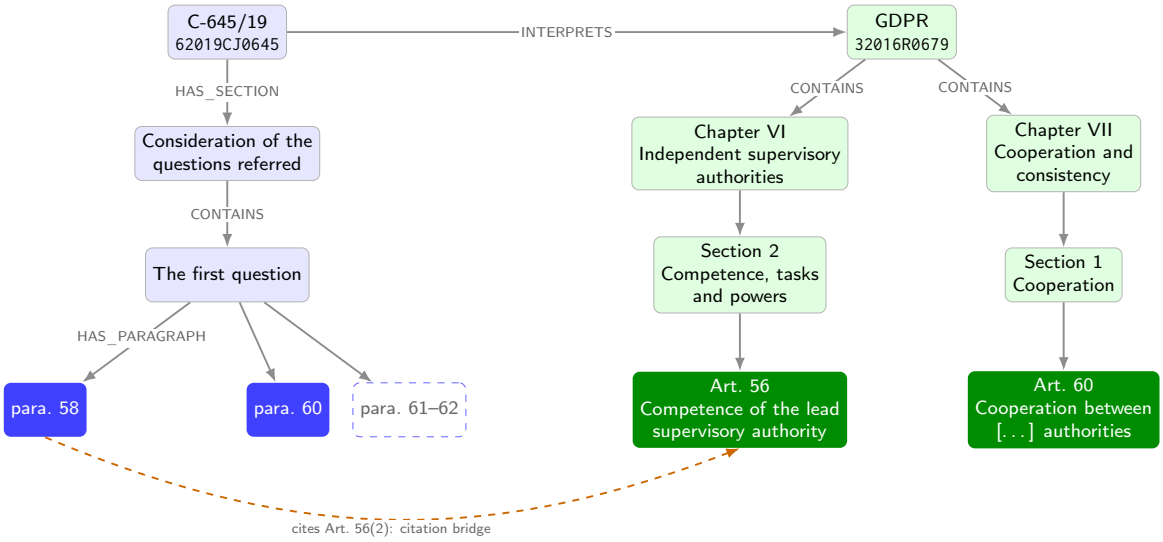


Figure 7.9: Example 3. The judgment paragraphs ThemisRAG retrieved, the article the bridge derives from the citation in paragraph 58, and the place of both in their hierarchies. Among its `INTERPRETS` edges, C-645/19 points to Article 56(1) and Article 60 of the GDPR.

Example 4: a block of reasoning and the holding. The last example concerns C-311/18 (*Schrems II*), a case on the transfer of personal data from Facebook Ireland to its parent company in the United States, in which the Court examined whether the standard data protection clauses[2] and the EU-US Privacy Shield ensure the level of protection the GDPR requires; it upheld the former and invalidated the latter.

[2]Standard data protection clauses are model contract clauses adopted by the Commission (Article 46(2)(c) GDPR): an exporter that agrees them with the recipient may transfer personal data to a third country for which there is no adequacy decision.

The reference in Table 7.13 is drawn from paragraphs 139 to 143 of Schrems II (C-311/18). ThemisRAG retrieves paragraphs 141, 142 and 143, a contiguous block of the section that answers the seventh and eleventh questions, together with point 3 of the operative part and Article 46 of the GDPR, which that point cites (Figure 7.10). Its answer follows the Court's reasoning step by step, from the verification prior to any transfer (paragraph 141) to the return or destruction of the data and the right to compensation (paragraph 143), cites the paragraph each step comes from, and states all five steps of the reference.

Table 7.13: Example 4, C-311/18 (Schrems II): the reference answer and the answer of ThemisRAG.

Question. What must the data exporter and the recipient do under the standard data protection clauses when the law of the third country prevents compliance with them?

Reference	**ThemisRAG**
"The exporter and the recipient must verify, prior to any transfer, whether the legislation of the third country allows the recipient to comply with the clauses[1]. Under Clause 5(a) and (b), the recipient must promptly inform the exporter of any inability to comply[2], including changes in its legislation, and the exporter is then bound to suspend the transfer and/or terminate the contract[3]. Under Clause 12, data already transferred and its copies must be returned or destroyed[4], and under Clause 6 a breach of the clauses gives the data subject a right to compensation[5]."	"[...] The data exporter (controller established in the EU) and the recipient of personal data in the third country are required to verify whether the legislation of the third country allows for compliance with the standard data protection clauses before any transfer of personal data occurs[1] [...] (para. 141). If the recipient of the personal data notifies the data exporter that the legislation of the third country does not permit compliance with the standard data protection clauses[2], the data exporter must take action. Specifically, the data that has already been transferred must be returned or destroyed in its entirety[4] (para. 143). [...] the data exporter is obliged to suspend the transfer of data and/or terminate the contract with the recipient[3] [...] (para. 142). Breach of the standard data protection clauses results in the right for the data subjects to receive compensation for any damage suffered[5] [...] (para. 143)."

The baselines answer the same question with less of the reasoning (Table 7.14). HippoRAG, the closest, states the verification, the notification and the suspension, but neither the return of the data nor compensation. LightRAG starts from the notification and omits both ends of the chain, the verification before the transfer and the compensation after a breach; among its references it lists the Data Act, which has no bearing on the question. PathRAG states the notification and the return of the data, yet RAGAS scores its factual recall at 0.00.

Table 7.14: Example 4: the baseline answers, highlighted with the colours of Table 7.13, and the scores of every system on this question.

System	Answer (excerpt)	Expert answer covered	Claims supported
ThemisRAG	see Table 7.13	**0.60**	**0.80**
HippoRAG	"The recipient must inform the data exporter promptly if it cannot comply with its obligations under the SCCs due to the applicable legislation in the third country[2] (Clause 5(a)). [...] Upon being notified by the recipient of its inability to comply, the data exporter is entitled to suspend the transfer of data and/or terminate the contract[3] (Clause 5(a)). The data exporter must ensure that the transfer of personal data is compliant with the applicable data protection law, which includes verifying that the legislation of the third country allows for compliance with the SCCs[1] (Clause 4(a) and (b))."	0.57	0.40
LightRAG	"The recipient of the personal data in the third country must inform the data exporter promptly about any inability to comply with its obligations under the SDPC[2]. [...] Upon being notified of the recipient's inability to comply, the data exporter is required to suspend the transfer of personal data to that third country[3]. [...] the data exporter must ensure that any data already transferred to the third country, and any copies thereof, are returned or destroyed in their entirety[4]." References given in the answer: C-311/18, Data Act	0.17	0.33
PathRAG	"[...] if the law in a third country would prevent the data importer from adhering to these clauses, the data exporter has to assess the situation thoroughly(1). [...] the data exporter must promptly alert individuals whose data is being transferred or processed and possibly halt the transfer and processing of data until suitable guarantees are secured(3). [...] If at any point the data importer realizes it cannot meet its obligations due to local laws, it is required to promptly notify the data exporter about this inability to comply[2]. [...] the data importer should cease processing the personal data or return or delete the personal data[4] as dictated by the data exporter's instructions."	0.00	0.00

Expert answer covered: factual recall. Claims supported: factual precision.

Table 7.15: Example 4: the steps of the reference and the answers that state them (✓ stated, (✓) stated in part, – missing). For ThemisRAG, in brackets, the paragraph of C-311/18 the step is drawn from.

Step of the reference	ThemisRAG	LightRAG	PathRAG	HippoRAG
1 Verify the third-country law before any transfer	✓ (141)	–	(✓)	✓
2 The recipient informs the exporter	✓ (143)	✓	✓	✓
3 The exporter suspends the transfer or terminates the contract	✓ (142)	✓	(✓)	✓
4 Data already transferred are returned or destroyed	✓ (143)	✓	✓	–
5 A breach gives the data subject a right to compensation	✓ (143)	–	–	–

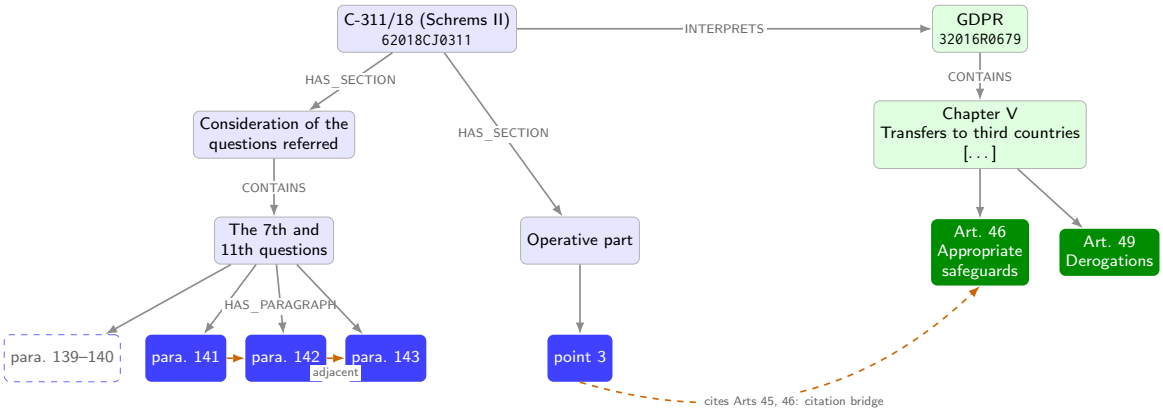


Figure 7.10: Example 4. A contiguous block of the section that answers the seventh and eleventh questions, the point of the operative part, and the article that point cites. Among its `INTERPRETS` edges, C-311/18 points to Articles 45 and 46 of the GDPR. Article 49 is retrieved by the act branch.

7.2.3 Error Analysis

We now analyse some examples in which ThemisRAG did not perform well. Only one of them reflects a recognised pattern: on the GDPR, questions whose reference goes beyond the text of the act recur, and they explain most of the gap observed in Figure 7.6. The other two are isolated cases: we found no recurrence of questions failing in the same way, and we report them because they show how the system can fail. Examples 1 and 2 mark the limits of the answers to RQ2.1 and RQ2.2, Example 3 that of the answer to RQ2.3.

Example 1: a question routed to the wrong act. Routing is the source of the largest gain of ThemisRAG (Figure 7.4), and it rarely fails: 3 of the 52 legislation questions are routed to the wrong act, and only on one of them does ThemisRAG end below the baselines.

The question in Table 7.16 refers to Article 28 of the GDPR. In this case the classification does not recognise the context of the question, and ranks the Data Act above the GDPR, probably also because the question makes no explicit reference to the GDPR. This carries over to the step that selects the passages governing the answer, which finds none: the context holds Articles 2, 3, 33 and 36 of the Data Act, and none of them concerns sub-processors. ThemisRAG therefore states that no retrieved passage governs the question, rather than building an answer from the wrong act, and states none of the steps of the reference.

Table 7.16: Example 1, GDPR: the reference answer and the answer of ThemisRAG.

Question. What are the requirements for engaging a sub-processor?

Reference	**ThemisRAG**
"Sub processing requires prior specific or general written authorisation from the controller[1]. Under general authorisation models, the processor must inform the controller of intended changes, allowing the controller to object[2]. Contracts with sub processors must impose the same data protection obligations as between controller and processor[3], in particular providing sufficient guarantees to implement appropriate technical and organisational measures. [...] Onward transfer mechanisms must comply with Chapter V requirements [...]"	"The question at hand is about the requirements for engaging a sub-processor under the Data Act. [...] The relevant provisions from the Data Act primarily focus on the essential requirements regarding smart contracts for executing data sharing agreements (Article 36) and the essential requirements regarding interoperability of data and data sharing mechanisms (Article 33). [...] I must conclude that there is no governing provision in the retrieved material that addresses this question."

Table 7.17: Example 1: the answer of HippoRAG, highlighted with the colours of Table 7.16, and the scores of every system on this question.

System	Answer (excerpt)	Expert answer covered	Claims supported
ThemisRAG	see Table 7.16	0.23	0.50
HippoRAG	"According to Article 28(2), the processor must obtain prior specific or general written authorization from the controller before engaging another processor[1]. If general written authorization is given, the processor must inform the controller of any intended changes concerning the addition or replacement of other processors, allowing the controller the opportunity to object to such changes[2]. [...] This contract must impose the same data protection obligations on the sub-processor as those set out in the contract between the controller and the initial processor[3]. [...]"	0.40	0.58
LightRAG	states the same three steps, without citing Article 28	0.37	**1.00**
PathRAG	a general account of contracts, prior approval and audit rights, including onward transfers	**0.54**	0.72

Expert answer covered: factual recall. Claims supported: factual precision.

Example 2: a reference beyond the text of the act. This is the only failure we observed repeatedly, and it explains why ThemisRAG loses its advantage on the GDPR (Figure 7.6). Several GDPR questions ask how a principle is applied in practice, and their reference answers with practitioner guidance rather than with the content of the acts, and since ThemisRAG retrieves only from the acts and from the judgments that interpret them, have no passage to retrieve, and its context recall falls.

Table 7.18 lists the seven GDPR questions on which this happens, those with a context recall of 0.3 or less. They are where PathRAG overtakes ThemisRAG: on these seven its factual recall is 0.52 against 0.37, while on the other eleven the two systems are close (0.61 against 0.57). PathRAG does not owe this to a better context, since LightRAG retrieves as much of the reference and recovers no more than ThemisRAG. It owes it to the way it answers, at length and in the register of general guidance, which is the register of these references.

Table 7.18: The GDPR questions on which the context recall of ThemisRAG is 0.3 or less, with the factual recall of every system. Last row: the other eleven GDPR questions routed correctly.

Question (abridged)	Context recall ThemisRAG	Factual recall			
		ThemisRAG	LightRAG	PathRAG	HippoRAG
Transfer Impact Assessment	0.00	0.29	0.00	**0.65**	0.00
Controller's obligations on accuracy	0.11	0.16	0.18	**0.36**	0.21
Privacy notice, data collected directly	0.20	**0.73**	0.50	0.68	0.27
Storage limitation, retention schedules	0.20	0.37	0.60	**0.75**	0.10
Security of processing (Art. 32)	0.25	**0.40**	0.35	0.36	0.29
Data minimisation in practice	0.27	0.38	0.41	**0.48**	0.24
Data protection by design and by default	0.29	0.28	0.35	**0.39**	0.19
Mean of the seven	0.19	0.373	0.341	**0.524**	0.186
Other GDPR questions (11)	0.69	0.569	0.520	**0.609**	0.409

The question in Table 7.19 is the clearest case. The Transfer Impact Assessment(TIA) is not in the GDPR: it comes from the practice that followed Schrems II, and the reference follows the roadmap of the European Data Protection Board.[3] ThemisRAG retrieves Article 35, on the data protection impact assessment, and Article 47, on binding corporate rules, but not Article 46, which lists the transfer tools the assessment tests (Figure 7.11). The answer acknowledges the gap, stating that the passages "do not explicitly mention a TIA", and builds a TIA by analogy with the DPIA. Only two steps of the reference survive, in part, and the third country, the object of the assessment, is never mentioned.

[3]EDPB, Recommendations 01/2020 on measures that supplement transfer tools to ensure compliance with the EU level of protection of personal data, version 2.0, adopted on 18 June 2021.

Table 7.19: Example 2, GDPR: the reference answer and the answer of ThemisRAG.

Question. What is a Transfer Impact Assessment (TIA) and what should it contain?

Reference	**ThemisRAG**
"A TIA evaluates whether, for a given transfer of personal data to a third country on the basis of the SCCs or other safeguards, the laws and practices in the destination country may impinge on the effectiveness of the transfer tool[1]. The TIA should document: the transfer details[2] [...]; the transfer tool used[2]; an assessment of the third country's legal landscape, particularly with respect to government access to data and redress[3]; the likelihood of problematic access to personal data in practice[3]; technical and organisational measures[4] (e.g., strong encryption with key control in the EEA [...]); and conclusions and residual risks[5], including whether to proceed [...]. Evidence[5] [...] should be maintained and the TIA should be re assessed periodically[5] [...]"	"[...] While the retrieved passages do not explicitly mention a TIA, they do provide relevant information regarding data protection impact assessments (DPIAs), which are closely related [...] A DPIA must contain at least the following elements (Article 35(7)) [...] While the GDPR does not explicitly define a Transfer Impact Assessment (TIA), the principles and requirements outlined for Data Protection Impact Assessments (DPIAs) in Article 35 can be applied to the context of data transfers [...] In summary, a TIA should include a systematic description of the transfer[(2)], an assessment of necessity and proportionality, a risk assessment, and proposed measures to mitigate those risks[(4)] [...]"

LightRAG and HippoRAG decline to answer (Table 7.20): their contexts hold 12 and 5 chunks, mostly from the AI Act. PathRAG answers and covers four of the five steps at least in part, and its answer is largely grounded. Its context contains paragraphs 129 to 136 of Schrems II, where the Court states that the exporter must verify, case by case, whether the law of the third country ensures adequate protection, adopt supplementary measures where needed, and otherwise suspend the transfer: the legal basis of the TIA, although the judgment never names it. The context of ThemisRAG contains neither these paragraphs nor Article 46.

Table 7.20: Example 2: the baseline answers, highlighted with the colours of Table 7.19, and the scores of every system on this question.

System	Answer (excerpt)	Expert answer covered	Claims supported
ThemisRAG	see Table 7.19	0.29	0.19
PathRAG	"A Transfer Impact Assessment (TIA) is a critical evaluation process undertaken to analyze the implications of transferring personal data to jurisdictions outside the European Union(1) [...] Assessing the Legal Framework: Organizations need to examine the legal environment of the third country to which data is being transferred(3). [...] Mitigation Measures: [...] establishing additional safeguards, such as data encryption, confidentiality agreements4 [...] Transparency and Documentation: The TIA process necessitates comprehensive documentation showing how the assessment was conducted(5) [...]"	**0.65**	**0.56**
LightRAG	"I do not have enough information to answer your query about a Transfer Impact Assessment (TIA)." Sources: 12 chunks, 10 AI Act, 2 GDPR	0.00	0.00
HippoRAG	"The passages do not cover the concept of a Transfer Impact Assessment (TIA) or specify what it should contain. [...]" Sources: 5 passages, 4 AI Act, 1 GDPR	0.00	0.00

Expert answer covered: factual recall. Claims supported: factual precision.

Table 7.21: Example 2: the steps of the reference and the answers that state them (✓ stated, (✓) stated in part, – missing).

Step of the reference	ThemisRAG	LightRAG	PathRAG	HippoRAG
1 Whether third-country law impinges on the transfer tool	–	–	(✓)	–
2 Describe the transfer and the tool used	(✓)	–	–	–
3 Assess the third country's law and practice	–	–	(✓)	–
4 Technical and organisational measures	(✓)	–	✓	–
5 Conclusions, evidence and periodic reassessment	–	–	(✓)	–

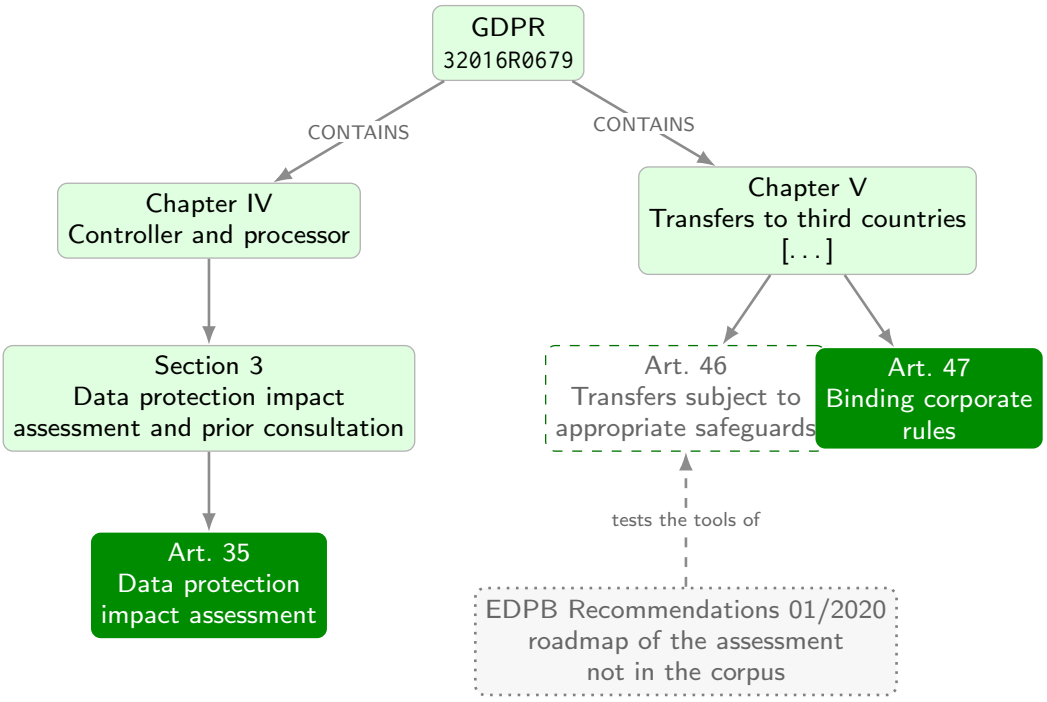


Figure 7.11: Example 2. The two articles ThemisRAG retrieves, Article 46, which lists the transfer tools a TIA assesses, and the guidance the reference follows, which is not part of the corpus (dotted).

Example 3: the paragraph that asks, not the one that answers. The question in Table 7.22 asks whether the protection required for transfers to third countries is measured against the ECHR or against the national constitutions. The Court answers "neither" in paragraphs 98 to 101 of Schrems II (C-311/18): the standard is the GDPR, read in the light of the Charter.[4]

ThemisRAG retrieves paragraph 97 instead (Figure 7.12). It is the paragraph in which the Court restates the question, and it mentions the Charter, the ECHR and national law together, so it matches the question better than any other paragraph. The four paragraphs that follow, which contain the answer, are not retrieved.[5] The generator takes the question for the answer: it lists the ECHR among the standards of protection and concludes that it "plays a significant role".

[4]The ECHR is not part of EU law as long as the Union has not acceded to it, and EU law cannot be assessed against the law of a Member State, even its constitution.

[5]The retriever also considers the paragraph before and the two after each one it retrieves (here 96, 98 and 99), but it scores them against the question like any other candidate, and here they score too low to be kept (Section 5.4.2).

Table 7.22: Example 3, C-311/18 (Schrems II): the reference answer and the answer of ThemisRAG.

Question. Is the level of protection required for transfers to third countries determined in the light of the ECHR or of the national constitutions of the Member States?

Reference	**ThemisRAG**
"Neither[1]. Although the ECHR rights are general principles of EU law, the ECHR is not a legal instrument formally incorporated into EU law as long as the Union has not acceded to it[2], and the interpretation and validity of EU law cannot be assessed in the light of national law, even of constitutional status[3]. The level of protection required by Article 46(1) GDPR must be determined on the basis of the GDPR, read in the light of the fundamental rights enshrined in the Charter[4]."	"[...] This assessment must be conducted in light of EU law, particularly ~~the rights guaranteed by the Charter of Fundamental Rights of the European Union and the fundamental rights enshrined in the ECHR~~×. [...] In summary, the level of protection required for transfers of personal data to third countries is determined primarily in light of EU law(4), ~~specifically the Charter of Fundamental Rights and the ECHR~~×, rather than the national constitutions of the Member States(3). [...] Thus, ~~the ECHR plays a significant role in this assessment~~×, while national constitutions are not the primary reference point [...]"

HippoRAG is the only system that answers correctly: the level of protection is determined "in light of the GDPR and the Charter [...] rather than the [ECHR] or the national constitutions" (Table 7.23). RAGAS, however, ranks LightRAG first (0.75), although it opens with the same error as ThemisRAG: LightRAG states most of the reasoning of the Court, and the metric counts the claims it shares with the reference, not whether the conclusion is right. The case also shows a limit of context recall, which scores the context of ThemisRAG at 1.00: the metric finds the terms of the reference in a paragraph that only asks the question. We observed the same configuration on one other question, in C-245/20, where ThemisRAG retrieves paragraph 12, which introduces the reply of the Dutch supervisory authority, and misses paragraphs 13 and 14, which report it; two questions are too few to call it a pattern.

Table 7.23: Example 3: the baseline answers, highlighted with the colours of Table 7.22, and the scores of every system on this question.

System	Answer (excerpt)	Expert answer covered	Claims supported
ThemisRAG	see Table 7.22	0.29	0.11
HippoRAG	"The level of protection required for transfers of personal data to third countries is determined primarily in light of the GDPR and the Charter of Fundamental Rights of the European Union[4], rather than the European Convention on Human Rights (ECHR) or the national constitutions of the Member States[1]. [...] Thus, while the ECHR may inform the context, the primary framework for assessing adequacy is rooted in EU law."	0.37	0.37
LightRAG	"~~The level of protection required for transfers of personal data to third countries is determined primarily in the light of the European Convention~~× [...] (ECHR) and the rights outlined in the Charter [...] particularly in Article 46(1), the fundamental rights guaranteed by the Charter must be considered(4). [...] while the ECHR serves as a reference for general principles of EU law(2), the interpretation of EU law and the legality of its legislation must not be based solely on national law, including national constitutions of the Member States(3) [...]"	**0.75**	**0.67**
PathRAG	"The level of protection required for transfers of personal data to third countries is primarily determined by the General Data Protection Regulation (GDPR)(4), ~~specifically considering the protections outlined in the European Convention on Human Rights (ECHR)~~×. [...] they remain secondary to the EU's unified approach(3) [...]"	0.50	0.36

Expert answer covered: factual recall. Claims supported: factual precision.

Table 7.24: Example 3: the steps of the reference and the answers that state them (✓ stated, (✓) stated in part, – missing, × contradicted).

Step of the reference	ThemisRAG	LightRAG	PathRAG	HippoRAG
1 Neither the ECHR nor national constitutions	×	×	×	✓
2 The ECHR is not formally part of EU law	–	(✓)	–	–
3 EU law cannot be assessed against national law	(✓)	(✓)	(✓)	–
4 The GDPR read in the light of the Charter, Art. 46(1)	(✓)	(✓)	(✓)	✓

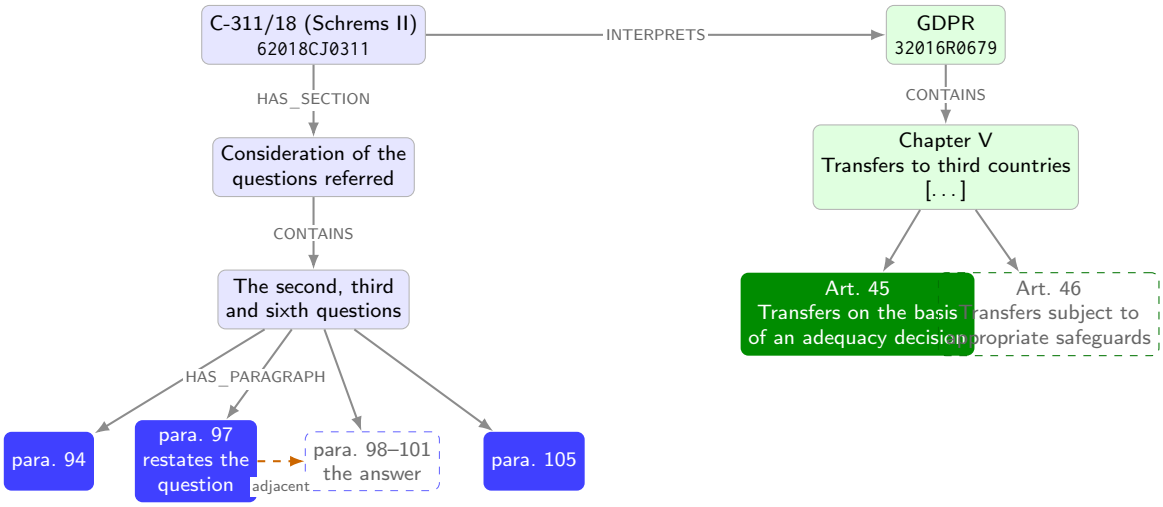


Figure 7.12: Example 3. ThemisRAG retrieves the paragraph in which the Court restates the question, but not the four that follow it and answer it, nor Article 46, whose paragraph 1 the answer interprets.

Chapter 8

Conclusions

This final chapter provides an analysis of the main findings, the contributions to the literature, the limitations encountered, and an overview of possible directions for future research.

8.1 Main Findings

- **A legal graph can be built by rules** (RQ1). The graph follows the structure of the sources instead of extracting it with a language model. The system built on this graph obtains the highest context precision and the best correctness scores, on average, of the four systems on both datasets. This is possible when the sources come from a structured, reliable repository. When they do not, a hybrid approach is the natural alternative: rules where the structure is available and a language model for the rest.
- **Intent and act routing** (RQ2.1). ThemisRAG rarely retrieves text from other acts (3.6%, against 26.8% to 50.7% for the baselines) and gives the most complete answers on legislation, except on the GDPR.
- **A small context may be enough** (RQ2.2). ThemisRAG answers from 3.5 to 5.3 passages and misses part of what the reference needs, yet its answers are the most complete: what it leaves out is mostly noise.
- **Links to the provisions make case law answers more precise** (RQ2.3). On judgments ThemisRAG makes fewer claims the reference does not support than LightRAG and PathRAG. The reason, in our view, is simple: the paragraphs of the judgment and the provision they interpret reach the generator together, so it has less reason to fill gaps with general statements. The comparison with HippoRAG, where the difference is not significant, points the same way: HippoRAG also answers from a small and clean context, which suggests that what matters most is a context limited to the relevant

passages, and that the links of the graph are one effective way of building it rather than the only one.

8.1.1 Contributions

A knowledge graph of EU legislation and CJEU case law built by rules. The structure of acts and judgments is read from the EUR-Lex markup rather than inferred by a language model, judgments are linked to the provisions they interpret, and every write is checked for conservation, structure and determinism before it reaches the database.

ThemisRAG. A retrieval architecture that routes each question to the acts it concerns, places each retrieved passage in its context in the graph, and extends a hybrid retrieval pipeline with mechanisms aimed at the ways retrieval over legal text fails.

A comparison with general-purpose graph-based RAG. An evaluation against LightRAG, PathRAG and HippoRAG on legislation and case law, with question-level statistics and an analysis of where the architecture succeeds and where it fails.

8.2 Limitations

The evaluation protocol. Only the legislation dataset was written by legal experts. The case law dataset was generated with a language model, imitating the questions and answers of the first, because no expert-written set of questions on judgments was available. Both datasets are small, 52 and 38 questions, so differences on a subset, such as the GDPR alone, rest on a few questions. All metrics are computed by a single LLM judge, whose verdicts on individual questions can diverge.

Coverage of the corpus. The evaluation covers only a small part of what the graph contains in terms of acts and case law. The results therefore describe the system on a narrow slice of EU data law, and how it behaves on the rest of the graph, on other areas of EU law or on national law has not been tested.

8.3 Future Work

Agentic steps. Agentic architectures, in which a language model decides step by step what to retrieve, have become a popular design for retrieval-augmented systems, and a legal knowledge graph could suit them well. An agent could work the way a lawyer does: start from the provision a question points to, follow it to the judgments that have interpreted it,

read on through the Court's reasoning until it reaches the holding, and step back to look elsewhere when what it has found does not answer the question. The graph would then be not only an index to search, but a body of law to explore.

Obligations. Much of what a legal professional asks of a regulation is not what a provision says but who must do what: which duties fall on a controller, which on a processor, which on a supervisory authority. This is the question behind compliance work, where an organisation needs to know the duties that apply to the role it plays, and behind much legal advice, where the first step is to establish who bears a duty and whether it is an obligation or a prohibition. A provision, however, states these duties in prose, often several in one paragraph, with the addressee named elsewhere and exceptions spread across other articles. Modelling obligations as nodes of the graph, each with its addressee, its modality and the passage it comes from, would make them queryable rather than only readable: the assistant could list the duties of an actor across an act, or across several acts, and trace each one back to the provision that imposes it.

Amendments and multilingualism. The models described in Sections 4.2.1 and 4.2.2, which the current graph supports only in its identifiers, would allow the assistant to answer about the law in force at a given date and in any official language. They have so far been designed but not built, and only an implementation would reveal their real implications and limits. Evaluating them would require questions written for the purpose, such as questions about the wording of a provision before and after an amendment, or the same question asked in different languages, and a comparison with the baselines, which would show how systems without a notion of time or language handle the same questions.

Bibliography

[1] Banko, M., Cafarella, M. J., Soderland, S., Broadhead, M., and Etzioni, O. (2007). Open information extraction from the web. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence*, pages 2670–2676.

[2] Bian, H. (2025). LLM-empowered knowledge graph construction: A survey.

[3] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

[4] Callan, J. P. (1994). Passage-level evidence in document retrieval. In *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '94)*, pages 302–310.

[5] Castano, S., Ferrara, A., Furiosi, E., Montanelli, S., Picascia, S., Riva, D., and Stefanetti, C. (2024). Enforcing legal information extraction through context-aware techniques: The ASKE approach. *Computer Law & Security Review*, 52:105903.

[6] Chen, B., Guo, Z., Yang, Z., Chen, Y., Chen, J., Liu, Z., Shi, C., and Yang, C. (2025). PathRAG: Pruning graph-based retrieval augmented generation with relational paths.

[7] Colombo, A., Bernasconi, A., Bellomarini, L., Guiso, L., Michelacci, C., and Ceri, S. (2025a). LegisSearch: navigating legislation with graphs and large language models. *Artificial Intelligence and Law*.

[8] Colombo, A., Bernasconi, A., and Ceri, S. (2025b). An LLM-assisted ETL pipeline to build a high-quality knowledge graph of the Italian legislation. *Information Processing & Management*, 62(4):104082.

[9] Cormack, G. V., Clarke, C. L. A., and Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759.

[10] d'Amato, C., Rubini, G., Didio, F., Francioso, D., Amara, F. Z., and Fanizzi, N. (2025). Automated creation of the legal knowledge graph addressing legislation on violence against women: Resource, methodology and lessons learned.

[11] de Martim, H. (2025). An ontology-driven graph RAG for legal norms: A structural, temporal, and deterministic approach.

[12] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186.

[13] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., and Larson, J. (2025). From local to global: A graph RAG approach to query-focused summarization.

[14] Es, S., James, J., Espinosa Anke, L., and Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158.

[15] Gao, L., Ma, X., Lin, J., and Callan, J. (2023). Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1762–1777.

[16] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2024). Retrieval-augmented generation for large language models: A survey.

[17] Guo, Z., Xia, L., Yu, Y., Ao, T., and Huang, C. (2025). LightRAG: Simple and fast retrieval-augmented generation.

[18] Hindi, M., Mohammed, L., Maaz, O., and Alwarafy, A. (2025). Enhancing the precision and interpretability of retrieval-augmented generation (RAG) in legal technology: A survey. *IEEE Access*, 13:46171–46189.

[19] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.

[20] Hofer, M., Obraczka, D., Saeedi, A., Köpcke, H., and Rahm, E. (2024). Construction of knowledge graphs: Current state and challenges. *Information*, 15(8):509.

[21] Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., Melo, G. d., Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., et al. (2021). Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37.

[22] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.

[23] Jiménez Gutiérrez, B., Shu, Y., Gu, Y., Yasunaga, M., and Su, Y. (2025). HippoRAG: Neurobiologically inspired long-term memory for large language models.

[24] Kalra, R., Wu, Z., Gulley, A., Hilliard, A., Guan, X., Koshiyama, A., and Treleaven, P. (2025). HyPA-RAG: A hybrid parameter adaptive retrieval-augmented generation system for AI legal and policy applications.

[25] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781.

[26] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

[27] LlamaIndex (2026). Retriever modules. `https://developers.llamaindex.ai/python/framework/module_guides/querying/retriever/retrievers/`. Accessed 2026-08-26.

[28] Ma, X., Gong, Y., He, P., Zhao, H., and Duan, N. (2023). Query rewriting for retrieval-augmented large language models. *arXiv preprint arXiv:2305.14283*.

[29] Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

[30] Nogueira, R. and Cho, K. (2019). Passage re-ranking with BERT.

[31] Page, L., Brin, S., Motwani, R., and Winograd, T. (1999). The PageRank citation ranking: Bringing order to the web. Technical report, Stanford InfoLab.

[32] Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., and Wu, X. (2024). Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599.

[33] Robertson, S. and Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

[34] Salton, G., Allan, J., and Buckley, C. (1993). Approaches to passage retrieval in full text information systems. In *Proceedings of the 16th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '93)*, pages 49–58.

[35] Salton, G. and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523.

[36] Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., and Manning, C. D. (2024). RAPTOR: Recursive abstractive processing for tree-organized retrieval.

[37] Tamašauskaitė, G. and Groth, P. (2022). Defining a knowledge graph development process through a systematic review. *ACM Transactions on Software Engineering and Methodology*, 32(1):1–40.

[38] Traag, V. A., Waltman, L., and van Eck, N. J. (2019). From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports*, 9:5233.

[39] Trklja, A. (2025). The textual organisation of CJEU judgments. *International Journal for the Semiotics of Law*, 38:783–810.

[40] Van Assche, D., Delva, T., Haesendonck, G., Heyvaert, P., De Meester, B., and Dimou, A. (2023). Declarative RDF graph generation from heterogeneous (semi-)structured data: A systematic literature review. *Journal of Web Semantics*, 75:100753.

[41] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

[42] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35.

[43] Xiao, Y., Dong, J., Zhou, C., Dong, S., Zhang, Q.-w., Yin, D., Sun, X., and Huang, X. (2025). GraphRAG-Bench: Challenging domain-specific reasoning for evaluating graph retrieval-augmented generation.

[44] Xue, B. and Zou, L. (2023). Knowledge graph quality management: A comprehensive survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(5):4969–4988.

[45] ZeroEntropy (2025). zerank-2: Model card. https://huggingface.co/zeroentropy/zerank-2. Accessed 2026-10-13.

[46] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36.

[47] Zhong, L., Wu, J., Li, Q., Peng, H., and Wu, X. (2024). A comprehensive survey on automatic knowledge graph construction. *ACM Computing Surveys*, 56(4):94.